Veridex One Platform — L3 v4.9 Physical ERD

Full P&C canonical model for a multi-tenant platform (carriers, MGAs, MGUs, wholesalers, brokers) | ACORD-aligned | Excludes Life & Health

NEW added table or column CHG changed from v2 DEL removed from v2 v3.1 changed in v3.1 v3.2 changed in v3.2 v3.3 changed in v3.3 v3.4 changed in v3.4 v3.5 changed in v3.5 v4.0 changed in v4.0 v4.1 changed in v4.1 v4.2 changed in v4.2 v4.3 changed in v4.3 v4.4 changed in v4.4 v4.5 changed in v4.5 v4.6 changed in v4.6 v4.7 changed in v4.7 v4.8 changed in v4.8 v4.9 changed in this revision (NEW 4.9 / CHG 4.9 in diagrams) CHECK explicit database or publish-time constraint shown on the module page | FK = foreign key inside the schema or to platform reference data · REF = cross-module reference, no foreign key, dashed line · EXT_REF = reference to an external system | Every business table also carries tenant_id, created_at/by, updated_at/by, row_version (not drawn). Empty boxes are defined in another module.

Every row has an owning tenant

All business tables carry `tenant_id` (not drawn). Enforced with Postgres row-level security and EF Core global query filters. Platform-global tables are the exception: `global_organization`, `jurisdiction`, `currency`, `line_of_business`, `classification_system`, `catastrophe_event`, `acord_semantic_map` and the Module 29 catalogue. Authorization catalogues (permissions, templates) are platform-wide documents in Cosmos.

Share by grant, never by copy

Another tenant sees a row only through `product_distribution_grant` (configuration) or `record_share` (transactions). Bulk carrier reporting flows through bordereaux.

Overrides store deltas

An MGU product version points at the carrier base version and stores only its changes, each authorised by an `override_permission`. Carrier revisions flow through automatically.

Risk is frozen per context

Submission, quote and policy versions each point at their own `risk_version`. Editing a quote never changes a bound policy. `lineage_key` tracks the same vehicle or building across versions.

Standard columns

Every table also has `created_at`, `created_by`, `updated_at`, `updated_by` and `row_version` for optimistic concurrency. Lengths are omitted here and fixed in the SQL phase.

Addresses are immutable v3.2

An `address` row is never edited. A move creates a new row and closes the old `party_address` link, so frozen risk versions and declarations keep the address they were issued with. `policy_party.mailing_address_id` freezes the declarations address per policy version.

OPA decides access, the model supplies facts v3.2

Rego policies live in Git and reach OPA as versioned bundles (`authorization_policy_version`). The database supplies the input data: memberships, roles, teams, scopes, shares and authority levels. Decision logs go to Azure Monitor, not this database. Underwriting, rating and charge rules stay in their own tables.

Every value has a source v3.3

Documents, broker APIs, users, third-party providers, telematics and AI all write `data_observation` rows with source, location and confidence. Conflicts become `data_discrepancy`; the trusted value is a `data_resolution`. Only resolved values flow into the risk version, rating and underwriting. One source-agnostic table, no per-provider copies.

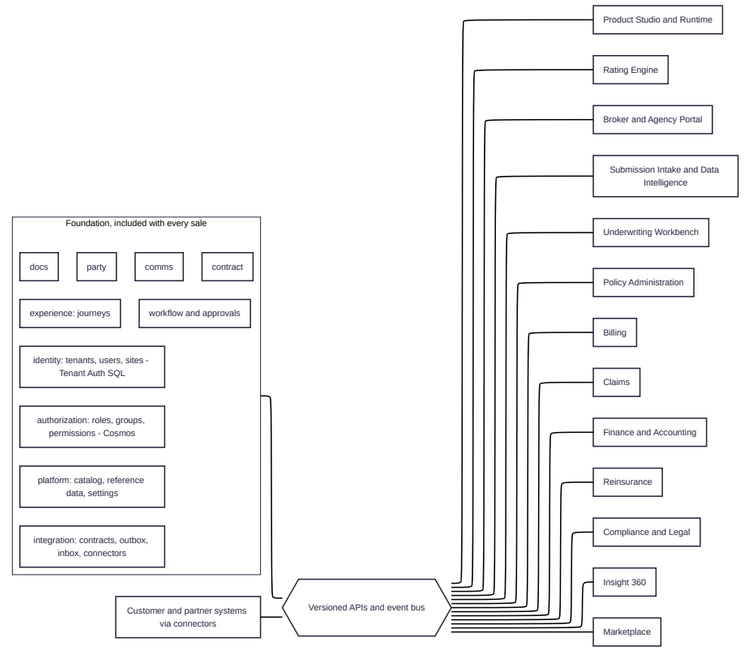
One current row, no overlaps v3.5

Wherever a fact has history, the database guarantees at most one current row (partial unique index on the fact key where `valid_to` is null), no overlapping validity periods for the same key, and a linear supersession chain (each row superseded at most once). `is_current` is derived from `valid_to`, never stored separately. Superseding closes the old row and opens the new one in one transaction.

Money and codes

Every amount is `numeric(19,4)` with a sibling `currency_code`. Status and type columns draw values from `code_value`. ACORD alignment lives in `acord_semantic_map`, not in columns.

How the modules connect



Modular architecture v4.9

Veridex One is sold as a full suite or module by module. Each sellable module owns its database schema; modules never hold foreign keys into another module's schema.

One system of record per concept

Tenant, user, role, group, permission, module, feature and entitlement each have exactly one owner (see the system-of-record page). Other stores hold projections or caches with a source version, never independent copies.

Physical stores

Tenant Auth SQL (identity), Authorization Cosmos (roles, groups, permissions, UI metadata), Platform SQL (Foundation and catalogue), Domain SQL (insurance and legal modules) and derived projections. Every module page shows its store; one ERD does not mean one database.

Each module owns its schema

Foreign keys only inside a schema and to platform reference data. Each schema has its own outbox (`business_event`) and inbox (`inbox_message`); delivery is at-least-once with idempotent consumers.

FK, REF and EXT_REF

FK: physical foreign key inside the owning schema or to replicated platform reference data (jurisdiction, currency, line of business, codes). **REF**: cross-module reference validated through an API or event, never a foreign key (dashed lines). **EXT_REF**: reference to a customer or partner system. For REF and EXT_REF the consumer stores source tenant, entity type and id, version, event position and last-updated time, plus a snapshot of the values it needs. Reconciliation reports orphans.

Any dependency can be external

`capability_binding` records, per customer, whether a capability (policy data, rating, payments, GL) comes from a Veridex module or the customer's system through a connector.

Configuration, not code

New lines arrive as product packs; behaviour is configured through layers, releases and journeys; extensions only through certified plugins at declared extension points. No customer-specific forks.

One runtime, one release

Every portal and module asks the Product Runtime for the published `product_release` for its scope, and records the release on what it creates.

Modules are not microservices

The 45 sections are domain-model areas, not 45 deployed services. Start with one PostgreSQL environment, separate schemas with clear ownership, a modular application or a few bounded services, and outbox/inbox from day one. Split a module physically only when standalone sale, security or scale requires it.

Planned modules are not built

Reinsurance, Portfolio & Accumulation, Claims servicing and advanced marketplace features keep their target schemas but are not implemented for the core launch.

JSON under governance

Configuration JSON has a versioned schema (`config_json_schema`), is validated before publishing, follows compatibility rules, is compiled into the immutable `product_release`, and is tested and simulated before activation. Transactional facts are never stored as JSON.

Governed AI

AI runs only through approved models and registered use cases (`ai_model`, `ai_use_case`); every execution is recorded with versioned prompt, inputs, outputs, confidence and cost, and reviewed by a person where the use case requires it.

Content responsibility

Veridex supplies technology and configurable templates. Carriers remain responsible for contractual language, rates, filings and regulatory approval, recorded in `content_attestation`.

Sellable module	Owns schemas	Requires	Can run with an external system for	Availability
Foundation (always included)	`identity, authorization, platform, experience, party, workflow, docs, comms, contract, integration, ai`	—	—	GA
Product Studio & Runtime	`product, distribution`	Foundation	—	GA
Rating Engine	`rating`	Foundation; rating plans from Product Studio	Callers may be Veridex modules or external systems via API	GA
Broker / Agency Portal (full agency management)	`agency, submission, certificate`	Foundation	Markets: Veridex tenants or external carriers and MGAs via connectors	GA
Submission Intake & Data Intelligence	`submission, risk, data`	Foundation; attributes and requirements from Product Studio	Underwriting: Veridex Workbench or customer system	GA
Underwriting Workbench	`underwriting (with submission, risk, data)`	Foundation, Product Studio	Rating and policy system: Veridex or external	GA
Policy Administration	`pas, certificate (with risk)`	Foundation, Product Studio, Rating	Billing and Claims: Veridex or external	GA
Billing	`billing`	Foundation	Policy transactions: Veridex PAS or external, held as snapshots	GA
Claims (core)	`claims`	Foundation	Policy and coverage: Veridex PAS or external, held as snapshots; payments to Veridex Finance or external	GA for core claims. Claims servicing (module 42) is planned and must not be sold or presented as available
Finance & Accounting	`finance`	Foundation	Financial events from Veridex modules or external systems	GA
Reinsurance	`reinsurance`	Foundation	Premium and loss transactions: Veridex or external	Planned, not for sale
Compliance	`compliance`	Foundation	Products and transactions: Veridex or external	GA
Legal Matter Management	`legal`	Foundation	Claims and contracts it relates to: Veridex or external; can run with no insurance module at all	GA
Insight 360 & Reporting	`insight, reporting`	Foundation	Events from any module or connector	GA; portfolio & accumulation planned, not for sale
Marketplace	`market`	Foundation, Product Studio	Handoff to Veridex submissions, agencies or external carriers	GA

System of record v4.9

Every identity, access and commercial concept has exactly one owner. Other stores hold projections or caches that carry a source version and are rebuilt from events, never edited independently.

Concept	System of record	Store	Projections and references	Replaces
Identity tenant (login boundary)	`identity_tenant`	Tenant Auth SQL	—	AuthService Tenant
Business tenant (workspace, data owner)	`tenant`	Tenant Auth SQL	tenant_id on every business row	v4.6 tenant + AuthService Organization
Person's login identity	`external_identity`	Tenant Auth SQL	—	v4.6 app_user identity columns
User in a tenant	`tenant_user`	Tenant Auth SQL	—	v4.6 app_user, tenant_membership; AuthService OrganizationUser
Sites and users in sites	`org_unit, tenant_user_org_unit`	Tenant Auth SQL	—	v4.6 org_unit (platform)
API clients, consent, delegation	`api_client, user_consent, access_delegation`	Tenant Auth SQL	—	v4.6 platform tables
Permission catalogue	`permission (+ permission_metadata)`	Authorization Cosmos	—	v4.6 permission; Cosmos PermissionCatalog
Roles and their permissions	`role (+ role_template)`	Authorization Cosmos	—	v4.6 role, role_permission; Cosmos Role
Groups and membership	`user_group, group_membership`	Authorization Cosmos	—	v4.6 team, team_member; Cosmos Group arrays; AuthService group membership
Role assignment	`subject_role_assignment`	Authorization Cosmos	—	v4.6 user_role; AuthService role assignment
Access scope (ABAC) and record shares	`access_scope_policy, record_share`	Authorization Cosmos	—	v4.6 data_access_scope, record_share
Personas	`persona (+ persona_template)`	Authorization Cosmos	tenant_user.persona_id	AuthService PersonaId
Authorization policy bundles	`authorization_policy_version`	Authorization Cosmos	—	v4.6 authz_policy_bundle
Modules and features	`module, module_feature`	Platform SQL (Module 29)	module_registration, feature_registration (authorization data only)	Cosmos Module, Feature
SKUs, prices, subscriptions	`sku, price_book_entry, tenant_subscription, subscription_item`	Platform SQL (Module 29)	license_projection	AuthService TenantLicense
Purchased entitlements	`tenant_module_entitlement, tenant_feature_entitlement`	Platform SQL (Module 29)	module and feature entitlement projections; entitlement_snapshot_cache	AuthService ModuleEntitlement, ModuleFeatureEntitlement
Tenant admin module and feature choices	`tenant_module_assignment, tenant_feature_state`	Tenant Auth SQL	—	AuthService OrganizationModuleAssignment, OrganizationFeatureState
Per-user module or feature access	`– (not modelled)`	—	—	AuthService OrganizationUserModuleAccess, OrganizationUserFeatureAccess: replaced by roles and scopes
UI screens and menus	`ui_form_definition, ui_menu_definition`	Authorization Cosmos	—	Cosmos FormDefinition, MenuDefinition
Localised text	`localization_entry`	Authorization Cosmos	—	v4.6 localization_string; Cosmos LocalizationEntry
Journeys and configuration schemas	`journey…, config_json_schema`	Platform SQL (Foundation)	—	v4.6 product schema
Legal entities (insured, carrier, MGA, broker)	`party, organization`	Platform SQL (Foundation)	tenant.party_id	—

1A. Identity & tenant

schema: identity **store: Tenant Auth SQL** Who can sign in and into which workspace: the identity tenant (Entra and login settings), the business tenant (the workspace and data-isolation boundary), each person's external identity and their membership in a tenant, sites, API clients, consent and delegation, plus read-only projections of what the tenant has licensed. System of record in the Tenant Auth SQL database (AuthService).

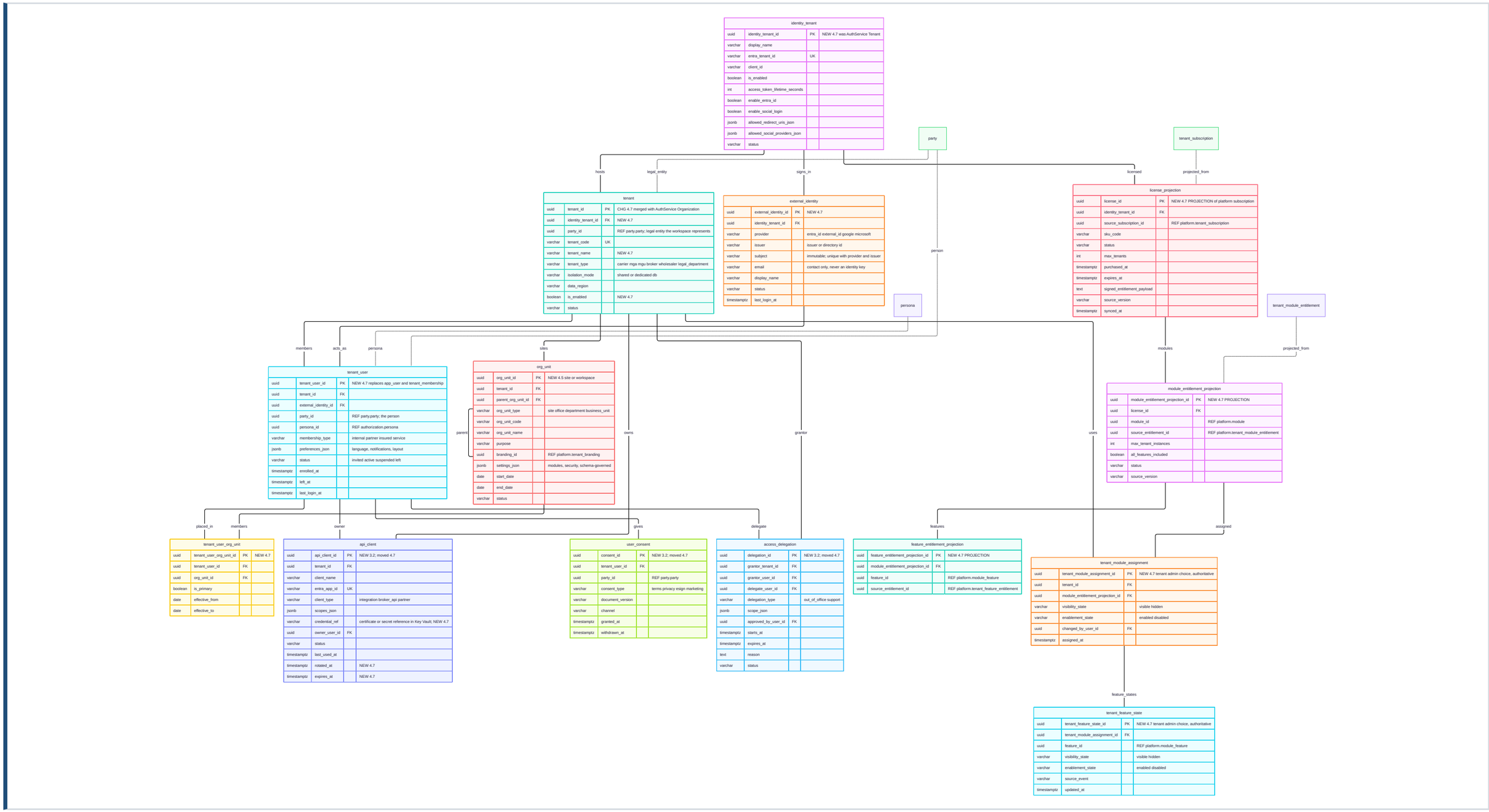
v4.7 Module rebuilt from the v4.6 Tenancy & access module and the AuthService tenant database. `identity_tenant` (was AuthService `Tenant`) is the login boundary; `tenant` (merged with AuthService `Organization`) is the business workspace and stays the owner of every business row through `tenant_id`.

v4.7 `license_projection`, `module_entitlement_projection` and `feature_entitlement_projection` are read-only projections of Module 29 for login and runtime gating. `tenant_module_assignment` and `tenant_feature_state` hold the tenant admin's own visibility and enablement choices and are authoritative here.

v4.7 `tenant_user` replaces `app_user` and `tenant_membership` (and AuthService `OrganizationUser`): a person's membership in one tenant, with persona, party, preferences and enrolment dates. `external_identity` holds the provider, issuer and immutable subject; email is never an identity key.

v4.7 Moved out: roles, permissions, groups, assignments, scopes, record shares and policy versions to Module 1B (Authorization); subscriptions and entitlements to Module 29; `uw_authority_level` to Module 8; `entity_tag` to Module 23. Not modelled: signing keys, issued tokens and authorization codes (process memory) and the social provider catalogue (static configuration). AuthService `OrganizationUserModuleAccess` and `OrganizationUserFeatureAccess` are not carried over: access comes only from roles and scopes.

v4.7 `org_unit`, `api_client`, `user_consent` and `access_delegation` move here; new `tenant_user_org_unit` places users in sites with dates.



1B. Authorization & UI metadata

schema: authorization **store: Authorization Cosmos** What a signed-in user may do and see: the permission catalogue, roles, personas and templates, groups and memberships, role assignments with dates and approval, attribute-based scope policies, record shares, versioned policy bundles, the authorization registration of modules and features, UI menus and screens, and localised text. System of record in the Authorization Cosmos DB; ids are stored references, no foreign keys are enforced.

VALIDATE `group_membership`: exactly one of `tenant_user_id` or `member_group_id`; `member_group_id <> group_id`; nested groups acyclic. Enforced by the Authorization service (Cosmos has no CHECK constraints) and rechecked by the nightly reconciliation.

v4.8 `group_membership`: exactly one of `tenant_user_id` or `member_group_id`; a group can never contain itself, directly or through nesting.

v4.7 `subject_role_assignment` assigns roles to users, groups or API clients with grantor, approval, dates and revocation (replaces `user_role` and AuthService role assignments). `access_scope_policy` (ABAC) replaces `data_access_scope`; `authorization_policy_version` replaces `authz_policy_bundle`; `record_share` moves here.

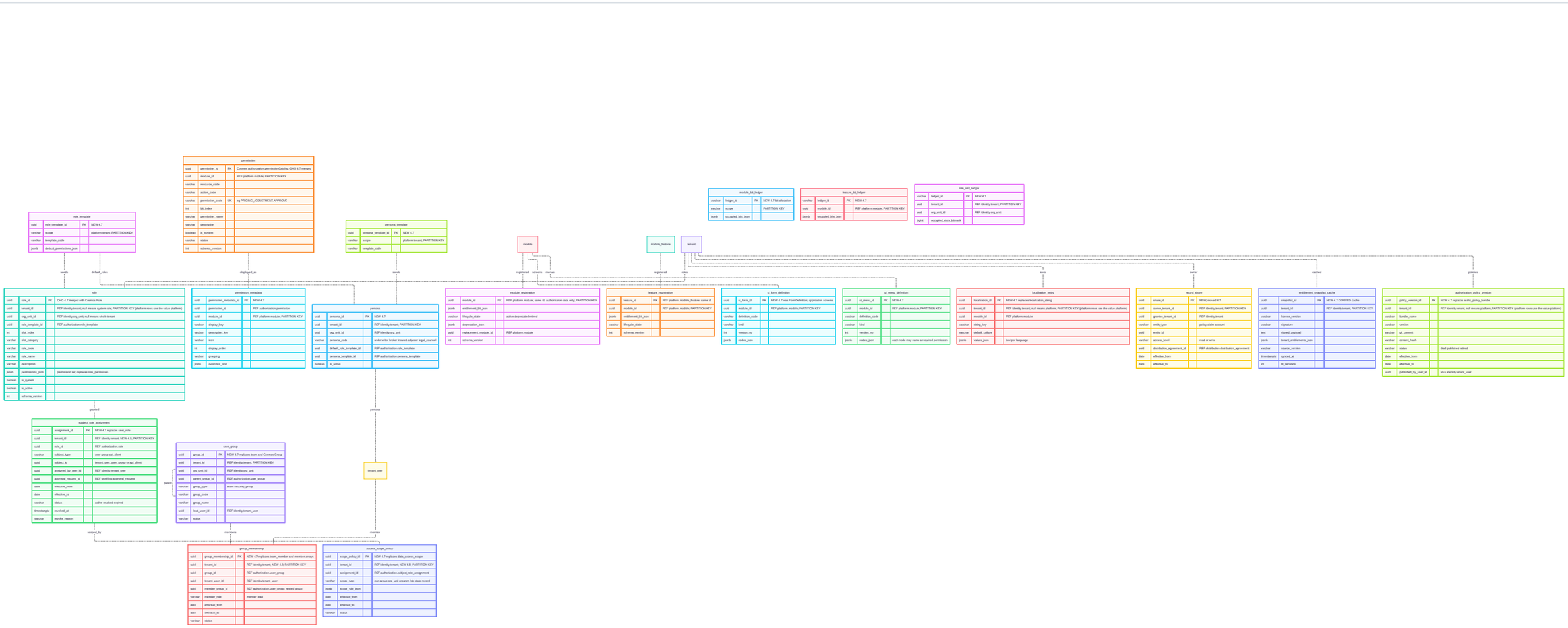
VALIDATE `subject_role_assignment`, `group_membership`, `access_scope_policy`: `tenant_id` equals the tenant of the role, group or assignment they reference.

v4.7 New module from the Authorization Cosmos DB. One permission catalogue (`permission` merges v4.6 `permission` with `PermissionCatalog`); one `role` per tenant and site carrying its permission set (replaces `role_permission`). Required records include `UW_ROLE.MANAGE`, `PRICING_ADJUSTMENT.CREATE` and `PRICING_ADJUSTMENT.APPROVE`.

v4.7 Modules and features keep their identity in Module 29; `module_registration` and `feature_registration` hold only authorization data under the same ids (entitlement bits, lifecycle, deprecation and replacement). Bit and slot ledgers are internal allocation documents. `ui_form_definition` (application screens, was `FormDefinition`) is distinct from journeys (business data capture) and from insurance forms. `localization_entry` replaces `localization_string`. `entitlement_snapshot_cache` is derived, with source version, signature and expiry.

v4.8 Every Cosmos entity names its **partition key** in the diagram: `tenant_id` for tenant data (platform rows use the value platform), `module_id` or `scope` for platform catalogues. `subject_role_assignment`, `group_membership` and `access_scope_policy` now carry `tenant_id` explicitly, because in Cosmos the partition key must be stored in every document.

v4.7 `user_group` (type team or security group) replaces `team` and Cosmos `Group`; `group_membership` replaces `team_member`, AuthService group memberships and member arrays. Work queues stay in Module 16 and point at groups.



2. Party, location & reference data

schema: party **schema: platform** **schema: rating** **store: Domain SQL** **store: Platform SQL** People and organisations, their identifiers, addresses and hierarchies, plus the shared reference tables every other module uses.

v4.8 `address.country_code` must agree with its jurisdiction: a composite foreign key `(jurisdiction_code, country_code)` to `jurisdiction`. `jurisdiction` can never be its own parent, and the hierarchy is acyclic.

v4.1 `party` gains `mdm_status` and `merged_into_party_id`; matching, merges, survivorship, aliases, groups, screening and party consent are in module 39.

NEW `address` existed only as a dangling FK in v2. Lat/long supports cat modelling and territory lookup.

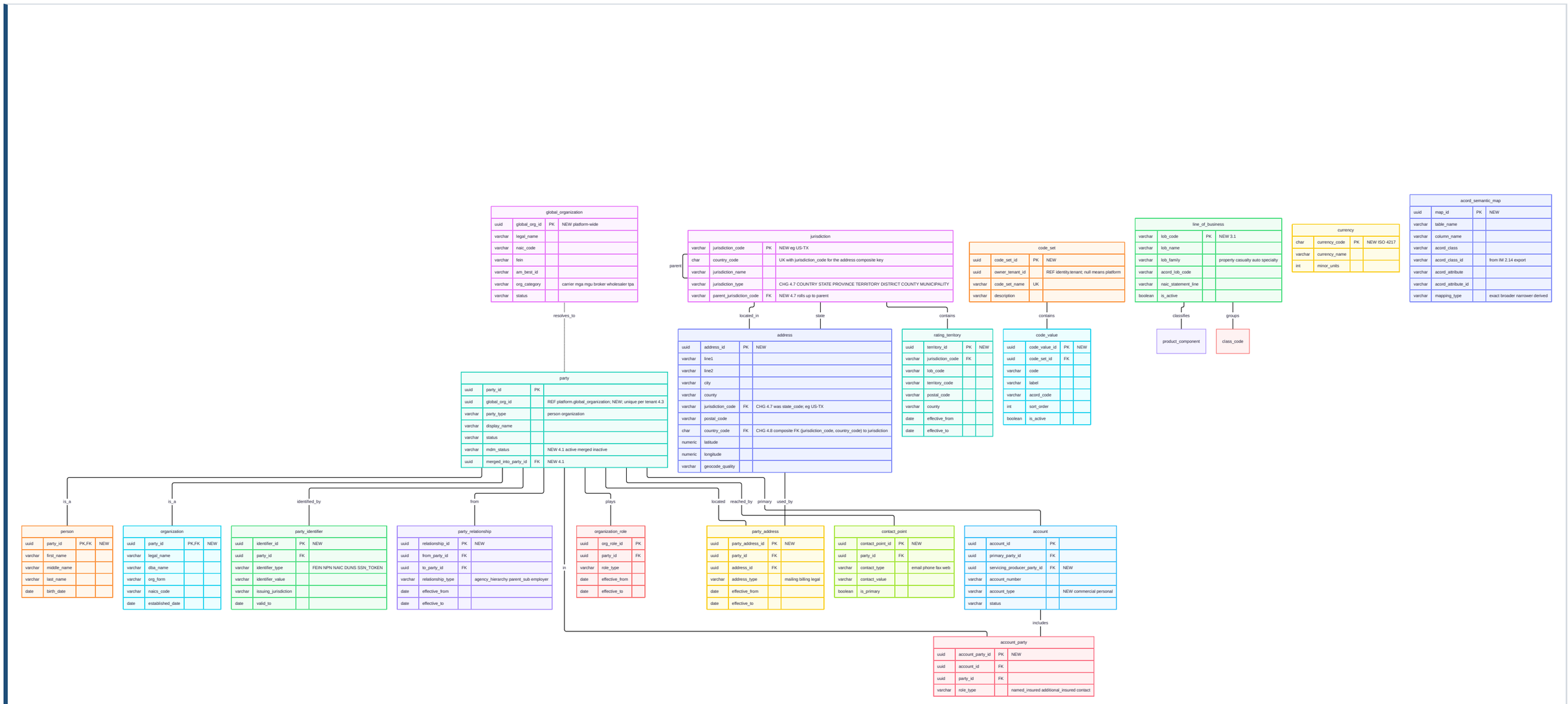
v4.7 `address.state_code` becomes `jurisdiction_code`, a foreign key to `jurisdiction` with canonical values such as US-TX or CA-ON. `jurisdiction` gains `parent_jurisdiction_code` (states roll up to countries) and a controlled `jurisdiction_type` (administrative level only).

v3.1 `line_of_business` becomes the single list of LOBs. Every remaining `lob_code` column in the model is now a foreign key to it, and it carries the ACORD LOB code and NAIC annual statement line for statutory reporting.

DEL `party.acord_class` and every other `acord_class` column, replaced by the mapping table.

v4.3 A tenant may hold only one `party` per `global_organization` (unique tenant_id + global_org_id), so market participants are never duplicated inside a tenant. Insureds are not shared across tenants.

NEW `global_organization`: one platform-wide record per carrier or MGA, so a carrier tenant and an MGA tenant refer to the same real company in agreements.



3. Distribution & delegated authority

schema: distribution **store: Domain SQL** How a carrier puts an MGA, MGU or wholesaler into its distribution channel, what that partner may do, and how programs draw capacity from one or more carriers.

v4.3 `authority_limit` is one row per limit: `limit_type` (premium, policy limit, TIV, claim reserve, claim settlement) and `limit_amount`, by line, state, class code and now coverage. Claim authority moves here from the single `binding_authority_agreement.claims_settle_limit`, so an MGA can settle cargo claims to 25,000 but liability claims only to 10,000.

v3.1 `program.lob_code` removed: a program can offer multi-line products, so its lines are derived from its products' components. `authority_limit.lob_code` stays as a filter and now references `line_of_business`.

CHG `program.carrier_party_id` replaced by `program_capacity`, which supports split capacity and fronting.

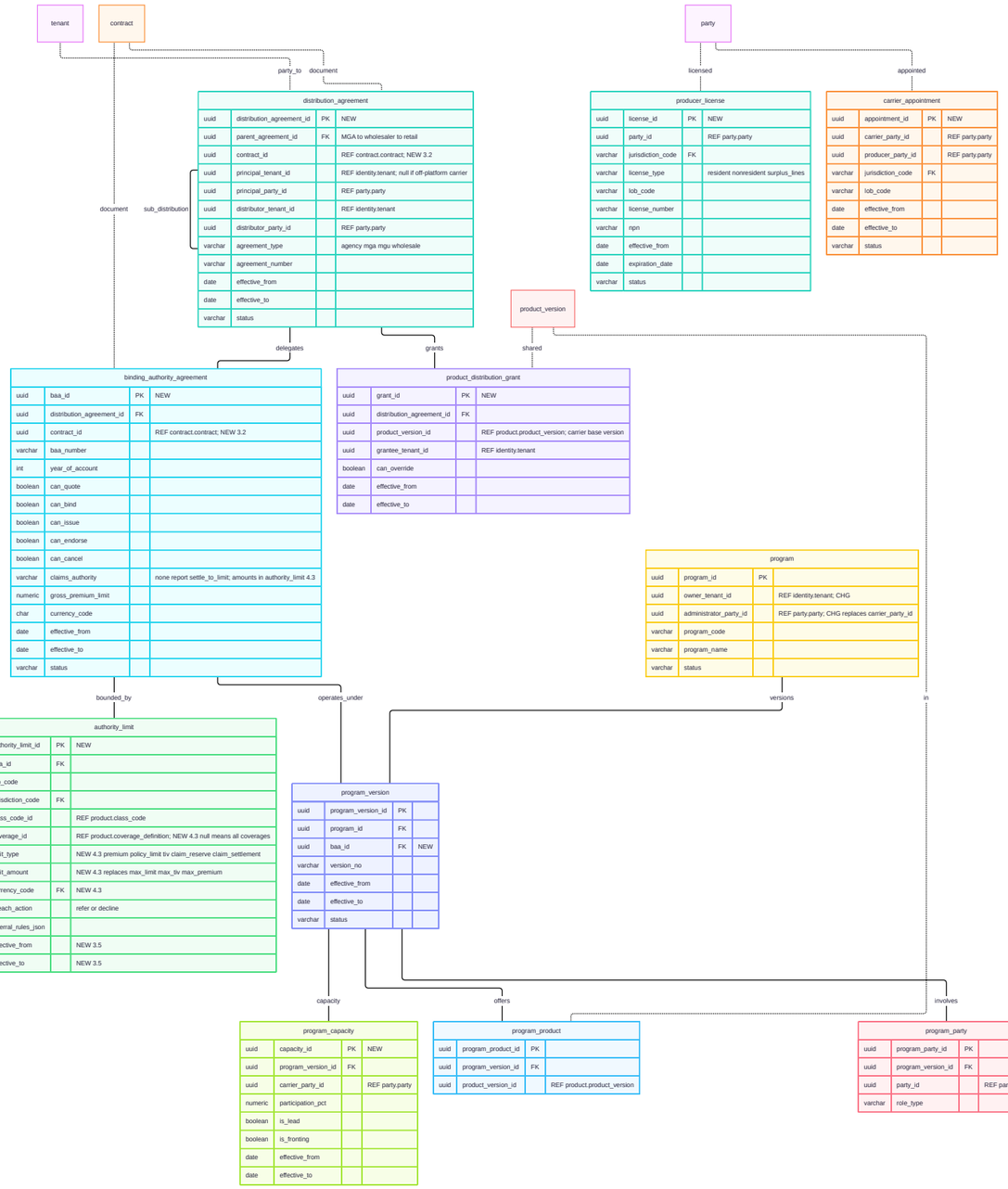
v3.5 `authority_limit` gains `effective_from` and `effective_to`, so a contract amendment that raises a limit adds a new row from its effective date while the old limit stays as history; no two limits for the same agreement, line, state and class may overlap.

NEW `distribution_agreement` is the "carrier configures MGA in its channel" record. `parent_agreement_id` chains MGA → wholesaler → retail broker.

NEW License and appointment checks run before any bind in a state.

v3.2 `distribution_agreement` and `binding_authority_agreement` point at a common `contract` record that holds parties, signatures, amendments and obligations (module 22).

NEW `binding_authority_agreement` + `authority_limit`: bind vs refer is decided against these, by LOB, state, class and size.



4. Product studio & override layering

schema: product · store: Domain SQL · Carriers build base products. MGUs create override versions that store only their changes, within limits the carrier set. All rating, rules, questions and forms hang off the product version.

PUBLISH `product_coverage_class`: its `class_code_id` must be eligible or refer in `product_component_class` for the coverage's own component, the same or a broader jurisdiction and an overlapping period.

UNIQUE `question_attribute_mapping`: one primary mapping per `(question_version_id, attribute_definition_id)` and period.

v4.7 `product_coverage_class`: class codes for one coverage. With no rows, a coverage inherits its line's `product_component_class` list; with rows, it can only narrow that list, never add a class the line prohibits.

v4.3 Forms and questions attach at coverage level: `product_form_rule` and `product_question_set` gain `product_component_id` and `product_coverage_id` (empty = whole product). A cargo endorsement attaches only when Cargo is selected; cargo questions are asked only then.

v3.4 `coverage_relationship_rule`: requires, allows, excludes, defaults-on and constrains rules between components and coverages (Physical Damage allows Collision and Comprehensive; UM/UIM limit at most the liability limit). Replaces `product_component.dependency_json`.

v3.3 `attribute_applicability`: which governed attributes each product, line and risk object type requires, at which stage, with defaults and stricter validation.

v3.1 `product.lob_code` replaced by `product_structure` (monoline or package).

NEW Override versions live under the MGU's own `product` row and point at the carrier version through `base_product_version_id`. Effective config = base + approved deltas.

NEW Explicit limit, deductible and term options per coverage, plus state availability and filing references.

EXCLUDE `product_coverage_class` and `product_component_class`: no overlapping periods for the same coverage (or component), class and jurisdiction; jurisdiction empty means all states.

v4.9 `question_attribute_mapping` connects each question version and its answer to the canonical `attribute_definition` that rating consumes, with the answer transform, unit conversion and the risk object type the value lands on. Answers become observations of that attribute; rating factors read the attribute, never the question.

v4.7 `premium_earning_rule`: how premium is earned on cancellation (pro rata, short rate, minimum earned, fully earned) by product, coverage and state, with the filing that approves it.

v4.3 `product_rating_plan` can target a single coverage; inside a plan, each coverage is rated by its own `coverage_rating_algorithm` (module 8).

v3.4 `attribute_applicability` gains freshness rules: `max_age_days`, `refresh_at_stage` and `stale_action` (warn, refresh, block), per attribute and product.

v3.3 `submission_requirement`: what makes a submission complete for this product (forms, documents, data, third-party reports, signatures). Evaluated in module 7; separate from underwriting rules.

v3.1 `override_permission` can target a whole component, and `requires_filing` marks deviations the carrier must file before the override version can go live.

NEW `override_permission` implements v2's promised "controlled overrides": the carrier decides what can change and by how much.

PUBLISH `rating_class_code` must exist in the plan's RATING_CLASS dimension allowed values.

v4.9 `rating_class_code` on `product_component_class` and `product_coverage_class` is a rating-plan-local code, governed: it must be one of the allowed values of the rating plan's RATING_CLASS dimension (`rate_table_dimension`).

v4.4 Scope integrity for `product_rating_plan`, `product_question_set` and `product_form_rule`: a coverage requires a component; the coverage must belong to that component and the component to the same product version (composite foreign keys on (component, version) and (coverage, component)); no duplicate assignment for the same scope, state and stage; effective periods never overlap. Question sets and form rules gain effective dates.

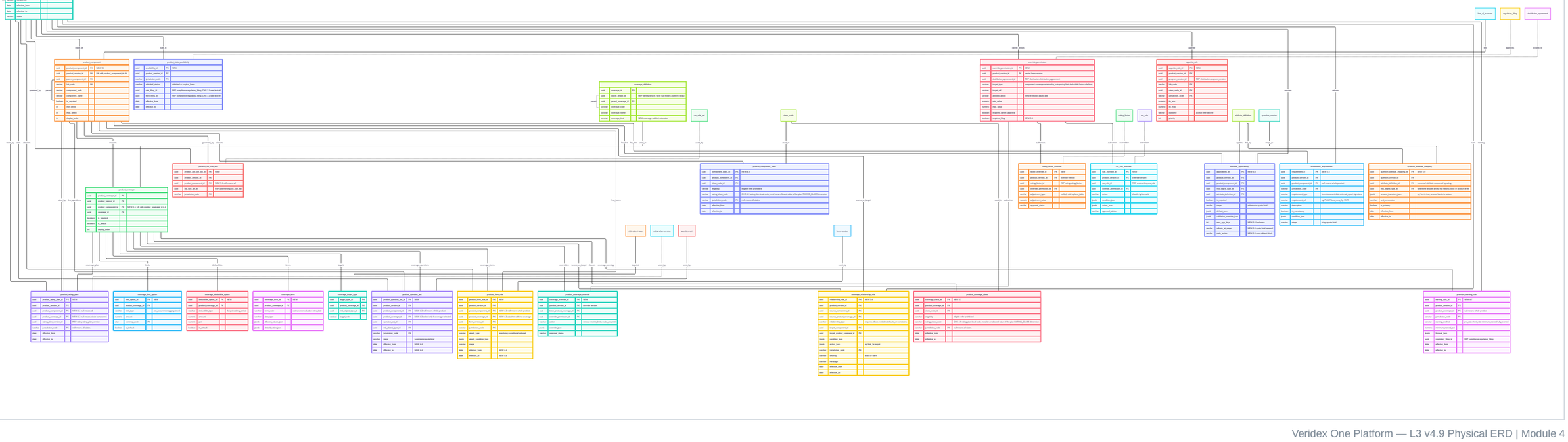
v4.3 `product_component_class`: the class codes each line accepts, with eligibility (eligible, refer, prohibited), the rating class, state and dates. One auto liability line can cover trucks and cars.

v3.4 `override_permission` can target relationship rules and pricing adjustments.

v3.1 `product_component`: a product version is made of components, each tied to one LOB (Property, GL, Crime, Inland Marine; or Auto Liability, Physical Damage, Motor Truck Cargo). Coverages, and optionally rating plans and rule sets, attach to a component.

v3.1 `product_state_availability` now points at real `regulatory_filing` rows instead of free-text references.

NEW Link tables to rating plans, rule sets, question sets and forms. In v2 none of these were attached to a product.


product
product_version
product_component
product_state_availability
coverage_definition
mv_rule_set
rating_factor
mv_rule
override_permission
override_rule
appetite_rule
attribute_definition
operation_version
product_coverage
product_rule_mv_rule_set
product_component_class
rating_factor_schedule
mv_rule_version
attribute_applicability
submission_requirement
question_attribute_mapping
product_rating_plan
product_form_rule
product_question_set
coverage_relationship_rule
coverage_attribute_applicability
coverage_limit_option
coverage_target_type
product_question_set
product_component_override
coverage_relationship_rule
product_coverage_class

5. Risk model

schema: product **schema: risk** **store: Domain SQL** The line-agnostic risk backbone. A risk now has versions, one per submission, quote or policy version, so each context owns a frozen copy of its risk objects.

v4.7 `risk_object_type` and `exposure_type` no longer belong to one `risk_type`; they are shared library entries, and `risk_type_object_type` and `risk_type_exposure_type` say which risk types use them. Vehicle, driver, location and organisation are defined once and reused by every product pack.

v4.1 `risk_object_schema`: a versioned JSON schema per object type, built from governed attribute definitions (usually shipped in a product pack); `attributes_json` is validated against it and undeclared attributes are rejected.

v3.1 `risk_schedule` + `risk_schedule_item`: vehicle, driver, location, building, equipment and underlying-policy schedules with stable item numbers (Vehicle #2 stays #2 across endorsements) and schedule-level terms such as a blanket limit.

NEW `risk_version`: copy-on-write per context. A quote edit creates a new version; the bound policy keeps pointing at its frozen one.

DEL `risk.source_context_type` / `source_context_id` (the context now lives on `risk_version`) and `exposure.risk_id` (redundant through the risk object).

v4.4 `classification_system`: a versioned registry of code systems (ISO GL, NCCI, NAICS, SIC, commercial-auto vehicle classes, carrier-specific systems) with publisher, version, line and dates. `class_code` points at its system instead of a fixed enum, so new systems need no database change. Codes are unique per system, code, state and effective period.

v3.3 `attribute_definition` + `attribute_value_option`: every attribute, typed column or JSON path, is defined once with type, unit, range, allowed sources and whether it is rateable or underwriting-relevant. `attributes_json` may only hold defined attributes.

v3.1 Schedules group objects; they do not say what is covered. Coverage is still recorded only in the `*_coverage_risk` link tables, which may now point at a whole schedule for blanket cover.

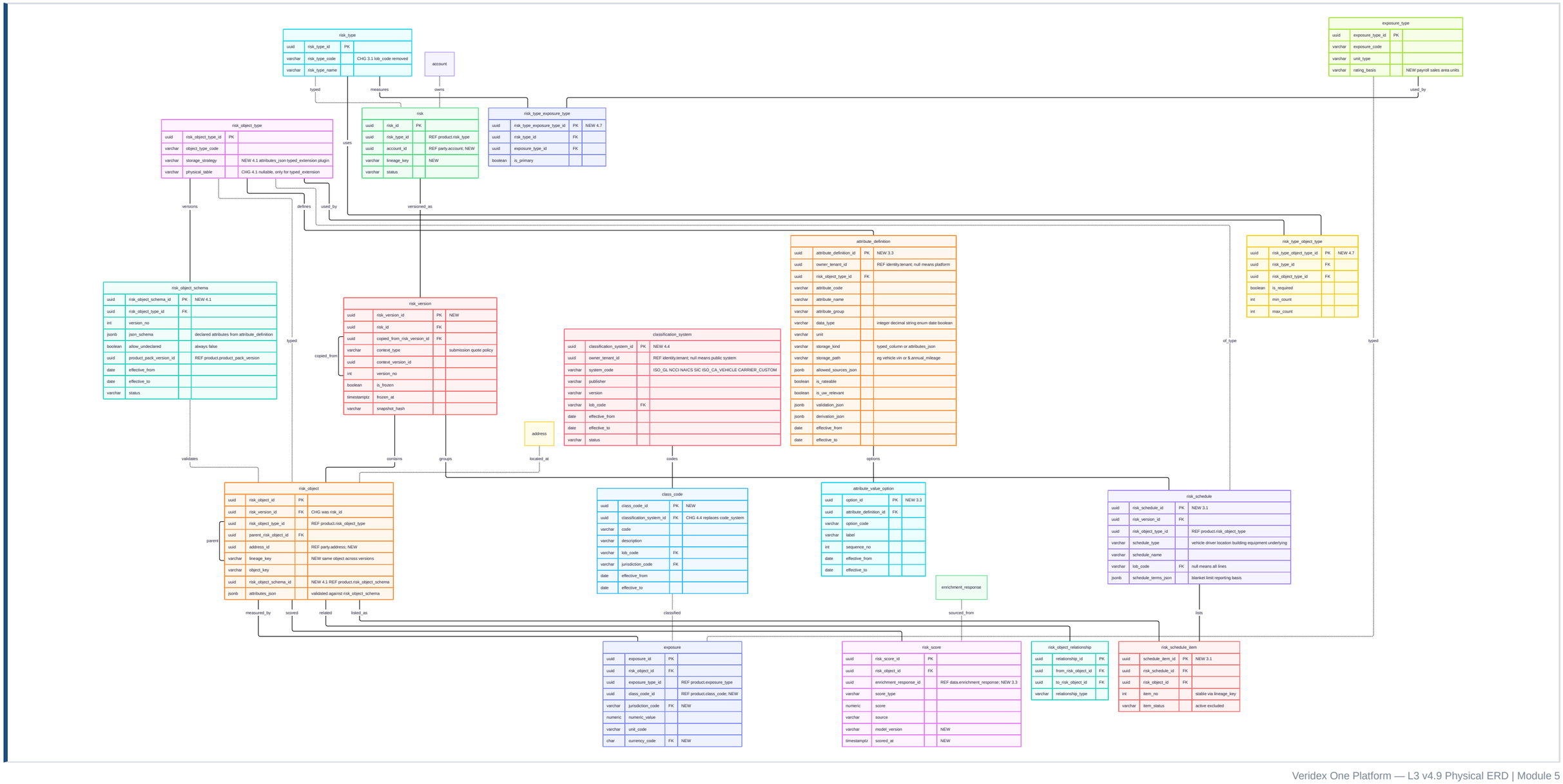
CHG `risk_object` now belongs to a `risk_version`. `lineage_key` lets you trace "vehicle 3" from submission through every endorsement.

v4.1 Risk types are schema-free: `risk_object_type.storage_strategy` is attributes_json, typed_extension or plugin, and `physical_table` is set only for typed_extension. Typed tables are optional optimisations for mature, high-volume types.

v3.3 `risk_score.enrichment_response_id`: every outside score points at the provider response it came from.

v3.1 `risk_type.lob_code` removed: a vehicle can be exposed under auto liability and cargo at once, so risk types are LOB-agnostic.

NEW `class_code` for ISO GL, NCCI and NAICS classes, used by exposures, appetite and authority limits.



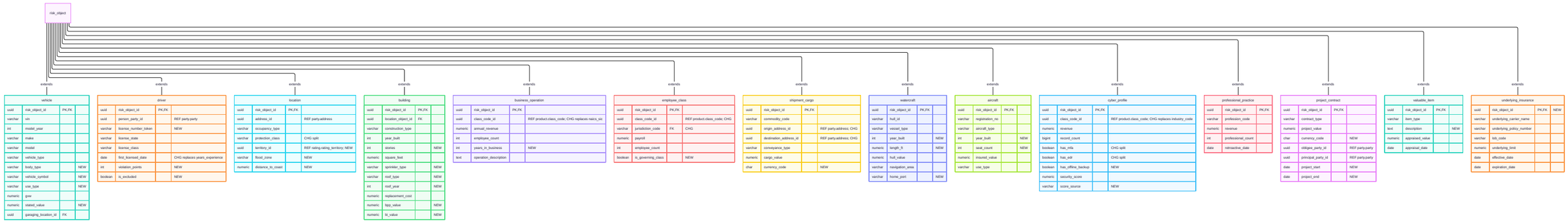
6. Typed risk object extensions

schema: risk **store: Domain SQL** Stable, rateable attributes per line of business. Each table is 1:1 with `risk_object` (primary key is also the foreign key). Sparse attributes stay in `attributes_json`.

NEW `underlying_insurance`: umbrella and excess need a schedule of underlying policies; v2 had no place for it.

CHG Combined columns split (`mfa/edr`, `territory/protection`) so each can be rated and validated on its own.

CHG Free-text class codes replaced by `class_code_id`; addresses replaced by `address_id`.



7. Application, submission & placement

schema: submission **store: Domain SQL** Intake through market placement. A submission version freezes its risk and product version; one version can go to several markets.

v4.3 `terms_json` holds the selected values of the coverage terms defined in `coverage_term` (coinsurance, valuation, retroactive date, waiting period, territory).

v3.3 The summary sits on `submission_version` (`completeness_pct`, `mandatory_missing_count`, `ready_for_uw`) rather than a one-to-one table, so queues can filter on readiness directly.

NEW `clearance_check`: wholesalers and MGAs must block duplicate submissions for the same insured from different brokers.

v4.0 Records the `product_release_id` it ran on, so the Product Runtime can show exactly which configuration produced it.

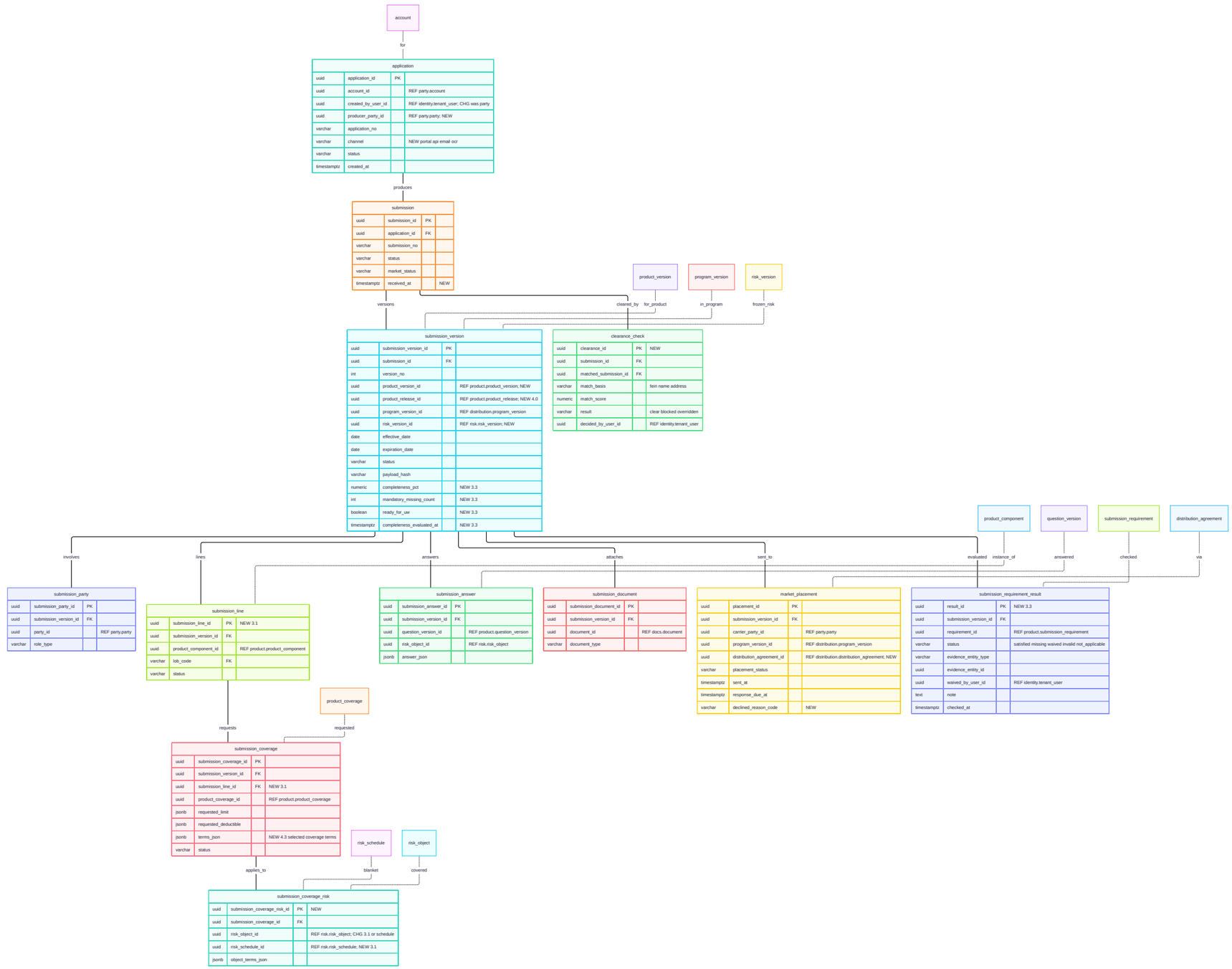
v3.1 `submission_line`: one row per requested product component (Property, GL, Auto...). Coverages sit under their line.

NEW `submission_coverage_risk` replaces the single `risk_object_id`, so blanket and scheduled coverages can apply to many objects.

v3.3 `submission_requirement_result` records each requirement as satisfied, missing, waived, invalid or not applicable, with its evidence.

v3.1 `submission.lob_code` removed; lines carry the LOB. Coverage-risk links can point at one object or a whole schedule, never both.

DEL `submission_risk`, replaced by `submission_version.risk_version_id`.



8. Underwriting & rating

schema: rating · schema: underwriting · store: Domain SQL — The underwriting workbench and the rating engine. Rating works for new business, endorsements and renewals, and always records which override version it used.

CHECK `rating_request_coverage`: `num_nonnulls(submission_coverage_id, policy_coverage_id) = 1`.

UNIQUE `rate_table_dimension`: `(rate_table_id, sequence_no)` and `(rate_table_id, dimension_code)`. `rate_table_row_condition`: `(row_id, dimension_id)`.

v4.9 `rate_table_dimension` defines each input of a rate table (for example challan count, vehicle type, driver age, rating class): order, the rating factor that supplies it, type, match type (exact, range or set), unit and allowed values. `rate_table_row_condition` stores each row's conditions relationally: operator, boundary values and inclusivity, so "1–5", "6–10" and "greater than 10" are rows with `between 1 and 5`, `between 6 and 10` and `gt 10`, each with its `rate_value` (1.50, 1.75, 2.00).

v4.7 Rating is fully relational and auditable: `rating_algorithm_step` (ordered calculation steps) and `rating_algorithm_factor` (which factors a coverage uses, from which scope or related object, required or not, how missing values are handled) are authoritative; the JSON columns on `coverage_rating_algorithm` are compiled caches.

v4.7 `uw_authority_level` moves here from Module 1 (it is underwriting authority, not identity) and points at `tenant_user`.

v4.5 `rule_evaluation_run`: one run per submission version (or case) and stage, with counts and outcome (proceed, refer, auto-declined) and the system decision; `rule_evaluation` rows belong to a run, so triage rules work before a case exists. `uw_note` removed in favour of the generic `note` (module 16).

v4.0 Records the `product_release_id` it ran on, so the Product Runtime can show exactly which configuration produced it.

v3.4 `underwriting_case.renewal_case_id`: renewal underwriting is a case like any other.

v3.2 `uw_override.approval_request_id`: overrides go through the shared approval chain (module 16).

CHG `rating_request` can now rate an endorsement (`policy_transaction_id`) and records the resolved product override version.

CHECK `rating_request_coverage` target: object → `risk_object_id` set and `risk_schedule_id` empty; schedule → `risk_schedule_id` set and `risk_object_id` empty; policy → both empty.

CHECK `rate_table_row_condition`: between → `value_from` and `value_to` set with `value_from <= value_to`; in → `value_list_json` set; other operators → exactly one value.

v4.9 Product Studio's rating configuration screen maintains rate tables, dimensions, rows and conditions; `rating_algorithm_factor` then connects the configured factor and table to each coverage algorithm. `dimension_schema` and `criteria_json` become compiled caches built at publish.

v4.7 `rating_factor` gains factor type, its lookup table and default, minimum and maximum values; `rate_table` gains its purpose. `rating_factor_result` records, per rated coverage and object, the input, matched table row, applied value and premium before and after. `rating_input` points at its factor.

v4.5 `uw_rule` gains severity (knockout, major, minor, info), outcome on fail (decline, refer, warn), stage, message, active flag and dates. Priority is evaluation order; severity is what a failure means. Knockouts auto-decline; majors refer.

v4.4 `rating_request_coverage`: one row per product coverage being rated, with the source submission or policy coverage, the risk object or schedule (exactly one), the selected limit, deductible and terms, and the algorithm used. `premium_component` carries `product_coverage_id` (authoritative) and the rated coverage row; `coverage_id` stays for reporting. Algorithms may be narrowed to a product coverage.

v3.4 `pricing_adjustment_rule` + `pricing_adjustment`: schedule rating, experience modification, underwriter debits and credits and negotiated changes, each within filed limits, checked against authority and approved through the approval chain. `uw_override` is now only for underwriting decisions and rule outcomes, not price.

v3.3 `assessment_framework`, `assessment_dimension`, `uw_assessment`, `assessment_response`: structured underwriter judgment (management, maintenance, driver controls, safety technology, finances) with weights, scores and evidence, alongside the deterministic rules.

v3.1 Fee and tax rows in `premium_component` record the `charge_rule` that calculated them; premium rows leave it empty.

NEW `uw_referral.authority_limit_id` shows exactly which delegated limit triggered the referral.

FK Composite keys keep `rating_algorithm_step`, `rating_algorithm_factor`, `rate_table_dimension`, factors and tables in one `rating_plan_version`.

PUBLISH Every row of a rate table has one condition per dimension, and no two rows can match the same input (no overlapping ranges); an input that matches no row follows the factor's `missing_value_action`.

v4.8 Rating configuration stays inside one plan version: `rating_algorithm_step` and `rating_algorithm_factor` carry `rating_plan_version_id`, and composite foreign keys require the algorithm, its steps, factors, base and lookup tables to share it.

v4.7 `rating_request_coverage.target_level` (object, schedule or policy) makes the target explicit: exactly one source coverage, and exactly one of object or schedule unless the level is policy.

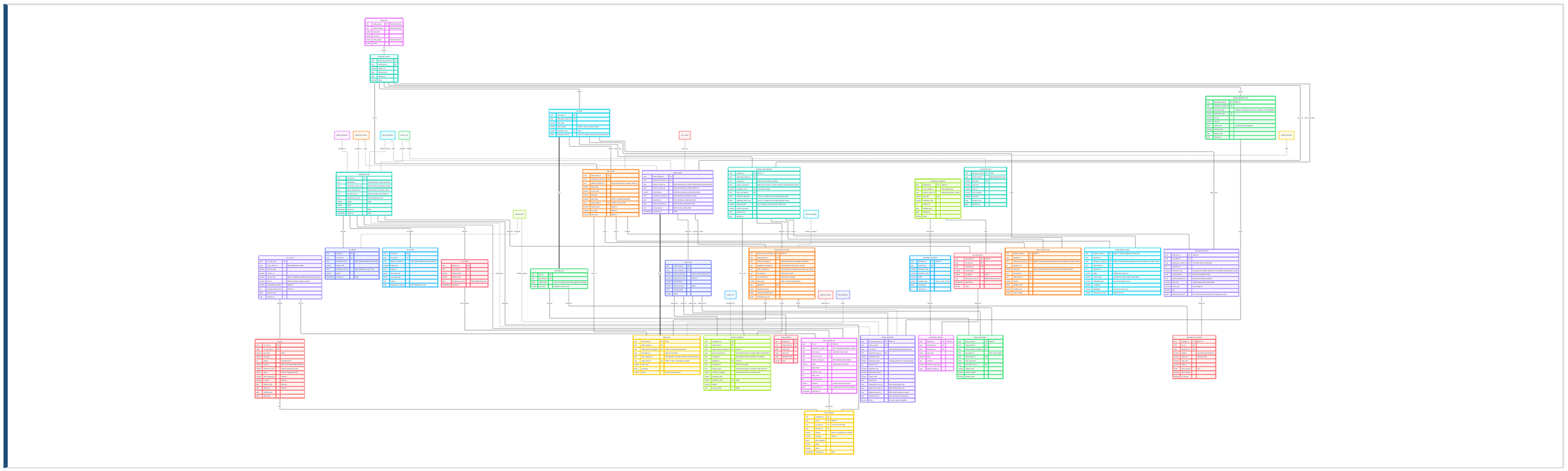
v4.5 `uw_rule_set` gains kind, source (Product Studio or Underwriting Admin) and decision policy (auto-decline enabled, warnings before refer). Both sources use the same tables; admin rule sets are tenant-owned and can only tighten.

v4.3 `coverage_rating_algorithm`: within a rating plan version, how each coverage is rated (base rate table, ordered steps, applicable factors, premium basis, minimum premium, state). Each `premium_component` records the algorithm that produced it.

v3.4 `rating_result` shows `technical_premium`, `adjustments_total` and the final `total_premium`, so the price lineage is technical premium → adjustments → fees and taxes → quoted premium.

v3.3 `rating_factor.attribute_definition_id` ties each factor to a governed attribute; `rating_input.data_resolution_id` proves which trusted value was rated.

NEW `rating_plan` was referenced by `rating_plan_version` in v2 but never defined.



9. Quote & bind

schema: pas **store: Domain SQL** Quotes carry their own frozen risk, fees and taxes separate from premium, and a bind request that records the authority check.

v4.3 `terms_json` holds the selected values of the coverage terms defined in `coverage_term` (coinsurance, valuation, retroactive date, waiting period, territory).

v3.4 `quote_condition` distinguishes pre-bind subjectivities (`blocks_bind`) from post-bind risk improvements, with evidence. Post-bind conditions are carried to `policy_condition` (module 10).

v3.1 `quote_charge.charge_rule_id`: every fee and tax on a quote shows the rule that produced it.

v4.0 Records the `product_release_id` it ran on, so the Product Runtime can show exactly which configuration produced it.

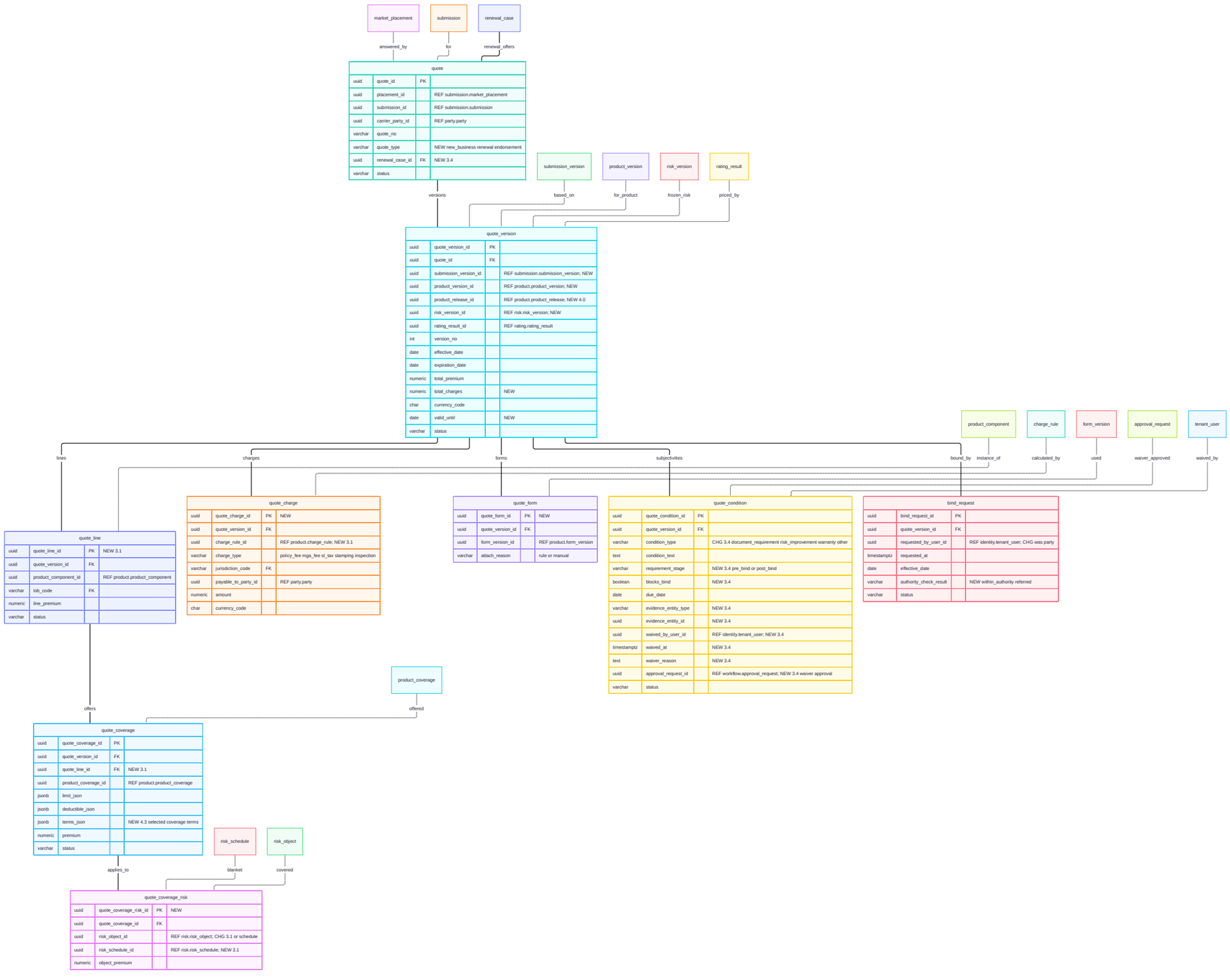
v3.4 A renewal offer is a quote: `quote.renewal_case_id` links each renewal quote to its renewal case; every revised offer is a new `quote_version`.

NEW `quote_charge`: surplus lines tax, stamping and MGA fees must never be mixed into premium, because they are remitted to different parties.

v3.4 Waivers on `quote_condition`: `waived_by_user_id`, `waived_at`, `waiver_reason`, and `approval_request_id` when the waiver needs approval (for example, a condition imposed by the carrier).

v3.1 `quote_line` with a premium subtotal per LOB; coverages sit under lines.

CHG `quote_version` now links back to the `submission_version` it was built from.



10. Policy administration

schema: pas · store: Domain SQL · Policy, term, transaction and version, with a traceable path back to the quote that was bound and the exact product and program versions in force.

UNIQUE `policy_premium_audit.policy_transaction_id`: one audit per audit transaction.

CHECK `audit_period_end > audit_period_start`; `premium_difference = final_earned_premium - deposit_premium`; one final audit per term.

v4.7 `policy_transaction` stays the common header; subtype detail lives in `policy_cancellation` (type, initiator, notice, earning rule, earned and return premium, penalty, fees, net refund), `policy_endorsement` and `policy_reinstatement` (the cancellation it reverses, lapse, conditions, fee).

v4.3 `terms_json` on `policy_coverage` holds the selected coverage terms. Renewal keeps the policy number and adds a `policy_term`; whether the printed number changes per term is a `number_sequence` rule (module 23).

v3.4 `policy_condition`: post-bind conditions copied from the bound quote onto the policy term, tracked to satisfaction or breach with evidence and consequence, independent of the quote.

v3.2 `policy_party.mailing_address_id` freezes the address printed on each policy version; addresses are never edited.

CHG `policy` points at product and program *versions*, so you can always answer "which rules applied to this policy".

DEL `policy_risk` and its jsonb snapshot, replaced by `policy_version.risk_version_id`.

TRIGGER The referenced `policy_transaction` has `transaction_type = audit` and belongs to the same `policy_term` as the audit.

v4.8 One reason per transaction: `policy_transaction.transaction_reason_code` (renamed from `reason_code`) is the only reason field, since reverse, correct and rescind transactions need one too; `policy_cancellation`, `policy_endorsement` and `policy_reinstatement` no longer repeat it. Their type fields (`cancellation_type`, `endorsement_type`) remain. `return_premium_method` removed: the method comes from the cancellation's earning rule.

v4.7 `policy_transaction_change` lists every object and field a transaction changed, with its rating, underwriting, forms and billing impact; `policy_transaction_status_history` is append-only. New types reverse, correct and rescind cancellation, with `reverses_transaction_id` and `supersedes_transaction_id`: completed transactions are never edited.

v4.0 Records the `product_release_id` it ran on, so the Product Runtime can show exactly which configuration produced it.

v3.3 `policy_limit_bucket`: each limit as its own record on the policy term (per occurrence, aggregate, combined single limit, sublimit through `parent_bucket_id`), so aggregates erode across endorsements.

v3.1 `policy_line`: the policy holds lines of business, each with its coverages, as in ACORD. Premium, commission, bordereaux and statutory reporting can now be split by LOB through coverage → line.

NEW Quote → policy traceability through `bind_request_id` and `source_quote_version_id`; renewals chain through `prior_term_id`.

TRIGGER Every `policy_audit_exposure.policy_coverage_id` belongs to a `policy_version` of the audited `policy_term`.

v4.8 `policy_premium_audit` and `policy_audit_exposure`: premium audit for auditable coverages (workers comp payroll, GL sales, hired auto cost of hire, for-hire revenue). The audit records period, method, auditor, deposit and final earned premium, the difference, the audit transaction that bills it and any dispute (through approvals and the complaint register). Each line holds reported and audited exposure by coverage, class, exposure type and location or vehicle, with the rate applied, so the premium change can be recalculated.

v4.3 `policy_version` is dated on two axes: `effective_from` / `effective_to` (when the coverage applies) and `recorded_at` / `superseded_at` (when we knew it). Non-superseded versions of a term never overlap, so "in force on the loss date" is one lookup, and "as we knew it on a past date" is answerable after out-of-sequence changes.

v3.4 Waivers on `policy_condition`: the same four fields, so a post-bind condition can be waived with a recorded person, time, reason and, above a set level, an approval.

v3.3 `policy_limit_bucket_coverage`: shared limits, such as one CSL drawn on by both bodily injury and property damage.

v3.1 `policy_charge.charge_rule_id` links each stored fee or tax to its rule.

NEW `policy_transaction_premium`, `policy_charge` and `earned_premium` give accounting written, earned and unearned figures per transaction.



11. Claims

schema: claims **store: Domain SQL** FNOL to closure. Claims link to the policy version in force on the loss date, and reserves are a movement ledger rather than a single overwritten amount.

v4.5 `claim_note` removed in favour of the generic `note` (module 16), so notes work the same way for claims, underwriting cases, legal matters and every other record.

v4.0 `claim` and `claim_exposure` point at Claims' own policy and coverage snapshots (module 35) instead of PAS tables, so Claims runs with the Veridex PAS or the customer's.

v3.5 `coverage_determination`: `valid_from`, `valid_to` and a generated `is_current`; one current determination per exposure, no overlaps, linear chain. `claim_exposure.coverage_decision` must equal the current determination (kept in step by the same transaction).

v3.4 `coverage_determination`: the history of coverage positions on each exposure (pending, reservation of rights, coverage counsel review, covered, partial, denied), each superseding the last, with letter and legal matter. `claim_exposure.coverage_decision` keeps the current value.

v3.3 `claim_incident`: what physically happened (damage to the truck, injury to a person, damage to another car); each exposure arises from one incident against one coverage, through `claim_exposure.claim_incident_id`.

v3.3 `policy_limit_transaction`: every reserve, payment and recovery that erodes a limit bucket, with the available limit after it.

v3.2 `litigation.legal_matter_id`: every lawsuit is also a legal matter, so holds, counsel and invoices are handled in one place (module 21).

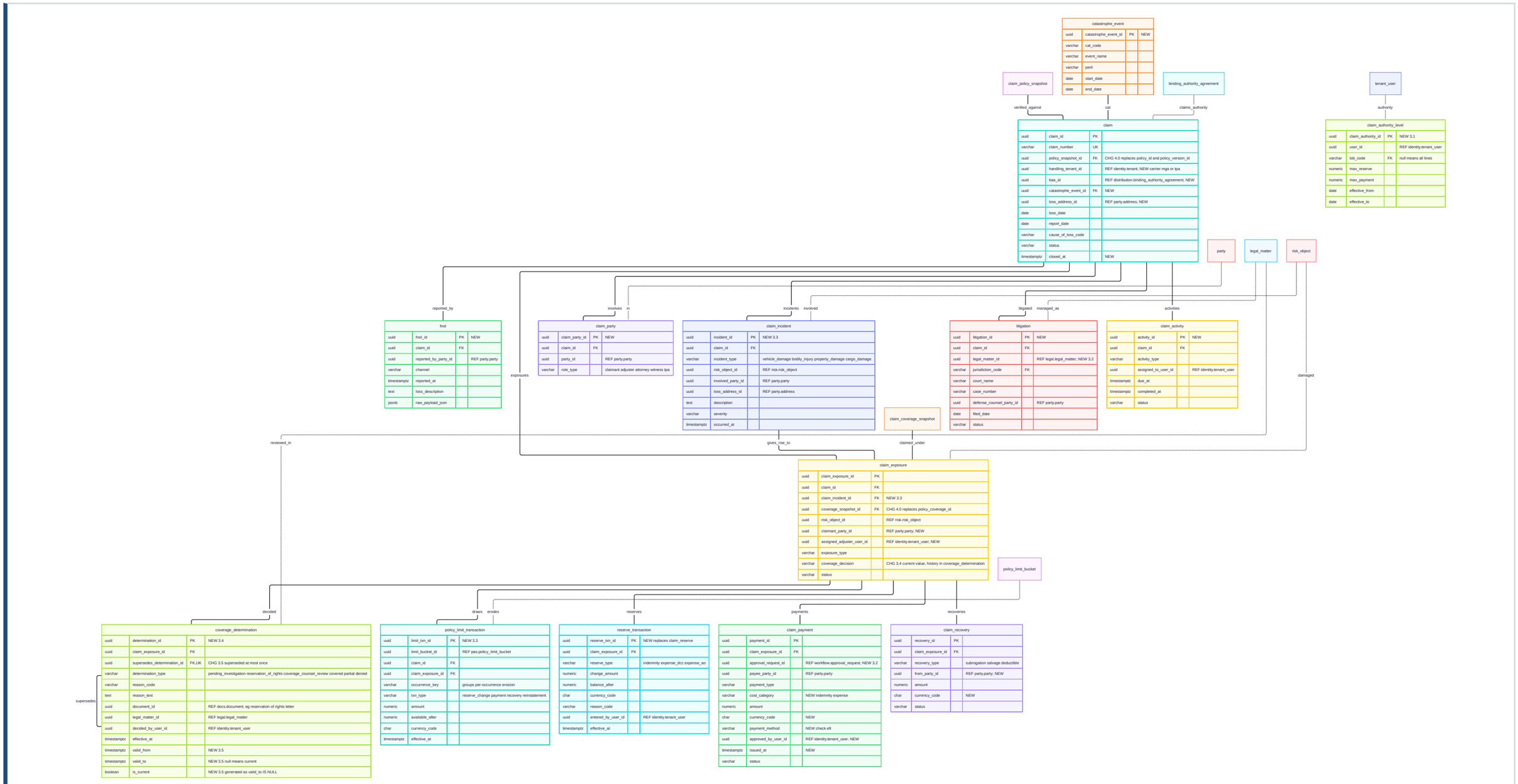
v3.2 `claim_payment.approval_request_id`: payments above the adjuster's authority go through the shared approval chain.

v3.1 `claim_authority_level`: per-user reserve and payment limits, the claims mirror of `uw_authority_level`. Tenant-level limits stay on the binding authority agreement. Vendor assignment and multi-level payment approval remain on the phase 2 roadmap.

CHG `claim.policy_version_id`: coverage verification uses the version in force on the loss date, which matters after out-of-sequence endorsements.

CHG `claim_reserve` becomes `reserve_transaction`. Actuarial reporting and claims bordereaux need reserve movements, not just the latest value.

NEW `handling_tenant_id` + `baa_id` support MGAs and TPAs handling claims under delegated authority.



12. Billing

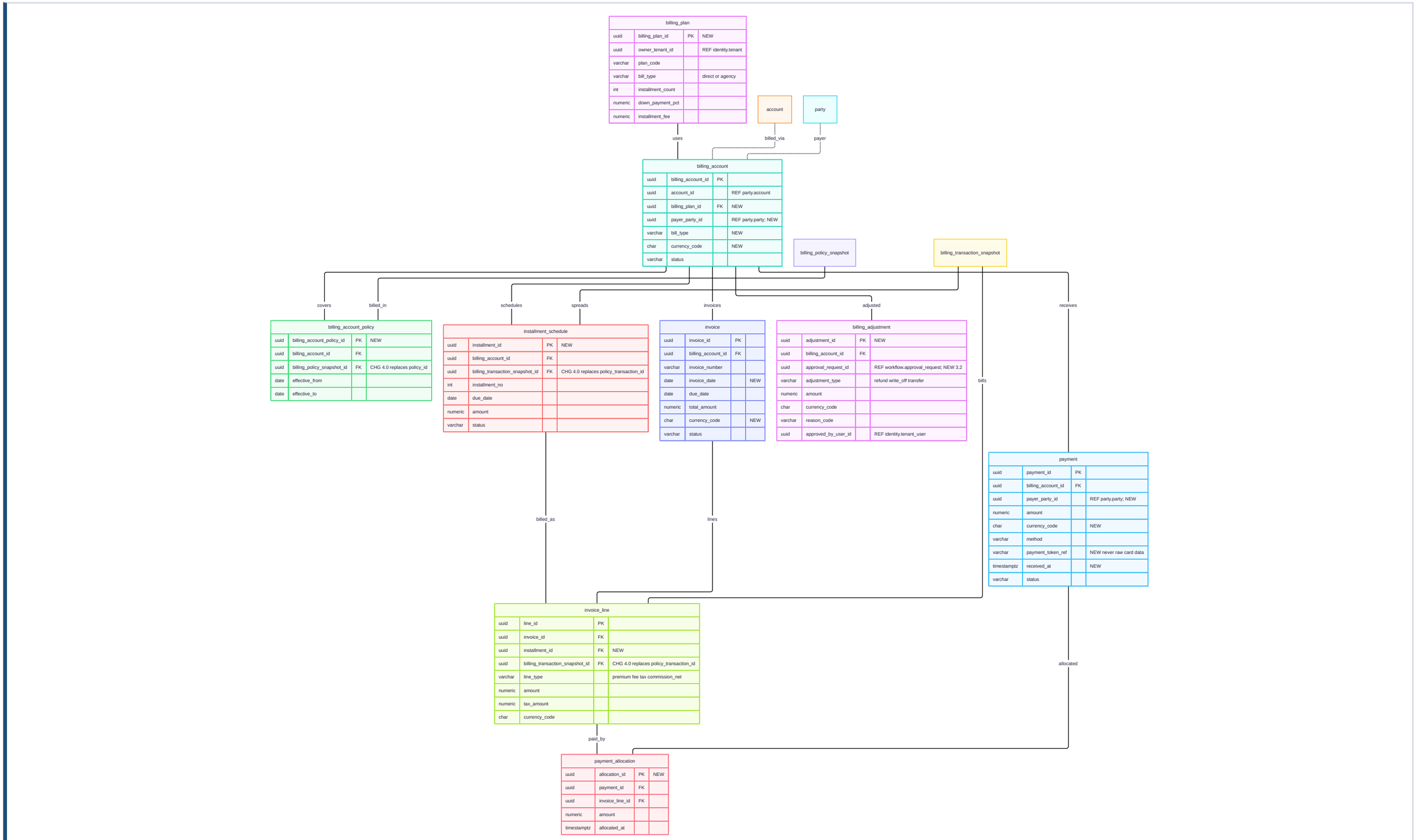
schema: billing **store: Domain SQL** Direct and agency bill, instalments, invoices tied to policy transactions, and payments allocated line by line.

v4.0 Billing works from its own policy and transaction snapshots (module 35), filled from the Veridex PAS or the customer's PAS.

NEW Payment allocation, instalments and adjustments. Without allocation you cannot tell which charges a partial payment settled.

v3.2 `billing_adjustment.approval_request_id`: refunds and write-offs use the shared approval chain.

CHG `billing_account.policy_id` moved to `billing_account_policy`, so one billing account can cover several policies for an insured.



13. Commission, bordereaux & settlement

schema: finance **store: Domain SQL** How money moves between the parties in the distribution chain. This is how an MGA reports to, and settles with, each carrier every period.

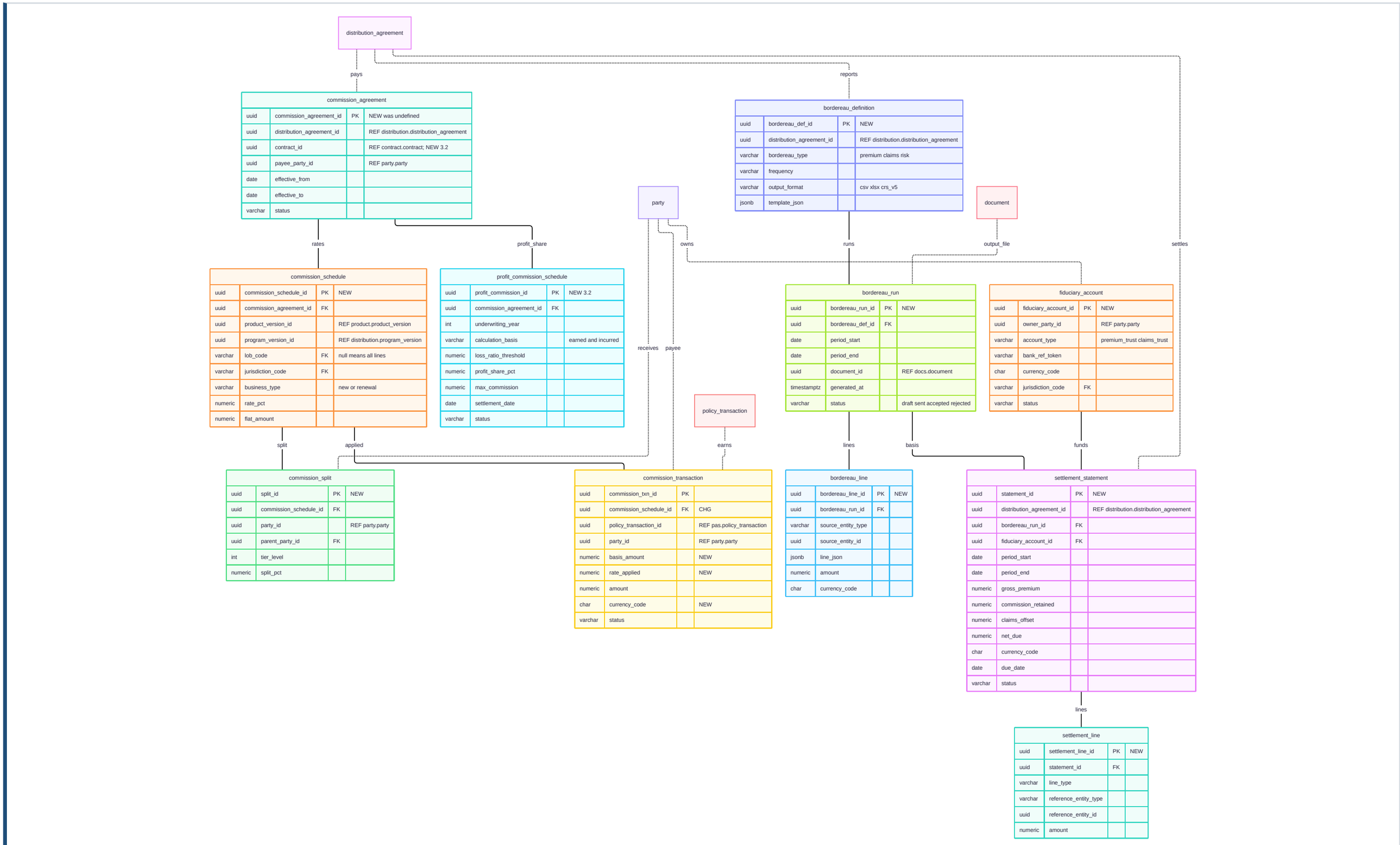
v3.2 `profit_commission_schedule`: contingent commission paid if the loss ratio stays under a threshold.

v3.2 `commission_agreement.contract_id` links to the common contract record.

NEW `commission_agreement` was referenced in v2 but never defined. Schedules vary by product, program, state and new vs renewal.

NEW `commission_split` handles the chain carrier → MGA → wholesaler → retail broker.

NEW Bordereaux and settlement statements: absent in v2, and central to every MGA–carrier relationship.

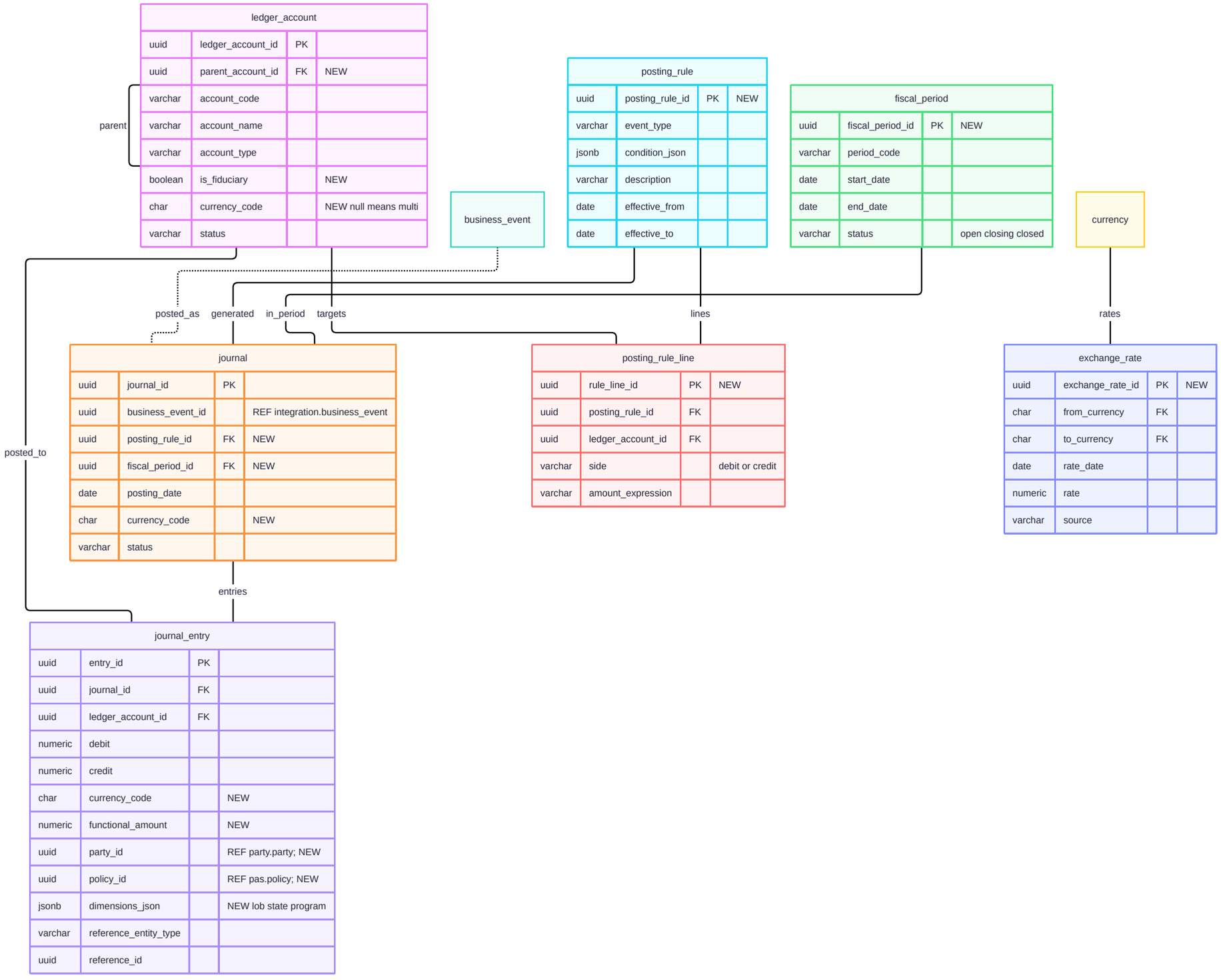


14. Accounting

schema: finance **store: Domain SQL** Each tenant keeps its own general ledger. Journals are generated from business events by posting rules, per fiscal period, in multiple currencies.

NEW `posting_rule` turns "policy issued" or "claim paid" events into balanced journals without hard-coding accounts in services.

NEW Fiscal periods and exchange rates for period close and marine, cargo and aviation business in foreign currency.



15. Questions, forms & document management

schema: docs · schema: product · store: Domain SQL · store: Platform SQL — Underwriting questions, the ACORD and carrier form library with field-level mappings, document storage, and OCR extraction results.

v4.7 Corrected `document_type.default_retention_policy_id`: a foreign key to `retention_policy` (v4.6 wrongly pointed at `pas.policy`).

v4.5 `document_folder`: nested folders owned by a matter, claim, policy, account or task (the legal app's `Folder`); `document.folder_id`. `document_version` records file name, size, uploader, upload time and the journey session it came from.

v3.2 `retention_policy` calculates each document's retention date by type and state; legal holds (module 21) can block disposal.

NEW `form_field` + `form_field_mapping`: this is what makes the ~900 ACORD forms dynamic. Each field maps once to a canonical column, attribute path or question.

v4.6 `document` gains the same privilege fields as notes. Privileged documents are readable only by roles permitted for privileged material, and are listed in the matter's `privilege_log` when withheld or redacted.

v3.3 `form_field_mapping.attribute_definition_id`: form fields map to governed attributes. `extraction_result` stays as the record of an OCR run; the field values it finds become `data_observation` rows (module 24).

v3.2 `document_share_link`: expiring download links for people who are not platform users.

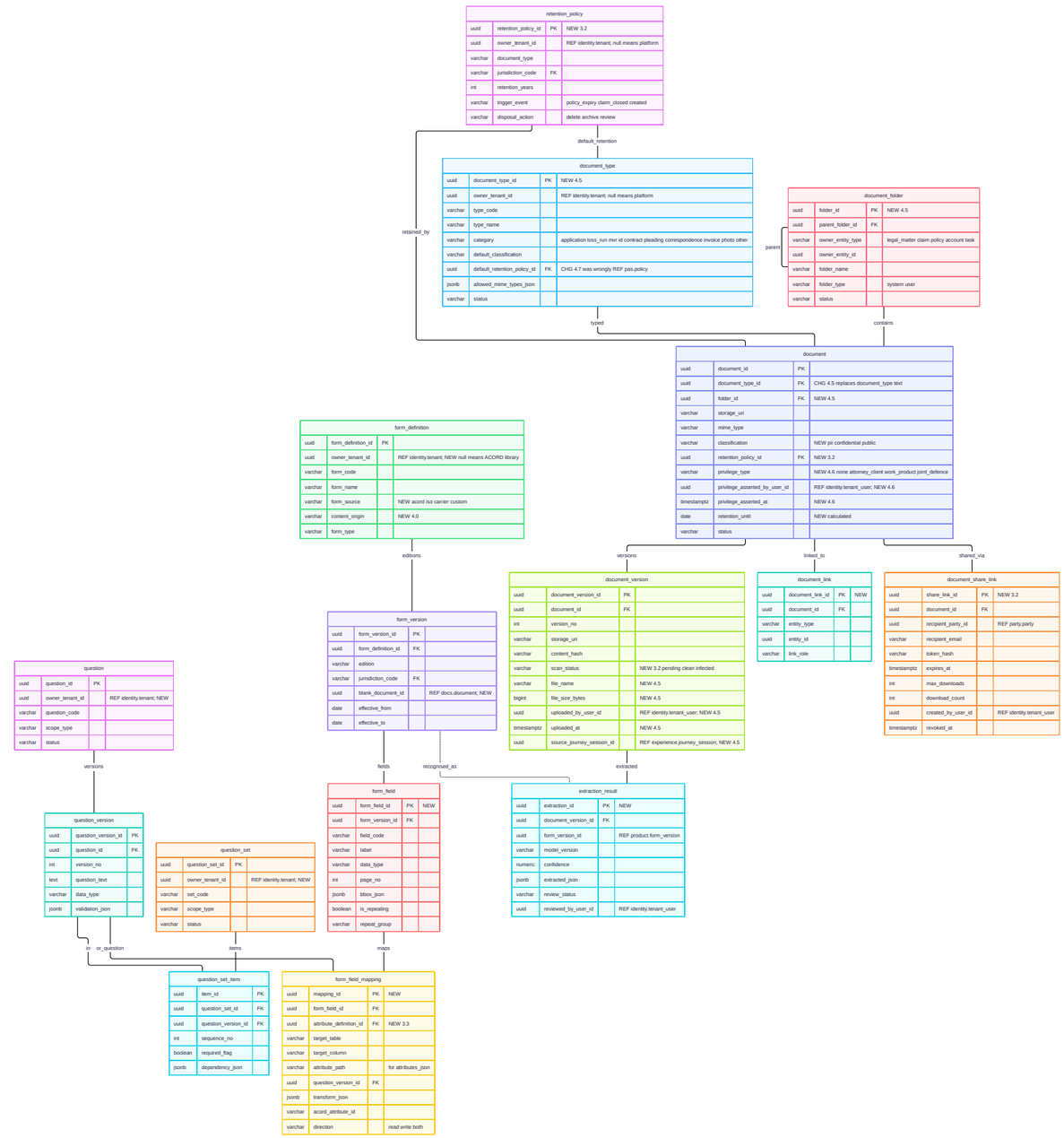
NEW `extraction_result` connects the OCR pipeline to the model: extracted values land here for review before they populate a submission.

v4.5 `document_type`: a governed list of document types with category, default retention and allowed file types; `document.document_type_id` replaces free text.

v3.2 `document_template` removed; templates for every output (forms, letters, emails, SMS) now live in `template` + `template_version` (module 19).

v3.2 `document_version.scan_status`: uploads are malware-scanned before use.

NEW `document_link` attaches any document to any entity, with classification and retention for compliance.



16. Workflow, approvals, audit & integration

schema: integration **schema: platform** **schema: workflow** **store: Platform SQL** Cross-cutting services. `business_event` doubles as the transactional outbox feeding integrations, notifications and accounting.

v4.7 `audit_event` gains `actor_type` and `actor_id`, so actions by API clients and services are attributed as precisely as user actions. Work queues and approval steps point at `user_group` (was `team`).

v4.6 `note` gains privilege: `privilege_type` (none, attorney-client, work product, joint defence), who asserted it and when. Access to privileged notes is decided by an authorization rule on privilege, separately from visibility.

v4.5 `note`: one notes table for any record (matter, claim, case, task, policy), with type, title, visibility (internal, team, partner, external) and pinning; replaces `uw_note` and `claim_note`. Attachments use `document_link`.

v4.5 `task` gains `parent_task_id` (subtasks), title, description and priority; `task_assignee` allows several assignees with roles.

v4.5 `automation_rule`, `automation_execution` and `automation_rule_result`: event-triggered rules (condition, ordered actions, sync or async) and a log of every run with attempts and per-rule results (the legal app's `WorkflowRule` and `ProcessExecution`).

v4.0 `business_event` becomes each module's outbox, with owning schema, per-record sequence, global position and event contract. `integration_endpoint` belongs to a connector instance (module 36).

v3.4 `approval_request.approval_type` now also covers pricing adjustments and condition waivers.

v3.2 `work_item_transition`: full step history of every work item.

v3.2 `work_queue` + `work_queue_member`: queues owned by teams, with capacity.

v3.2 `sla_policy` + `business_calendar`: due dates in business hours per state, with escalation; the calendar also drives notice dates.

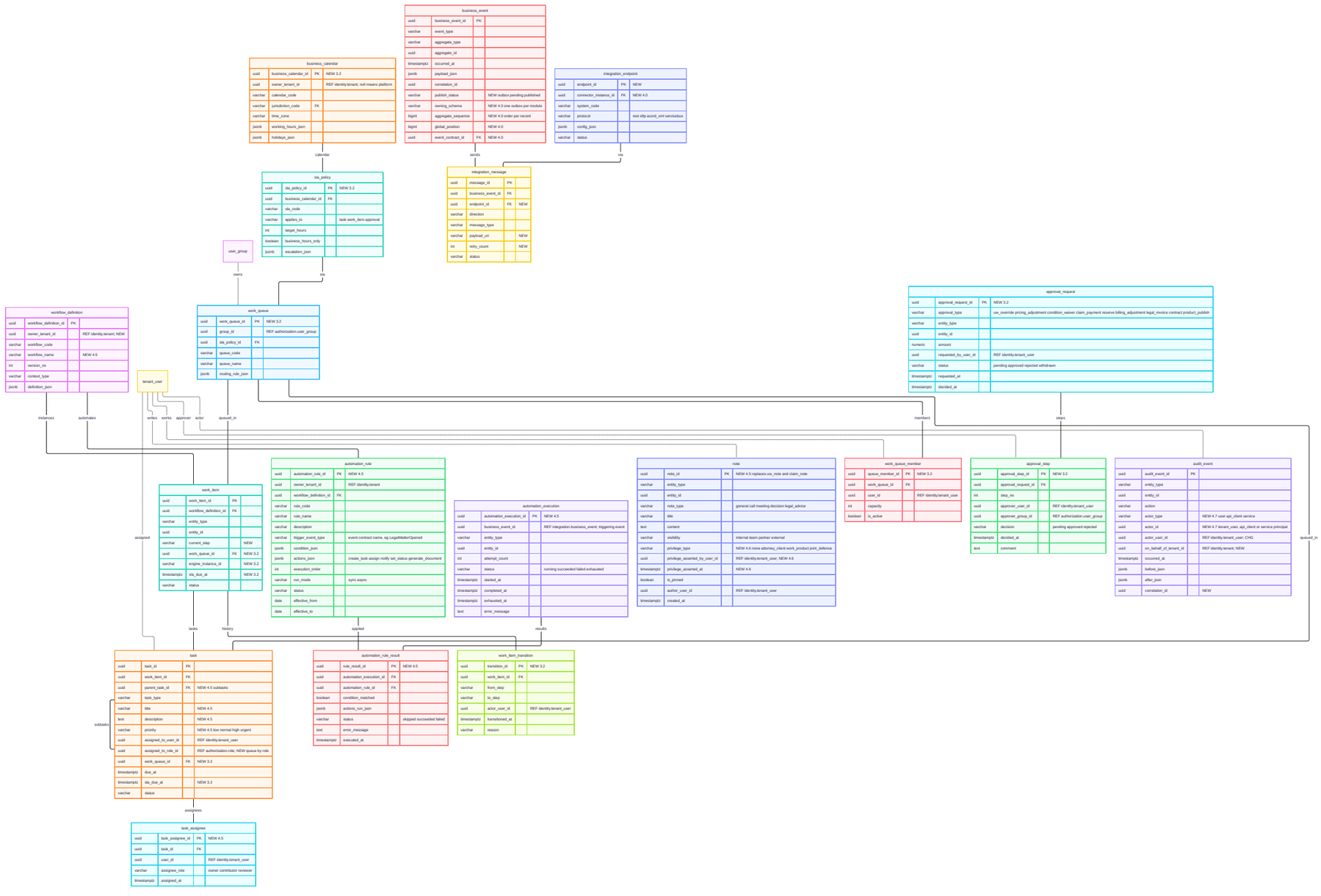
v3.2 `approval_request` + `approval_step`: one approval chain used by overrides, claim payments, reserves, billing adjustments, legal invoices, contracts and product publishing.

v3.2 `work_item.engine_instance_id` links to the workflow engine instance (Durable Functions, Temporal or Camunda).

v3.2 `notification` moved to module 20.

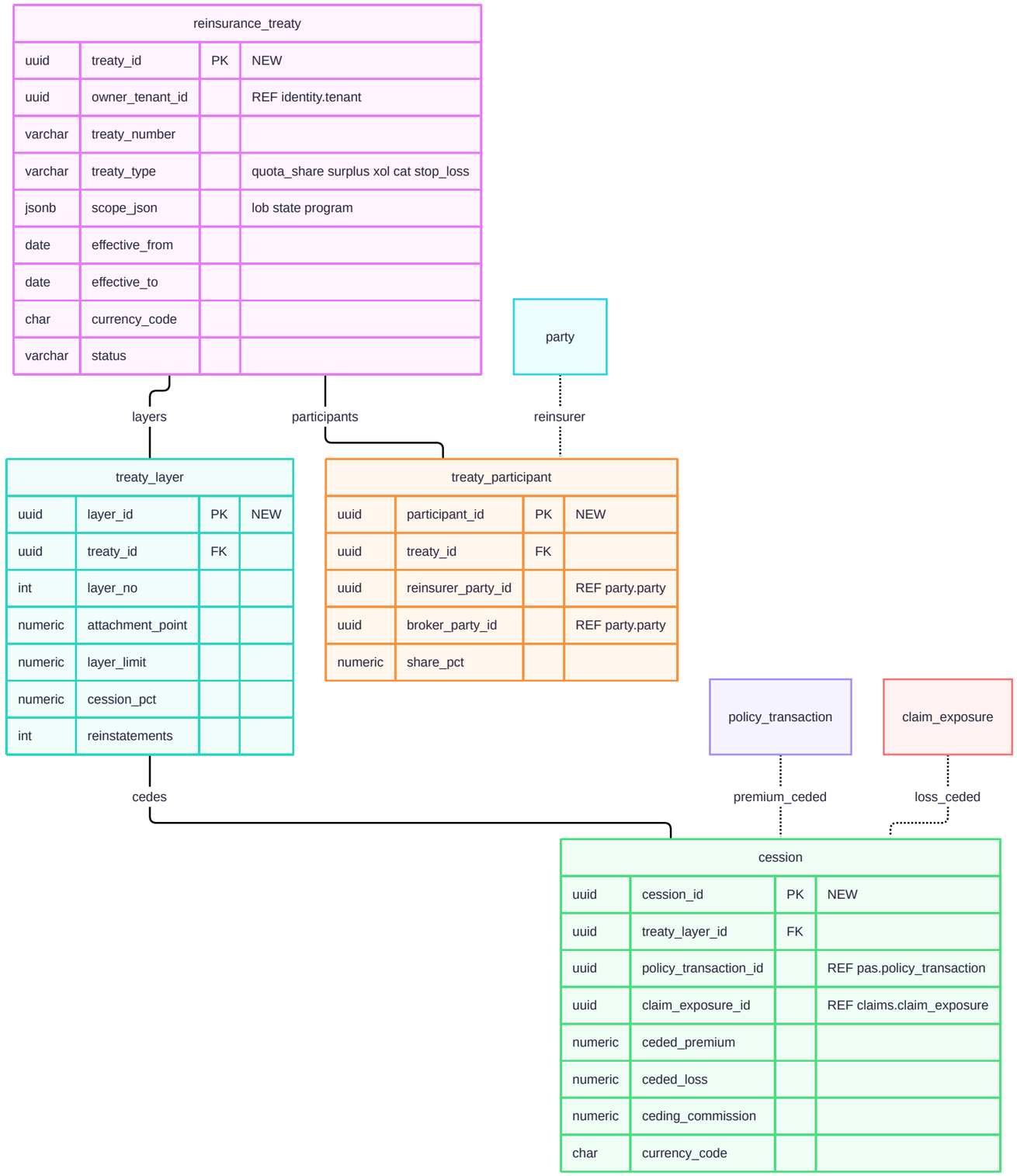
NEW `business_event.publish_status` makes it a transactional outbox for Azure Service Bus, so events are never lost between a DB commit and a publish.

NEW `audit_event.on_behalf_of_tenant_id` records when a user acted in a partner tenant's data, for example a carrier underwriter reviewing an MGA referral.



17. Reinsurance (planned)

schema: reinsurance | store: Domain SQL | planned, not generally available | Kept deliberately small so carrier tenants have a place for treaty cessions later. Not required for the first build.



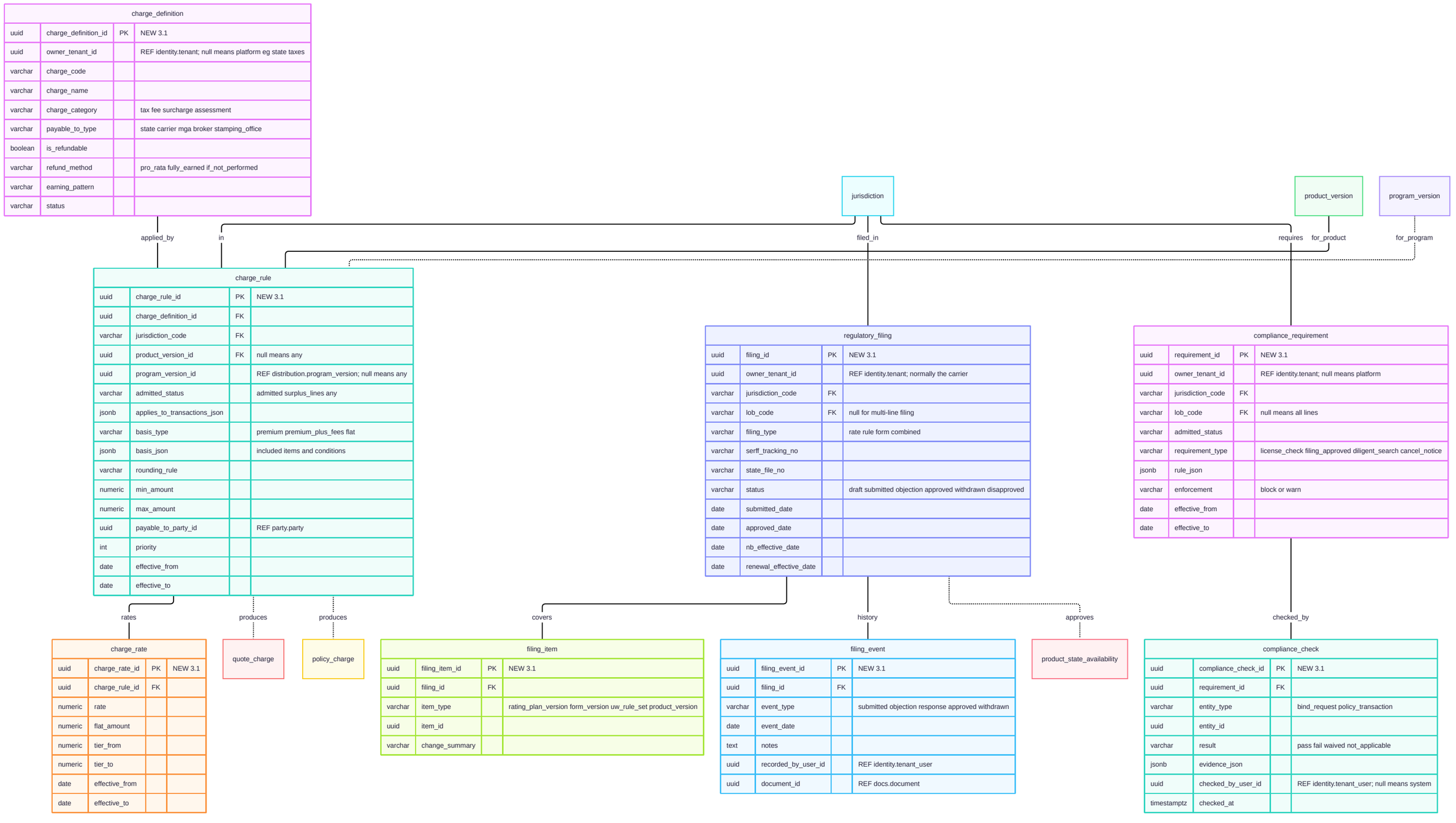
18. Charges, filings & compliance

schema: compliance **schema: product** **store: Domain SQL** Configuration that calculates taxes and fees instead of just storing them, the regulatory filings behind every rate plan and form, and the compliance checks run at bind and issue. Sits beside rating but stays separate from premium.

v3.1 Entire module. `charge_definition` → `charge_rule` → `charge_rate` calculates policy fees, inspection fees, surplus lines tax and stamping fees by state, product, program, admitted status and transaction type, with a taxable basis, rounding and min/max.

v3.1 `regulatory_filing` + `filing_item` link a filing to the exact versioned rate plans, forms, rule sets and override versions it covers, with separate new-business and renewal effective dates. This answers "can this product version be sold in Texas on this date with these rates and forms?"

v3.1 `compliance_requirement` + `compliance_check` record license, filing, diligent-search and cancellation-notice checks at bind and issue, with evidence.



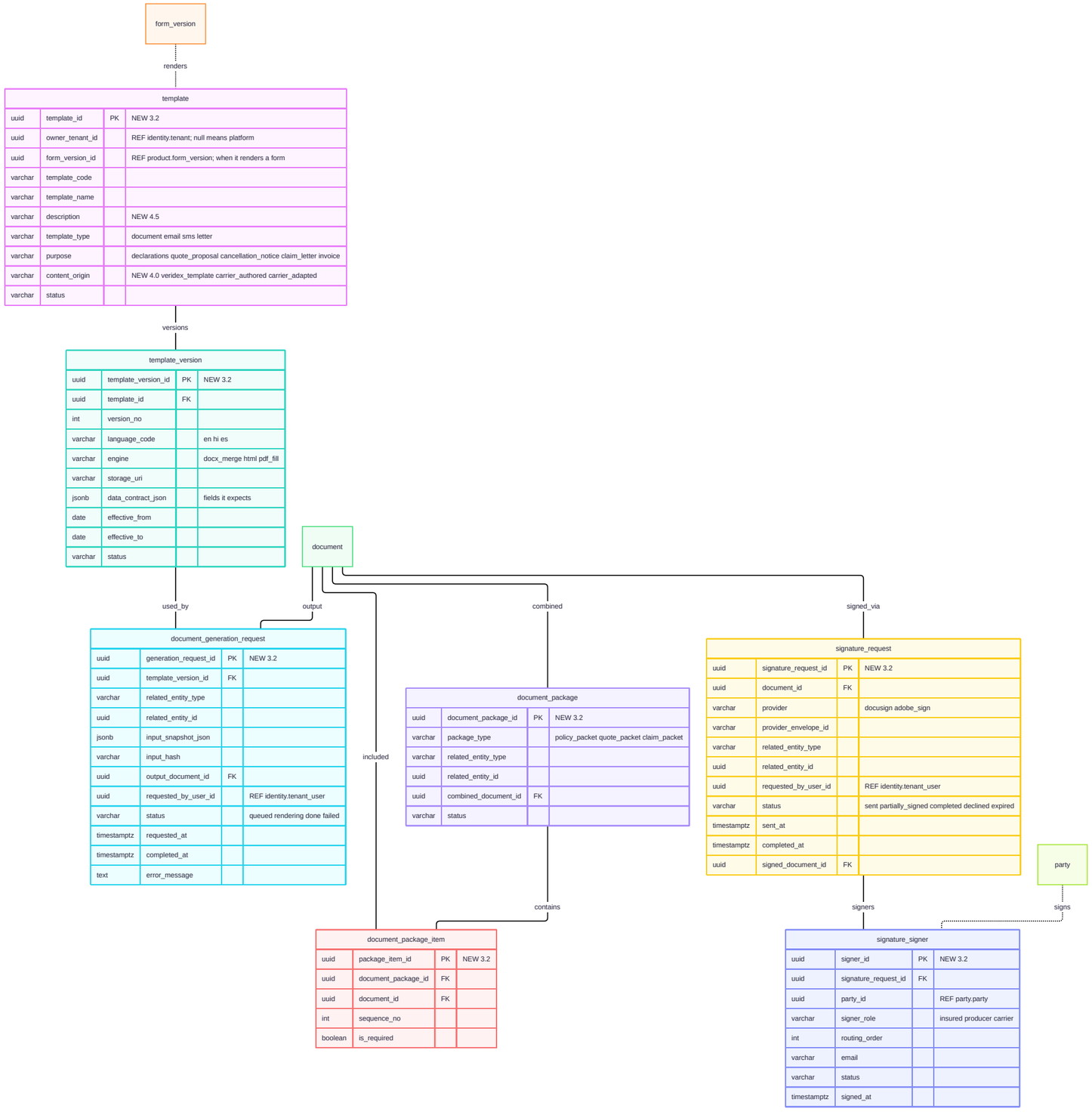
19. Document generation & e-signature

schema: docs **store: Platform SQL** Every document the platform produces (declarations, quote proposals, notices, claim letters) comes from a versioned template, is generated from a frozen data snapshot, can be bundled into packets and sent for e-signature.

v3.2 Entire module. `template` + `template_version` replace `document_template` and cover forms, letters, emails and SMS, per language.

v3.2 `document_generation_request` keeps the input snapshot and hash, so any document can be proved and regenerated.

v3.2 `document_package` assembles packets in order; `signature_request` + `signature_signer` track e-signature with the provider envelope id.



20. Communication & notifications

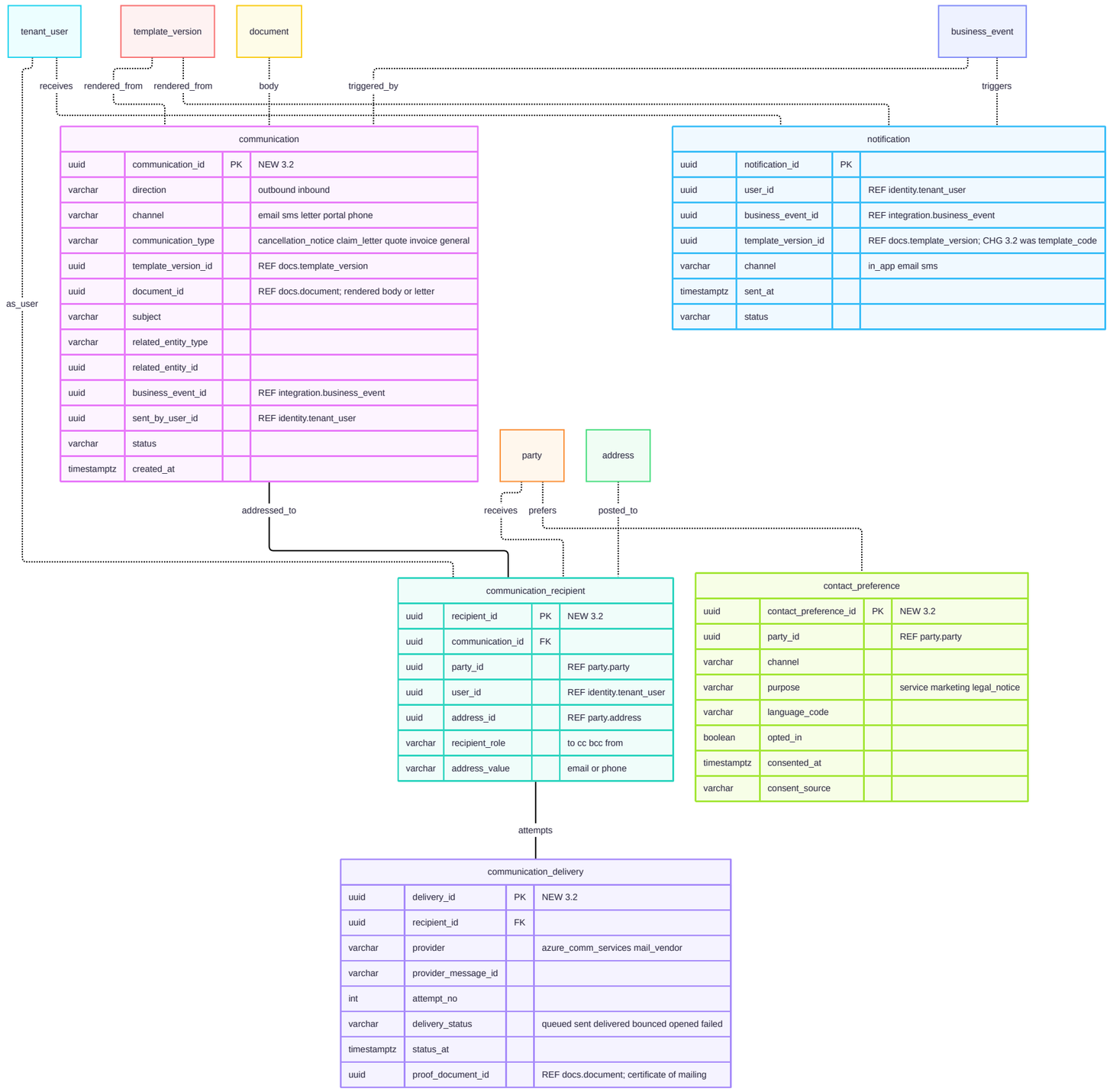
schema: comms **store: Platform SQL** Messages to and from any party, not only platform users: email, SMS, letters and portal messages, with delivery tracking and proof of mailing, plus each party's contact preferences. In-app notifications to users live here too.

v3.2 Entire module. `communication` + `communication_recipient` reach any party (claimant, attorney, insured without a login), outbound and inbound, linked to the record it concerns.

v3.2 `notification` moved here from module 16 and now uses `template_version`.

v3.2 `communication_delivery` records every attempt, bounce and the proof of mailing that cancellation and non-renewal notices legally need.

v3.2 `contact_preference`: opt-in and language per party, channel and purpose.



communication			
uuid	communication_id	PK	NEW 3.2
varchar	direction		outbound inbound
varchar	channel		email sms letter portal phone
varchar	communication_type		cancellation_notice claim_letter quote invoice general
uuid	template_version_id		REF docs.template_version
uuid	document_id		REF docs.document; rendered body or letter
varchar	subject		
varchar	related_entity_type		
uuid	related_entity_id		
uuid	business_event_id		REF integration.business_event
uuid	sent_by_user_id		REF identity.tenant_user
varchar	status		
timestamptz	created_at		

notification			
uuid	notification_id	PK	
uuid	user_id		REF identity.tenant_user
uuid	business_event_id		REF integration.business_event
uuid	template_version_id		REF docs.template_version; CHG 3.2 was template_code
varchar	channel		in_app email sms
timestamptz	sent_at		
varchar	status		

communication_recipient			
uuid	recipient_id	PK	NEW 3.2
uuid	communication_id	FK	
uuid	party_id		REF party.party
uuid	user_id		REF identity.tenant_user
uuid	address_id		REF party.address
varchar	recipient_role		to cc bcc from
varchar	address_value		email or phone

contact_preference			
uuid	contact_preference_id	PK	NEW 3.2
uuid	party_id		REF party.party
varchar	channel		
varchar	purpose		service marketing legal_notice
varchar	language_code		
boolean	opted_in		
timestamptz	consented_at		
varchar	consent_source		

communication_delivery			
uuid	delivery_id	PK	NEW 3.2
uuid	recipient_id	FK	
varchar	provider		azure_comm_services mail_vendor
varchar	provider_message_id		
int	attempt_no		
varchar	delivery_status		queued sent delivered bounced opened failed
timestamptz	status_at		
uuid	proof_document_id		REF docs.document; certificate of mailing

21. Legal & regulatory matters

schema: compliance **schema: legal** **store: Domain SQL** Lawsuits, coverage disputes, E&O claims and regulatory actions as legal matters; legal holds that stop deletion; panel counsel and their invoices; the regulatory complaint register; and data-subject privacy requests.

v4.7 `legal_matter_outcome` can be scoped to one party (`matter_party_id`) or one linked record (`matter_link_id`): a matter can settle with one defendant and continue against another. "One current outcome" now applies per matter and scope.

v4.6 `legal_matter_outcome`: what actually happened (dismissed, settled, judgment, consent order, withdrawn, resolved), amount, non-monetary terms, the claim it settles, approval, payment, the agreement or order, and confidentiality. One current outcome per matter; an appeal supersedes rather than overwrites.

v4.5 `legal_matter_type`: configurable matter types, each with an attribute schema, intake journey, default workflow and number sequence. `legal_matter` gains type, site (org unit), intake session, stage and schema-validated `attributes_json` (the legal app's case `Data`).

v3.2 `legal_hold` + `legal_hold_item` stop disposal of documents, communications and records, overriding retention policy.

v4.6 `legal_matter_link`: a matter can relate to any number of claims, policies, contracts, complaints, accounts, billing transactions or other matters, each with a role (primary, subject, evidence, originating, financially responsible). It replaces `legal_matter.related_entity_type/id`; at most one link per matter is primary.

v4.6 `privilege_log`: documents or notes withheld or redacted in discovery, with date, author, recipients, privilege type and basis, as courts require.

v4.5 `legal_matter_party`: everyone on a matter with role and side (plaintiff, defendant, opposing counsel, court, judge, witness, expert), and who represents them (the legal app's `ContactParty.MatterIds`).

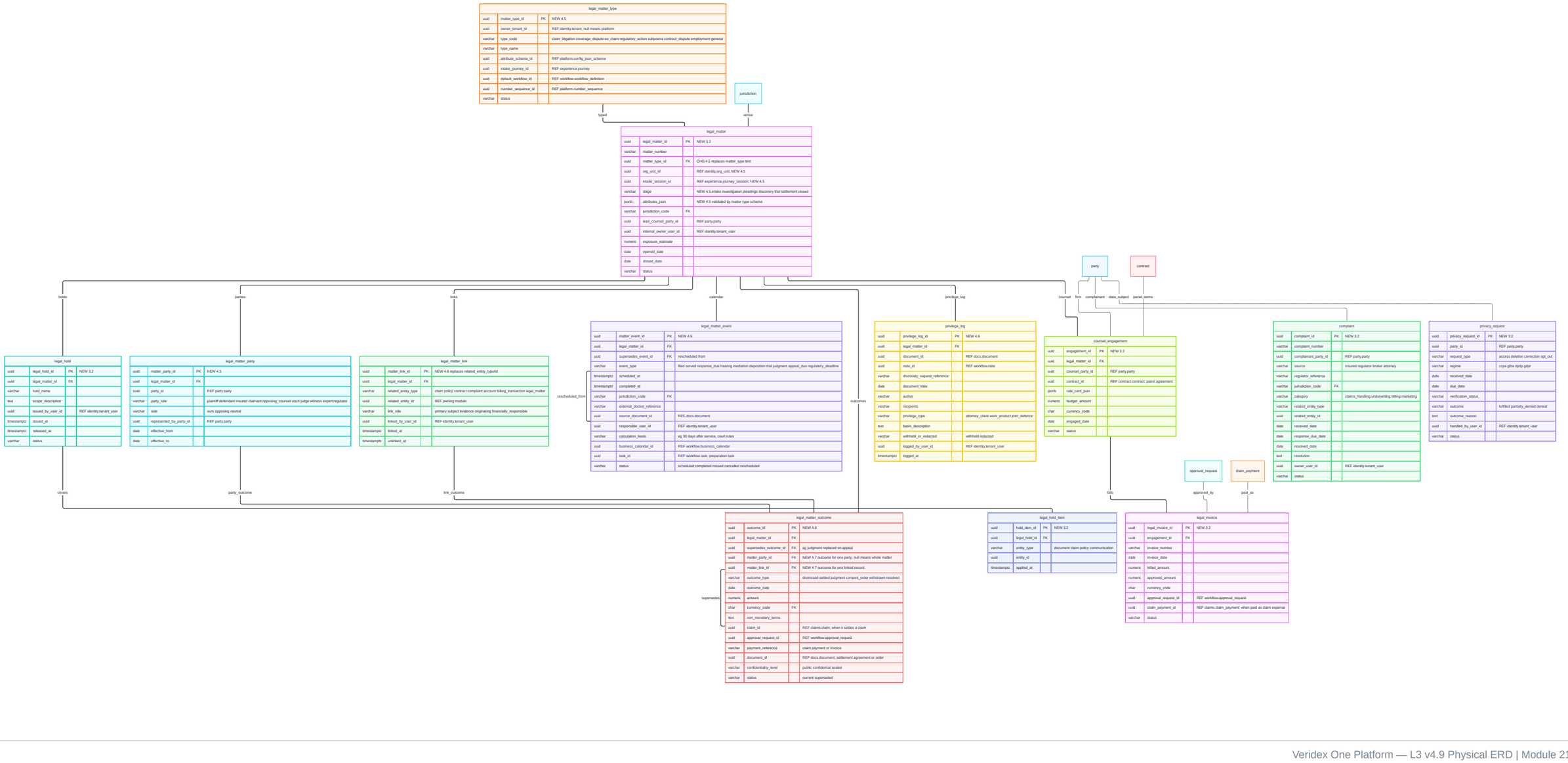
v3.2 `counsel_engagement` + `legal_invoice`: panel firms, budgets and invoice approval, paid through claims when it is claim expense.

v4.6 `legal_matter_event`: the legal calendar (filed, served, response due, hearing, mediation, deposition, trial, judgment, regulatory deadlines) with docket reference, source document, responsible person, how the date was calculated and on which business calendar. Rescheduling creates a new event that supersedes the old one; a preparation task can be linked, but the event, not the task, is the record of the date.

v4.5 Legal now owns a `legal` schema and is sellable on its own (Legal Matter Management): matters, holds, counsel and invoices. Complaints and privacy requests stay in `compliance`.

v3.2 Entire module. `legal_matter` covers every legal case; claim lawsuits keep their `litigation` detail and link to it.

v3.2 `complaint`: the regulatory complaint register with response deadlines; `privacy_request`: data-subject requests, which respect holds and retention.



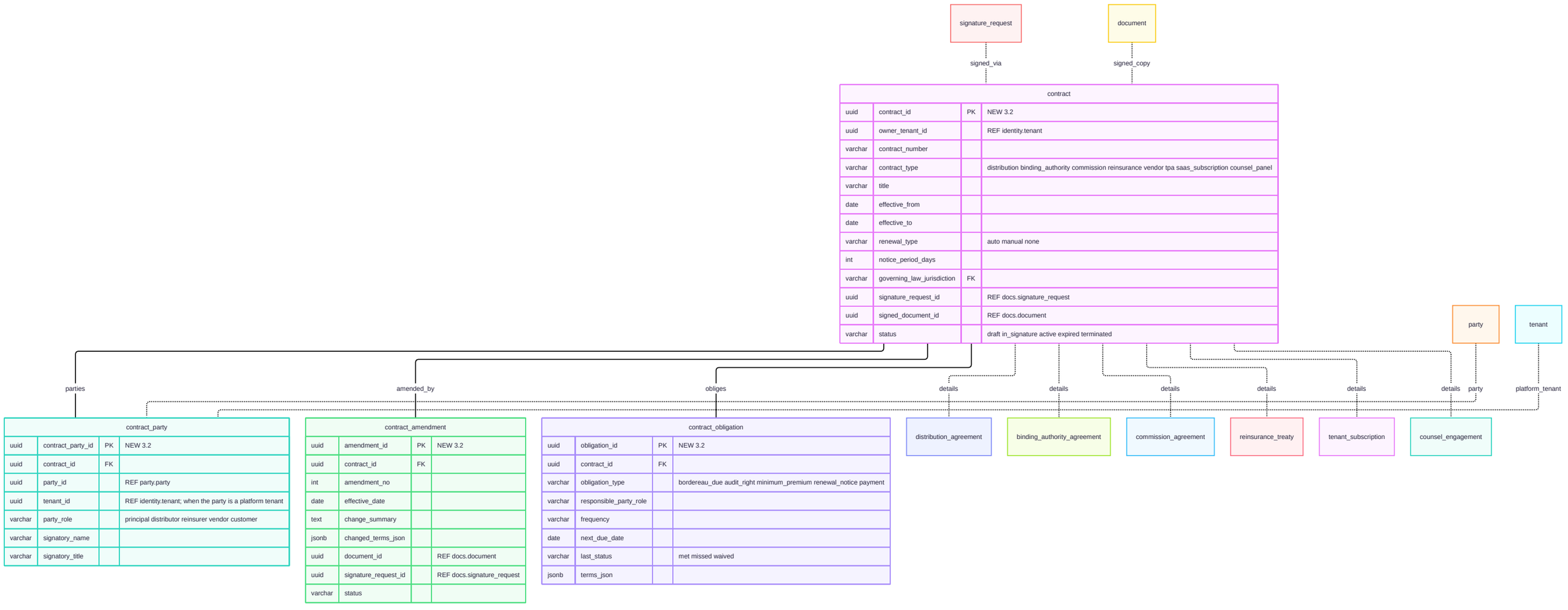
22. Contract management

schema: contract **store: Platform SQL** One contract record behind every agreement on the platform: distribution, binding authority, commission, reinsurance, vendor, TPA, counsel panel and the SaaS subscription itself, with parties, amendments and obligations.

v3.2 Entire module. `contract` gives every agreement one lifecycle: parties, term, renewal, notice period, governing law, signed copy and e-signature.

v3.2 `contract_amendment` keeps each amendment instead of overwriting dates; `contract_obligation` tracks due items such as bordereau deadlines and audit rights.

v3.2 Existing agreement tables keep their business detail and point at their `contract` through `contract_id`.



23. Platform services

schema: platform **store: Platform SQL** Scheduled jobs, bulk imports, per-tenant settings and feature flags, white-label branding and localised text.

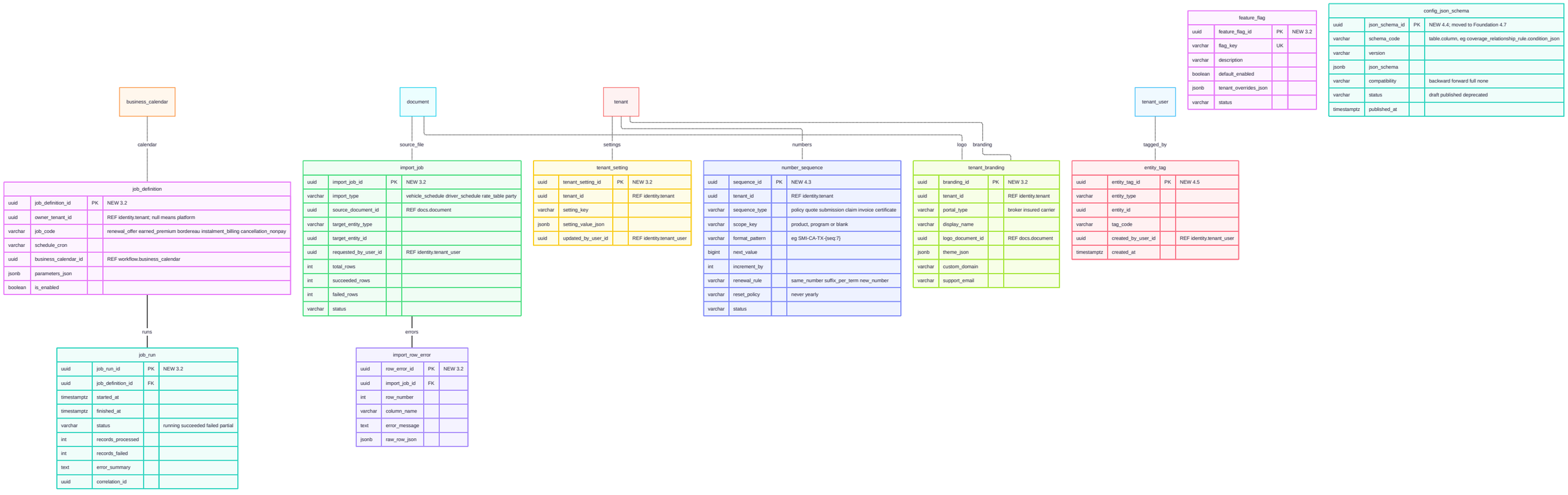
v4.7 `config_json_schema` moves here from Module 31: it governs JSON in every module, including Legal, so it belongs to Foundation. `entity_tag` moves here from Module 1. Localised text is now `localization_entry` in Module 1B.

v3.2 `import_job` + `import_row_error`: bulk uploads such as a fleet vehicle schedule, with row-level errors.

v4.3 `number_sequence`: per tenant, type and scope, the format and next value for policy, quote, submission, claim, invoice and certificate numbers, and the renewal rule (same number, suffix per term, or new number). Allocation takes a row lock so numbers are never duplicated.

v3.2 `tenant_setting`, `feature_flag`, `tenant_branding` and `localization_string`: per-tenant configuration, gradual rollout, white-label portals and multi-language text.

v3.2 Entire module. `job_definition` + `job_run`: scheduled batch work (renewals, earned premium, bordereaux, instalment billing, non-payment cancellations) on a business calendar.



24. Data intelligence & enrichment

schema: data **store: Domain SQL** The layer between raw inputs and the canonical model. Every value from a document, user, broker API, third-party provider, telematics feed, prior policy, derivation or AI model is an observation with source and confidence. Conflicts are discrepancies; the trusted value is a resolution. Third-party data is requested, received and kept as evidence.

v4.5 `external_record`: list-type outside data (FMCSA inspections and crashes, MVR violations, NHTSA recalls, DMV registrations) as one row per item with date, provider key, severity and normalised attributes, linked to the response and the vehicle, driver or party it concerns. Single values still go to `data_observation`.

v3.4 `data_resolution` keeps lineage: `supersedes_resolution_id`, `valid_from`, `valid_to` and a reason code, so 47 → 42 → 44 is a visible chain, not just a current flag.

v3.3 `data_resolution` records the trusted value, which observation it came from, how it was chosen and by whom. One resolution table serves both simple and conflicted values (the separate discrepancy_resolution table was folded in).

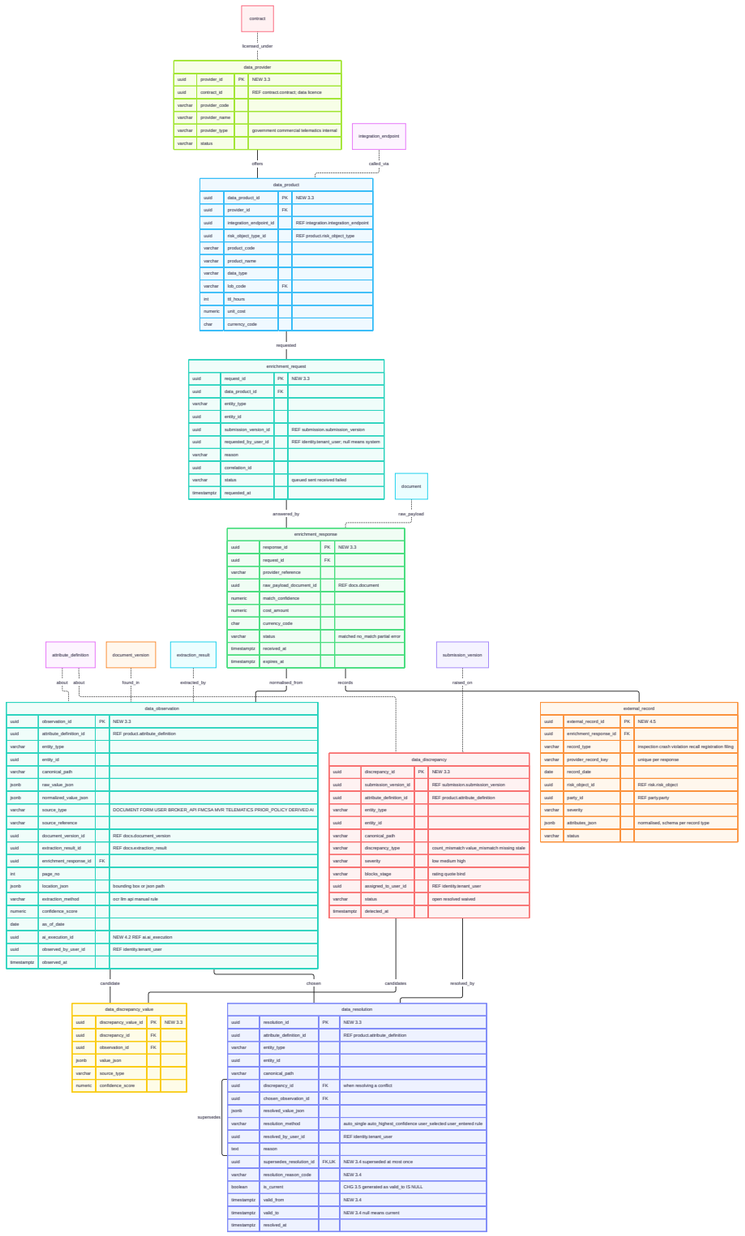
v4.2 `data_observation.ai_execution_id`: values extracted by AI point at the governed execution that produced them.

v3.3 Entire module. `data_observation` is the single, source-agnostic store of observed values, with source, document page and location, extraction method, confidence and as-of date. No per-provider observation tables.

v3.3 `data_provider`, `data_product`, `enrichment_request`, `enrichment_response`: provider registry, products with freshness and cost, and every request and raw response kept as evidence. Normalised values land as observations; scores land in `risk_score`.

v3.5 `data_resolution`: at most one current resolution per tenant, entity, attribute and canonical path; no overlapping validity periods for that key; `valid_to` after `valid_from`; `supersedes_resolution_id` unique so the chain cannot fork; `is_current` generated from `valid_to`. Rating reads the current row or the row valid at a point in time, and `rating_input.data_resolution_id` pins what was rated.

v3.3 `data_discrepancy` + `data_discrepancy_value` keep every conflicting candidate and can block rating, quote or bind until resolved.



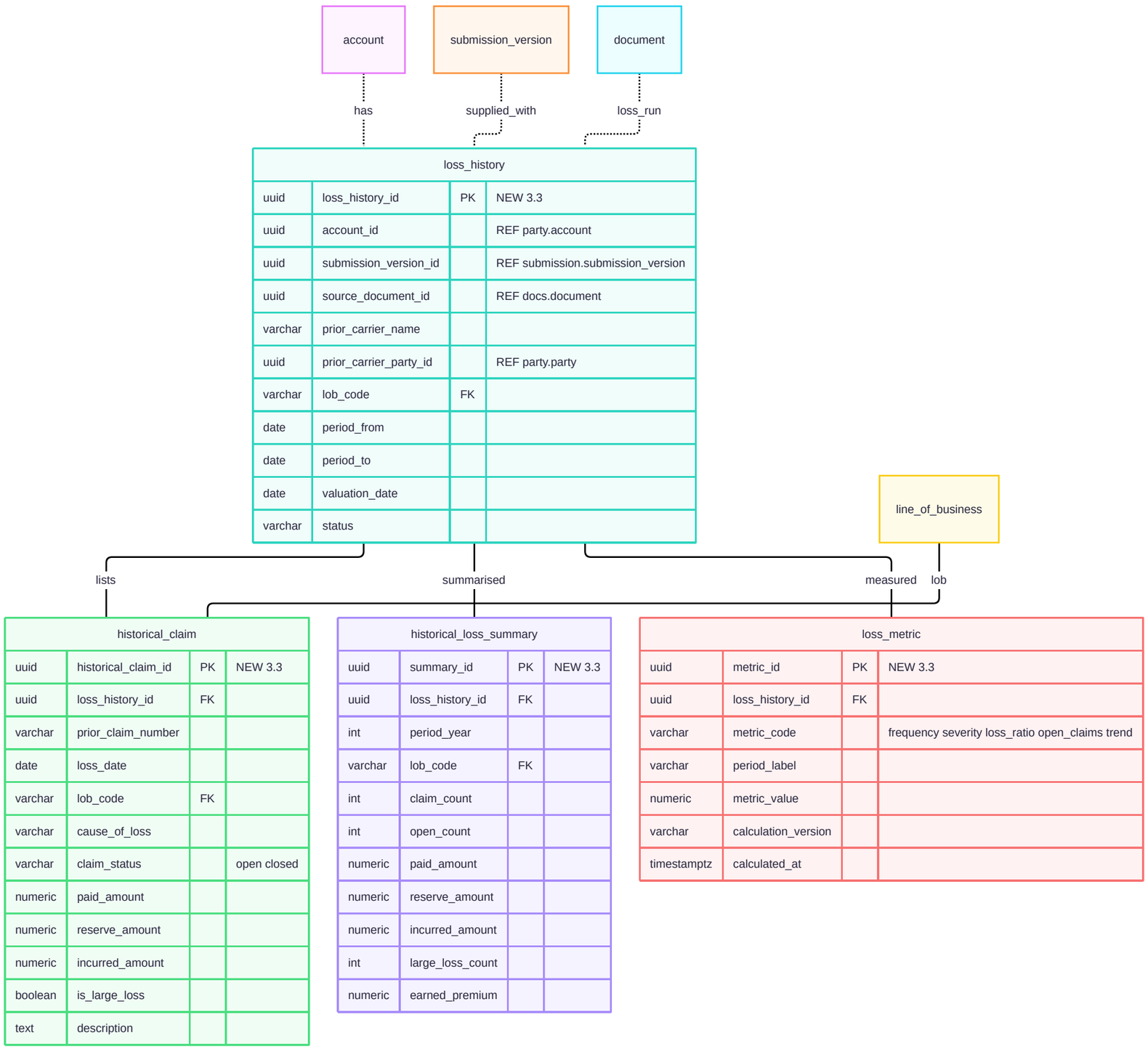
25. Prior loss history

schema: data store: Domain SQL Loss runs from previous insurers, normalised. These are observations about past insurance experience, kept apart from the operational claims the platform administers.

v3.3 Entire module. `loss_history` per prior carrier and period, with valuation date and the source loss-run document.

v3.3 `historical_claim` and `historical_loss_summary` hold normalised prior claims and yearly totals by LOB; large losses are a flag, not a separate table.

v3.3 `loss_metric` stores frequency, severity, loss ratio and trend with the calculation version, so the numbers underwriters see are reproducible. Values extracted from loss runs carry provenance through `data_observation`.

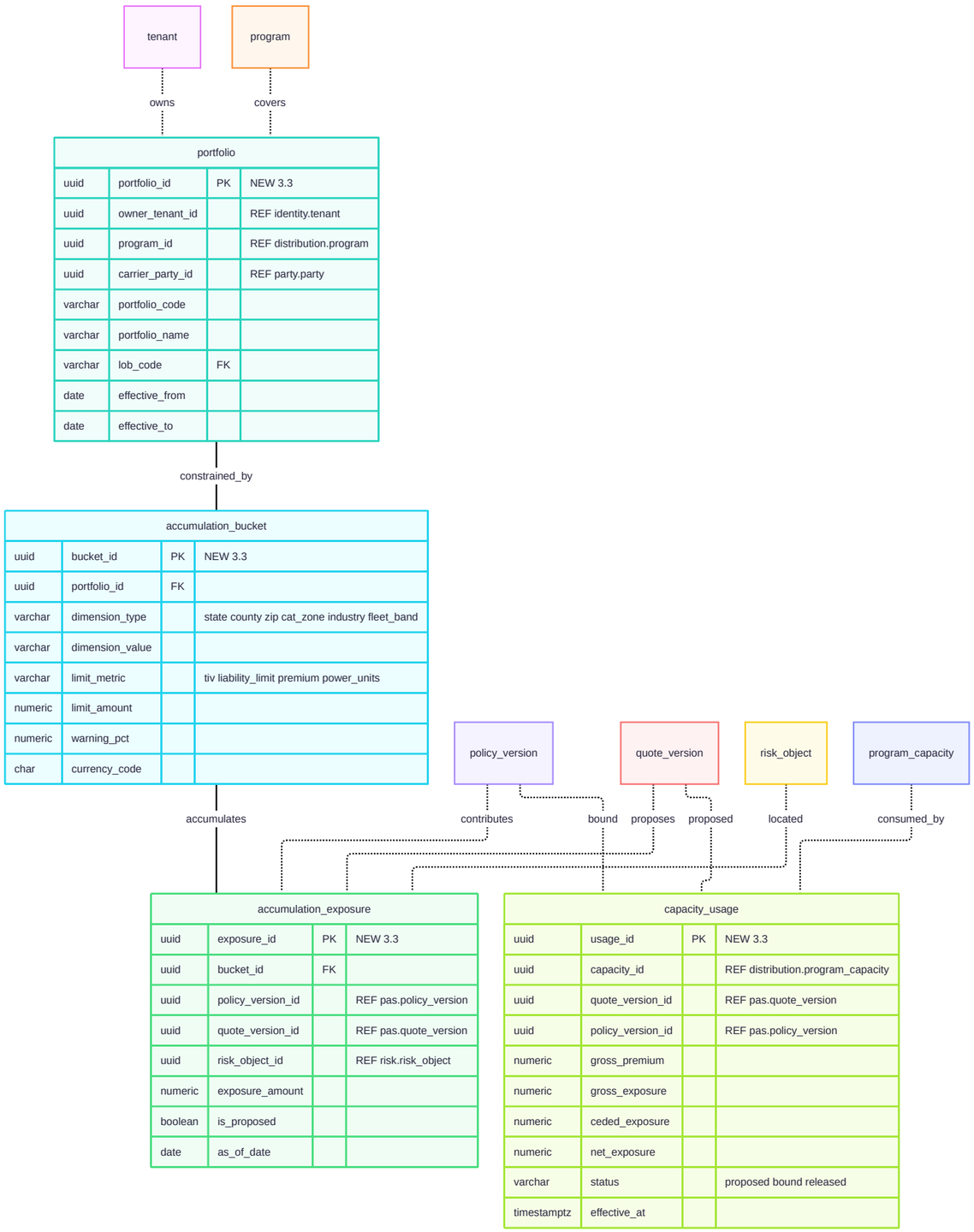


26. Portfolio & accumulation (planned)

schema: insight **store: Derived projections** **planned, not generally available** What writing one more risk does to the book: exposure by state, county, catastrophe zone, industry or fleet size, and how much delegated or carrier capacity a quote or policy consumes. Kept small for phase 2.

v3.3 Phase 2 module. `portfolio` and `accumulation_bucket` define where concentration is limited; `accumulation_exposure` adds existing and proposed exposure per risk.

v3.3 `capacity_usage` shows how much program or carrier capacity a quote proposes and a bound policy consumes, gross, ceded and net.



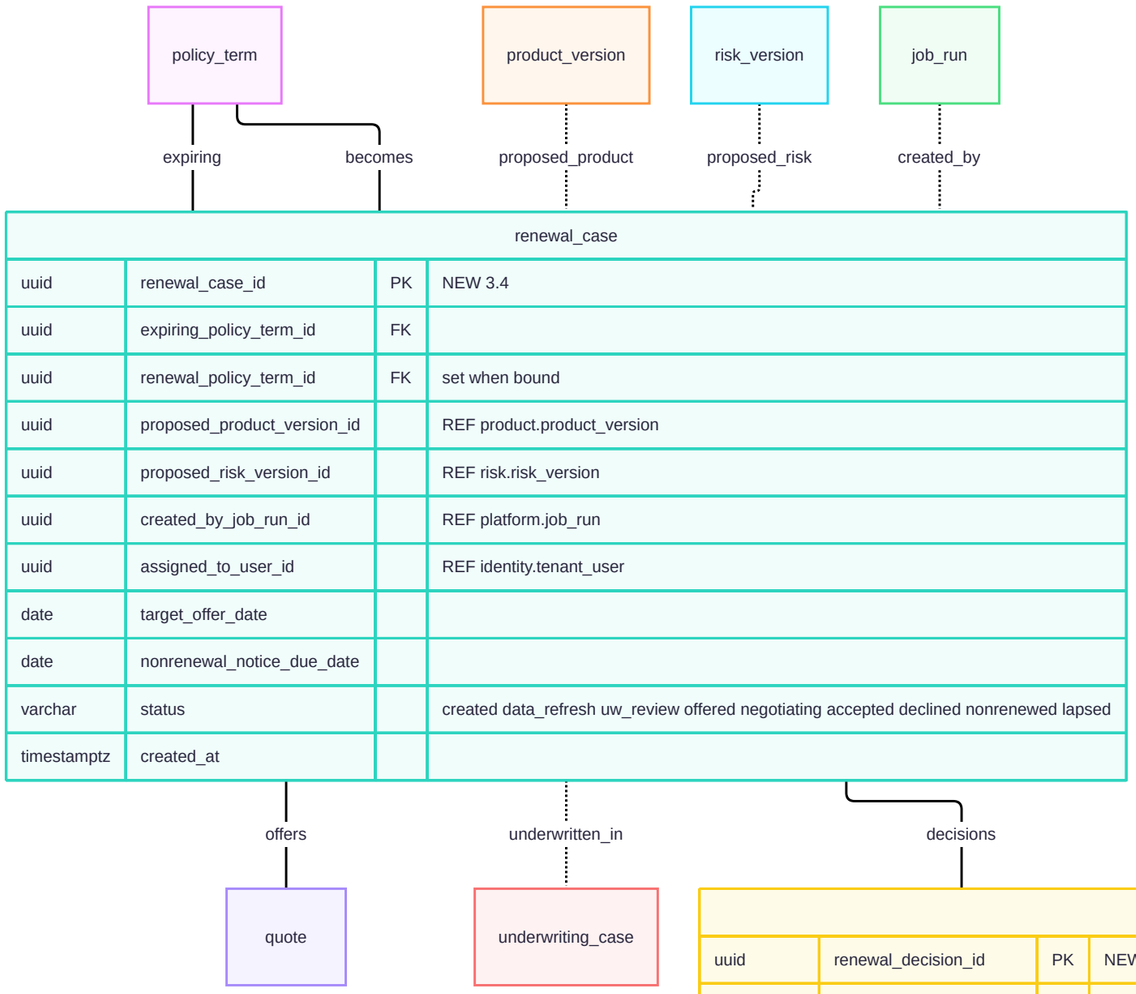
27. Renewal management

schema: pas store: Domain SQL Renewal as its own business process: created from the expiring term, refreshed with new risk data, loss experience and outside data, underwritten and re-rated, offered (possibly several times), then accepted, declined or non-renewed, ending in a new policy term.

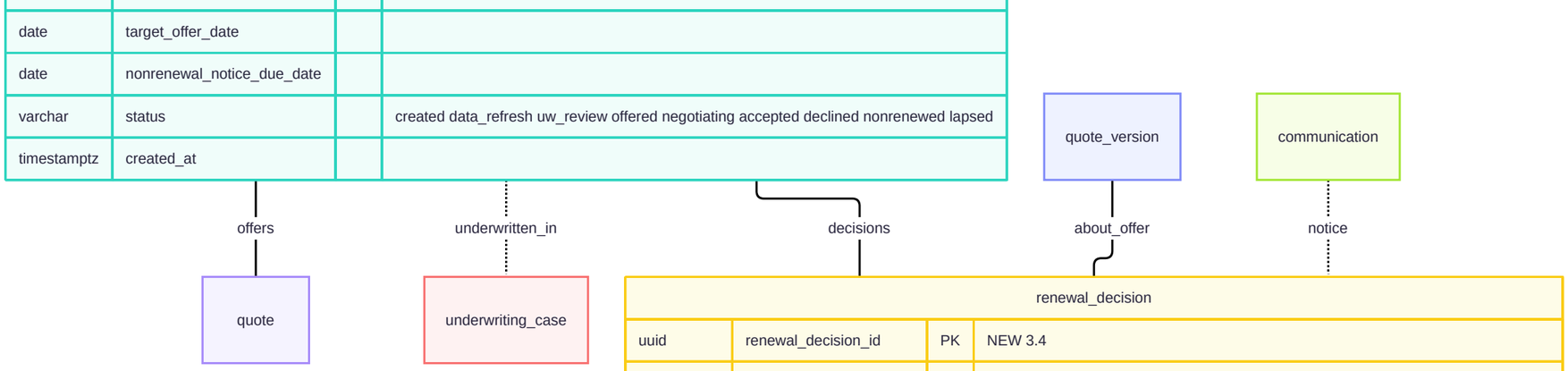
v3.4 Entire module. `renewal_case` runs from the expiring term to the new term, with the proposed product and risk versions and the non-renewal notice deadline.

v3.4 Offers are quotes (`quote.renewal_case_id`), so pricing, conditions and forms are reused; each counter-offer is a new quote version. The proposed separate renewal_offer and renewal_offer_version tables were not needed.

v3.4 `renewal_decision` records each decision in order: offer, insured declines or accepts, insurer non-renews, lapse, with reasons and the notice sent.



renewal_case			
uuid	renewal_case_id	PK	NEW 3.4
uuid	expiring_policy_term_id	FK	
uuid	renewal_policy_term_id	FK	set when bound
uuid	proposed_product_version_id		REF product.product_version
uuid	proposed_risk_version_id		REF risk.risk_version
uuid	created_by_job_run_id		REF platform.job_run
uuid	assigned_to_user_id		REF identity.tenant_user
date	target_offer_date		
date	nonrenewal_notice_due_date		
varchar	status		created data_refresh uw_review offered negotiating accepted declined nonrenewed lapsed
timestamptz	created_at		



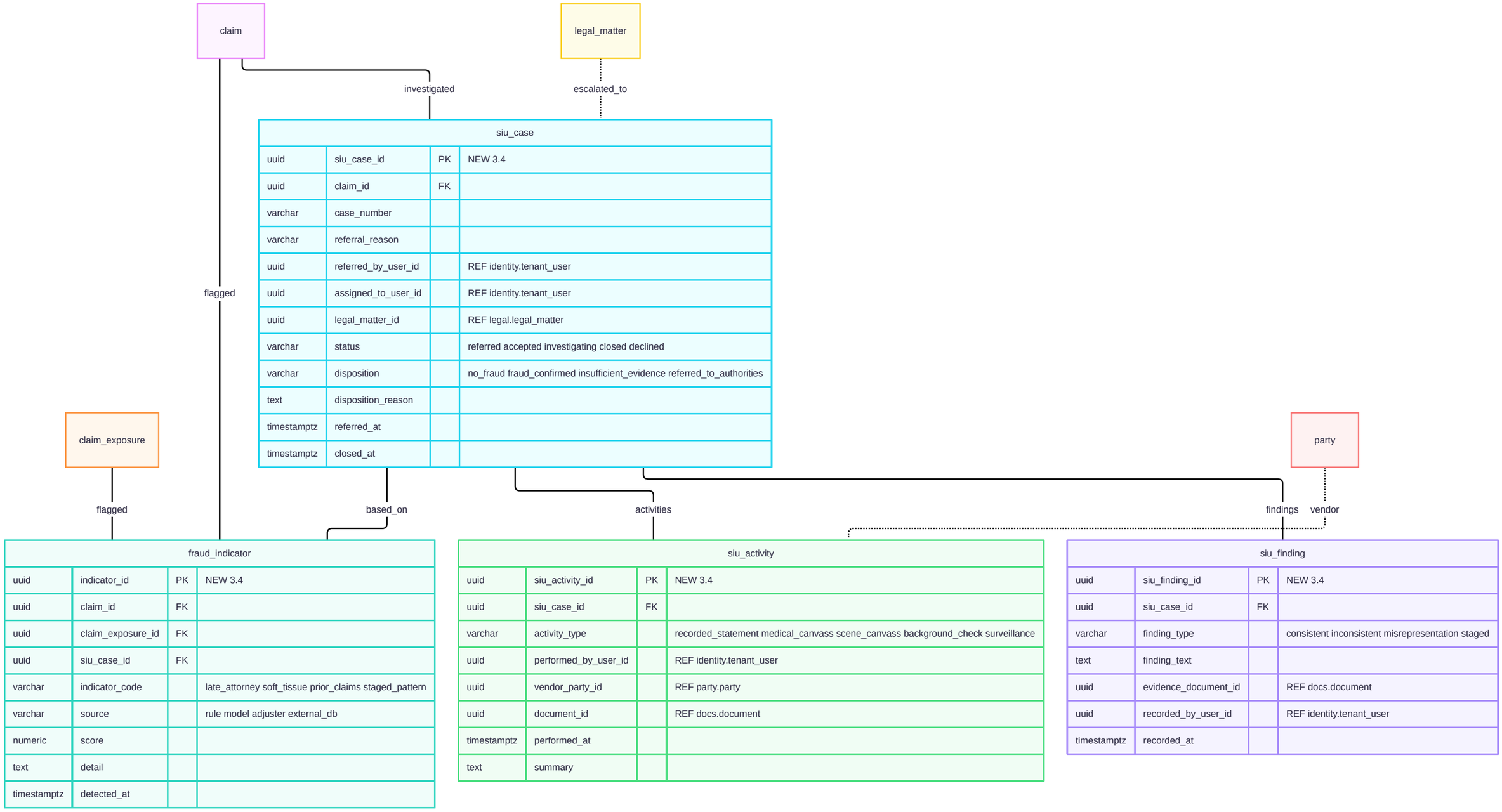
renewal_decision			
uuid	renewal_decision_id	PK	NEW 3.4
uuid	renewal_case_id	FK	
uuid	quote_version_id	FK	
varchar	decision_type		offer_renewal insured_declined_offer insured_accepted nonrenew_by_insurer lapsed
varchar	reason_code		
text	reason_text		
uuid	decided_by_user_id		REF identity.tenant_user
uuid	notice_communication_id		REF comms.communication
uuid	notice_document_id		REF docs.document
timestamptz	decided_at		

28. Fraud & special investigations

schema: claims **store: Domain SQL** Fraud indicators raised on claims and exposures, and the SIU case that investigates them, from referral to finding and disposition.

v3.4 Entire module. `fraud_indicator` stores each red flag with source and score; `siu_case` holds the referral, assignment, status and final disposition (the proposed separate siu_referral and disposition tables are fields of the case).

v3.4 `siu_activity` and `siu_finding` record the investigation and what it established, with documents and outside investigators.



29. Module catalog, provisioning & metering

schema: platform **store: Platform SQL** What Veridex sells and how each customer runs it: modules and their versions and dependencies, SKUs and prices, subscription lines, provisioned module instances (shared or dedicated), which system provides each capability a module needs, and usage metering.

v4.7 Module 29 is the single system of record for modules, features, SKUs, prices, subscriptions and entitlements. `tenant_subscription` and `tenant_module_entitlement` move here from Module 1; new `module_feature` and `tenant_feature_entitlement` record features (absent before) and `all_features_included`. Tenant Auth (Module 1A) and Authorization (Module 1B) hold projections and registrations under the same `module_id` and `feature_id`, never their own copies.

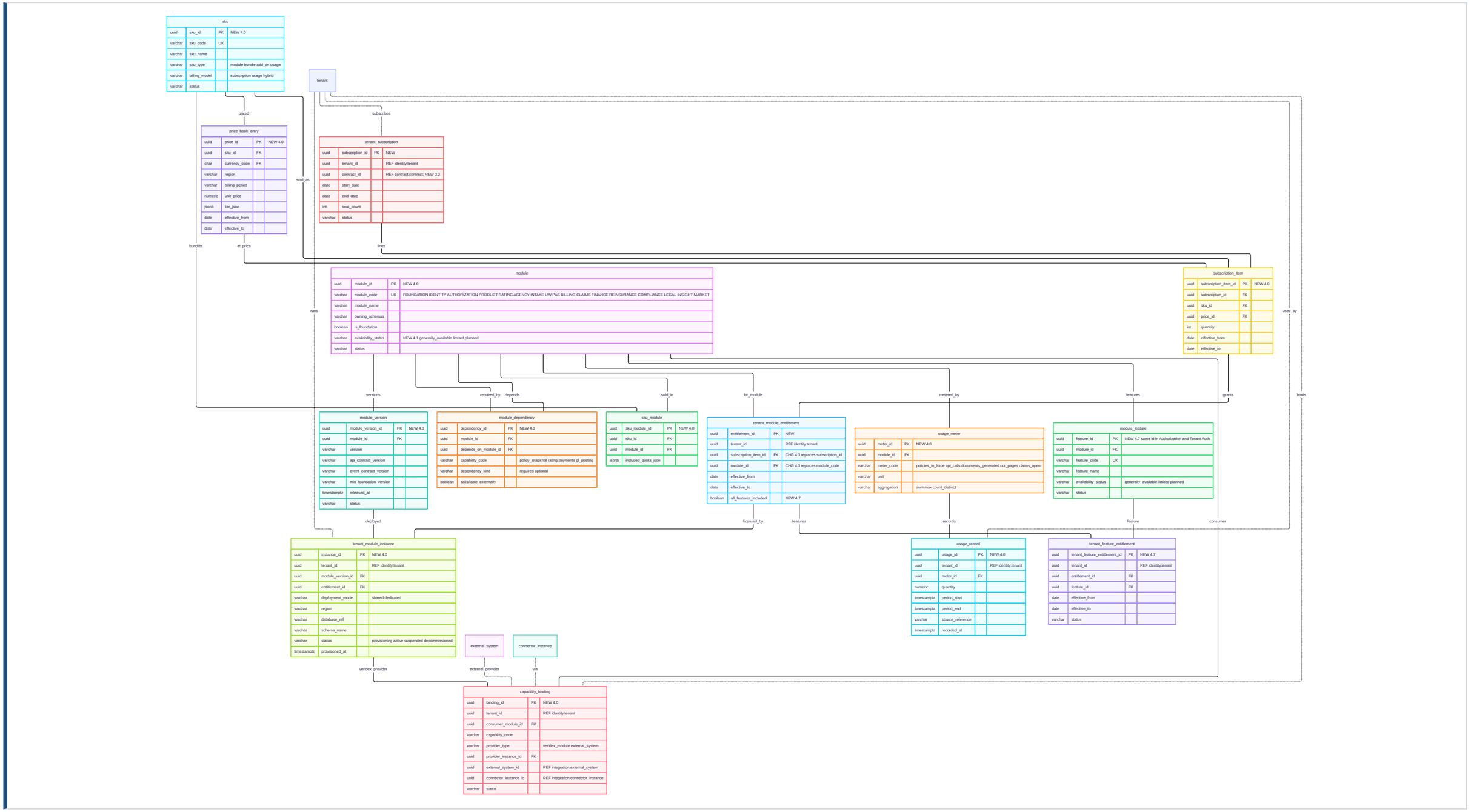
v4.1 `module.availability_status`: generally available, limited or planned. Reinsurance, Portfolio & Accumulation and Claims servicing are planned and not sold until complete.

v4.0 Entire module. `module`, `module_version` and `module_dependency` describe what can be sold and what each module needs; a dependency can be met by a Veridex module or, where allowed, an external system.

v4.0 `sku`, `sku_module`, `price_book_entry` and `subscription_item` turn modules into priced products; `tenant_subscription` now has lines.

v4.0 `tenant_module_instance`: each module provisioned for a customer, shared or dedicated, with its database and schema. `capability_binding` records, per customer, who provides each capability (our PAS or theirs, our GL or theirs).

v4.0 `usage_meter` + `usage_record`: usage-based charging (policies in force, API calls, OCR pages, documents).



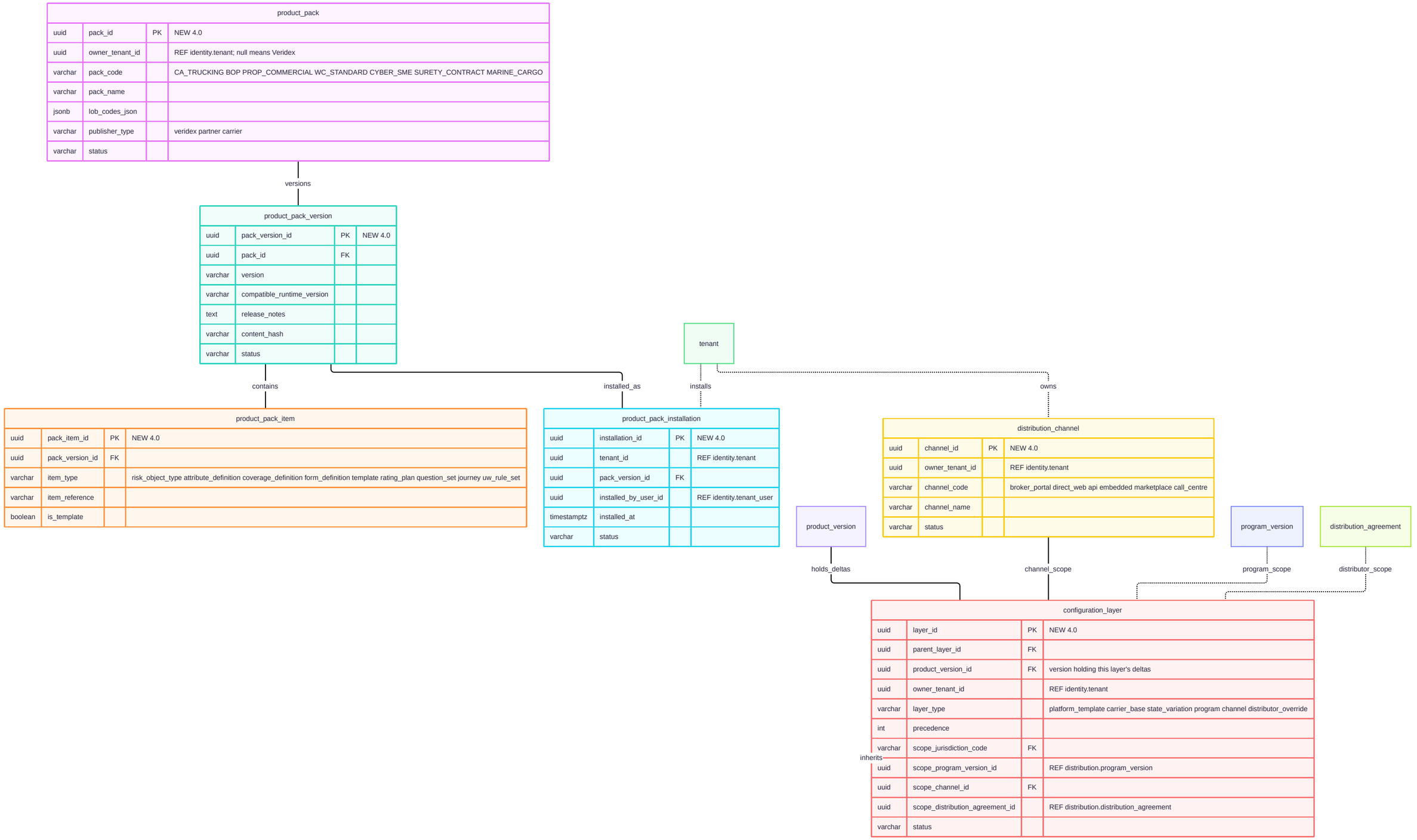
30. Product packs, configuration layers & channels

schema: product **store: Domain SQL** Lines of business arrive as installable product packs, not schema changes. Configuration is resolved through ordered layers: platform template, carrier base, state variation, program, channel and distributor override.

v4.0 Entire module. `product_pack`, `product_pack_version`, `product_pack_item` and `product_pack_installation`: a line of business (trucking, BOP, property, workers comp, cyber, surety, marine) is a versioned bundle of object types, governed attributes, coverages, forms, templates, rating and rule templates and journeys. New lines need no schema change; typed tables are kept only for stable core lines.

v4.0 `configuration_layer`: ordered layers, platform template → carrier base → state variation → program → channel → distributor override, each holding its deltas in a product version and limited by `override_permission`.

v4.0 `distribution_channel`: broker portal, direct web, API, embedded, marketplace and call centre as a configuration scope.



31. Product Runtime, testing & publishing

schema: product **store: Domain SQL** One runtime resolves the right configuration for every portal and module. A product release is the compiled, immutable result of all layers for a scope; it is tested, simulated, approved, published, rolled back and retired as a unit, and every transaction records the release it ran on.

v4.7 `config_json_schema` moved to Module 23 (Foundation), so modules other than Product Studio can use it.

v4.4 `config_json_schema`: a versioned JSON Schema for every configuration JSON column (conditions, scopes, algorithm steps, rules), with compatibility rules. Compiling a release validates every JSON value against its published schema; failures become `release_validation_issue` rows and block publishing. JSON is for configuration only, never for financial movements or policy history.

v4.0 Entire module. `product_release` + `product_release_item`: the compiled, hashed, immutable set of versions the runtime executes; `release_scope` says where and when it applies (state, program, channel, distributor, new or renewal), with no overlaps.

v4.0 `release_validation_issue`: dependency, filing, broken-reference and rule-conflict checks block publishing.

v4.0 `test_suite`, `test_case`, `test_run`, `test_result`, `rate_simulation` and `impact_analysis`: regression tests, book-of-business rate simulations and impact analysis before approval.

v4.0 Publishing, rollback (re-publishing a superseded release) and retirement act on releases; quotes, policies, submissions and rating requests record `product_release_id`.

32. Digital journeys (Foundation)

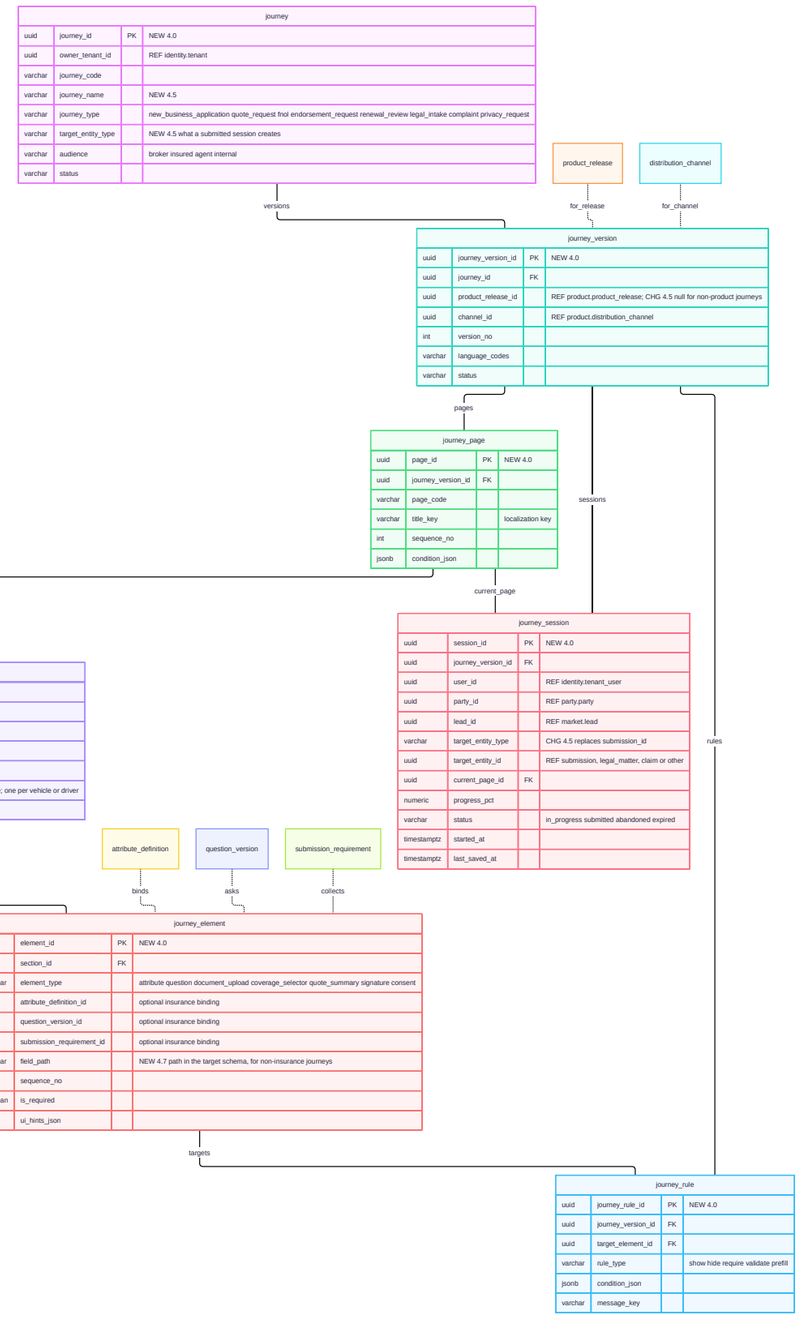
schema: experience **store: Platform SQL** Application, quote, FNOL and service journeys configured as pages, sections and elements bound to governed attributes, questions and submission requirements, with show, hide, require and validate rules, instead of coded screens per product.

v4.7 Journeys move to a Foundation schema, `experience`, so Legal and any standalone module can use them without Product Studio. Product Studio keeps only optional insurance bindings (product release, attribute definitions, questions, submission requirements); `journey_element.field_path` binds a field to any target schema.

v4.0 `journey_rule`: show, hide, require, validate and prefill rules; `journey_session` saves an application in progress. Answers are stored as `data_observation` rows (source USER), so journeys add no separate answer store.

v4.5 Journeys are no longer insurance-only: `journey.target_entity_type` and `journey_session.target_entity_type/id` say what a submitted session creates (a submission, a legal matter, a claim, a complaint). Non-product journeys have no product release. This replaces the legal app's `Form` and `Submission`.

v4.0 Entire module. `journey` to `journey_element`: pages, sections (repeatable per vehicle or driver) and elements bound to governed attributes, questions, document requirements, coverage selection, signatures and consent; text through localization keys.



33. Agency management

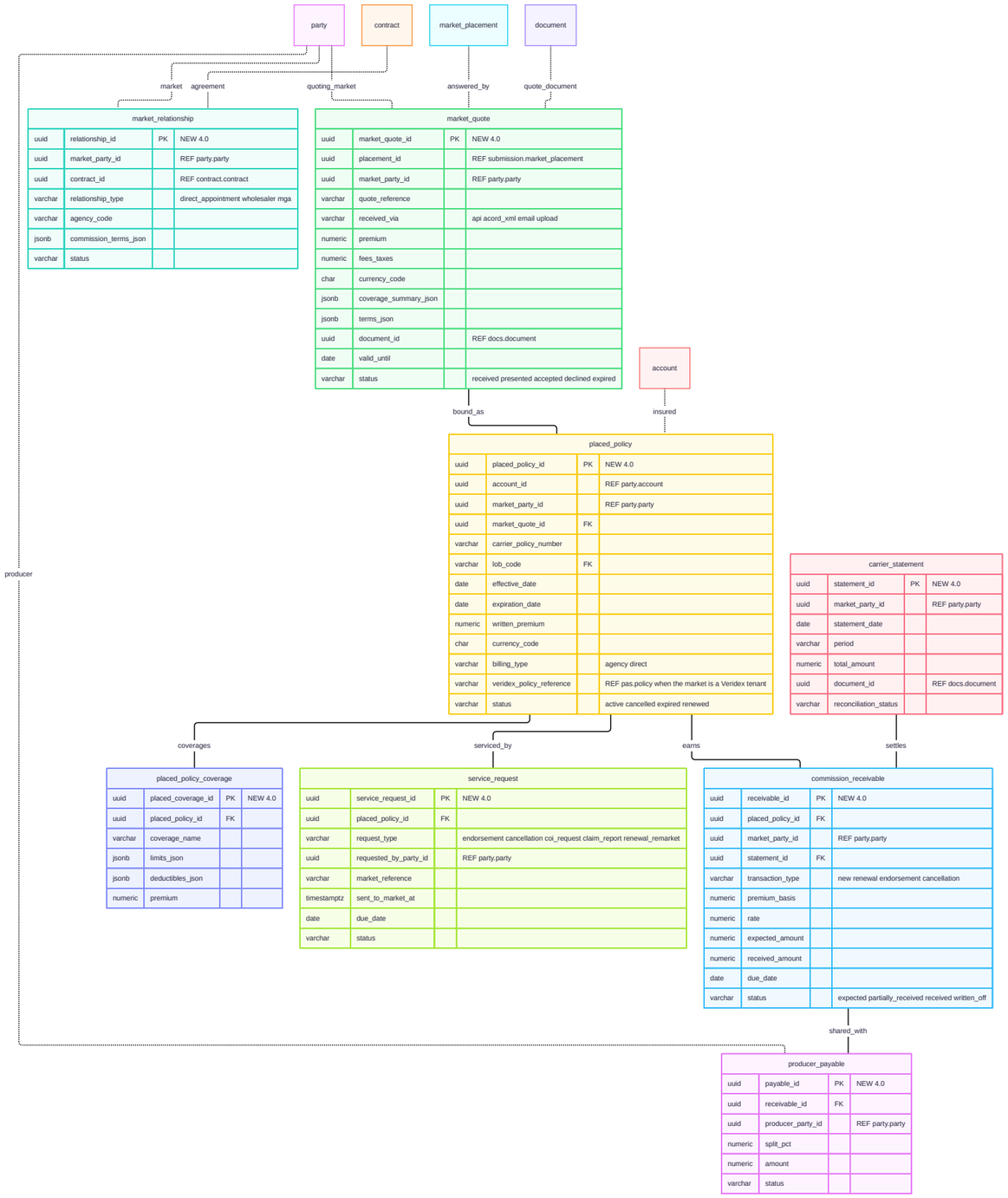
schema: agency **store: Domain SQL** For brokers and agencies whose policies sit with outside carriers and MGAs: market relationships, quotes received, the placed book of business, service requests to markets, commission receivable, carrier statements and producer pay.

v4.1 `certificate_of_insurance` moved to the generic certificate model in module 43, which covers both placed and Veridex-issued policies.

v4.0 `commission_receivable`, `carrier_statement` and `producer_payable`: commission the agency expects and receives from markets, reconciled to carrier statements and shared with producers.

v4.0 Entire module, sold with the Broker / Agency Portal. Submissions and placements reuse the submission schema; `market_quote` captures what outside markets send back.

v4.0 `placed_policy` + `placed_policy_coverage`: the agency's book on other carriers' paper. `service_request` tracks endorsement, cancellation, certificate and remarketing requests to markets.

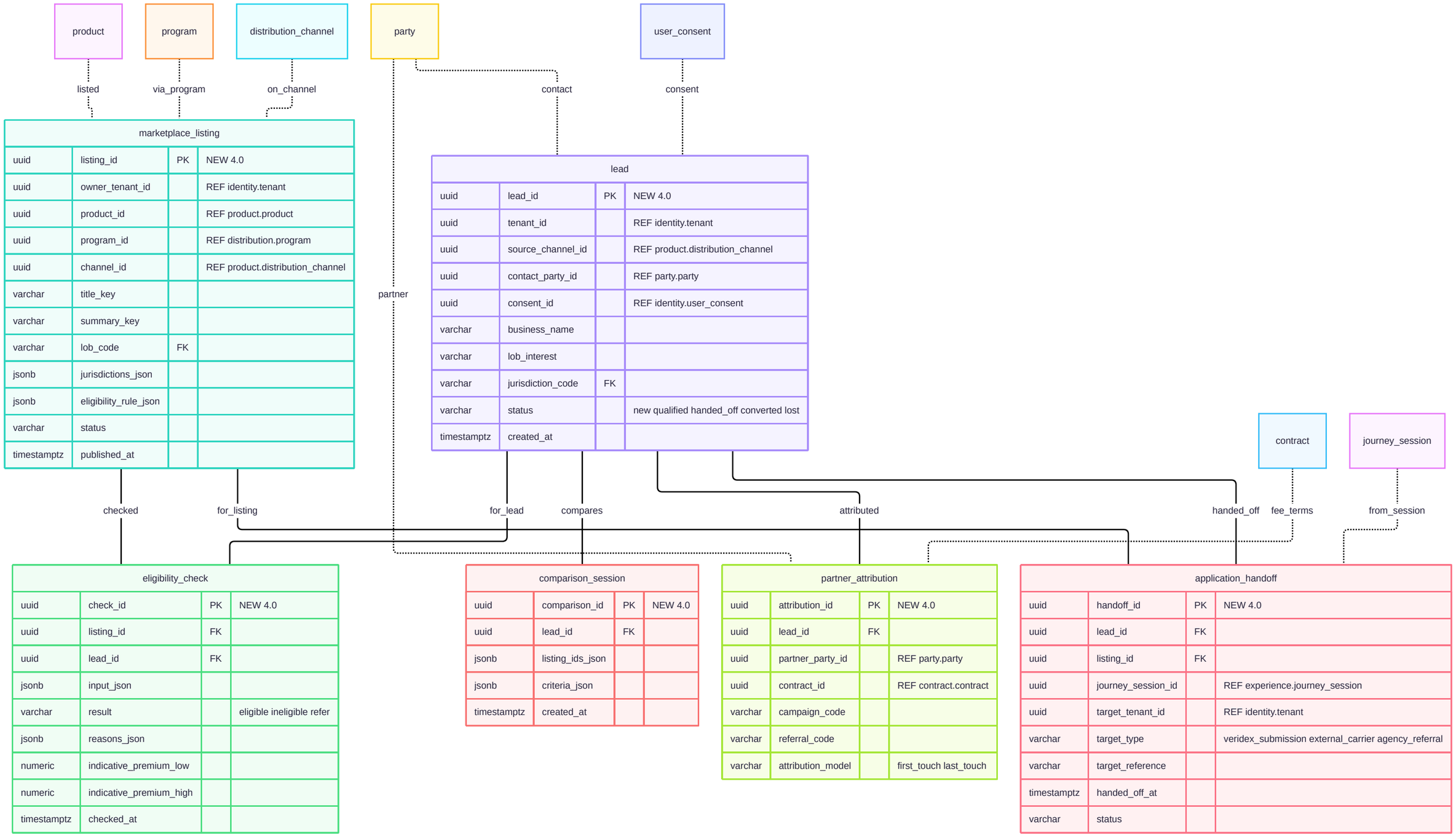


34. Marketplace

schema: market **store: Domain SQL** Product discovery for insureds and partners: listings, eligibility preview, comparison, lead capture, partner attribution and handoff into an application, a submission or an external carrier.

v4.0 Entire module. `marketplace_listing` publishes products and programs on a channel with eligibility rules; `eligibility_check` gives a preview and indicative premium range; `comparison_session` records side-by-side comparisons.

v4.0 `lead`, `partner_attribution` and `application_handoff`: consented leads, the partner and campaign that brought them, and the handoff into a Veridex submission, an agency or an external carrier.



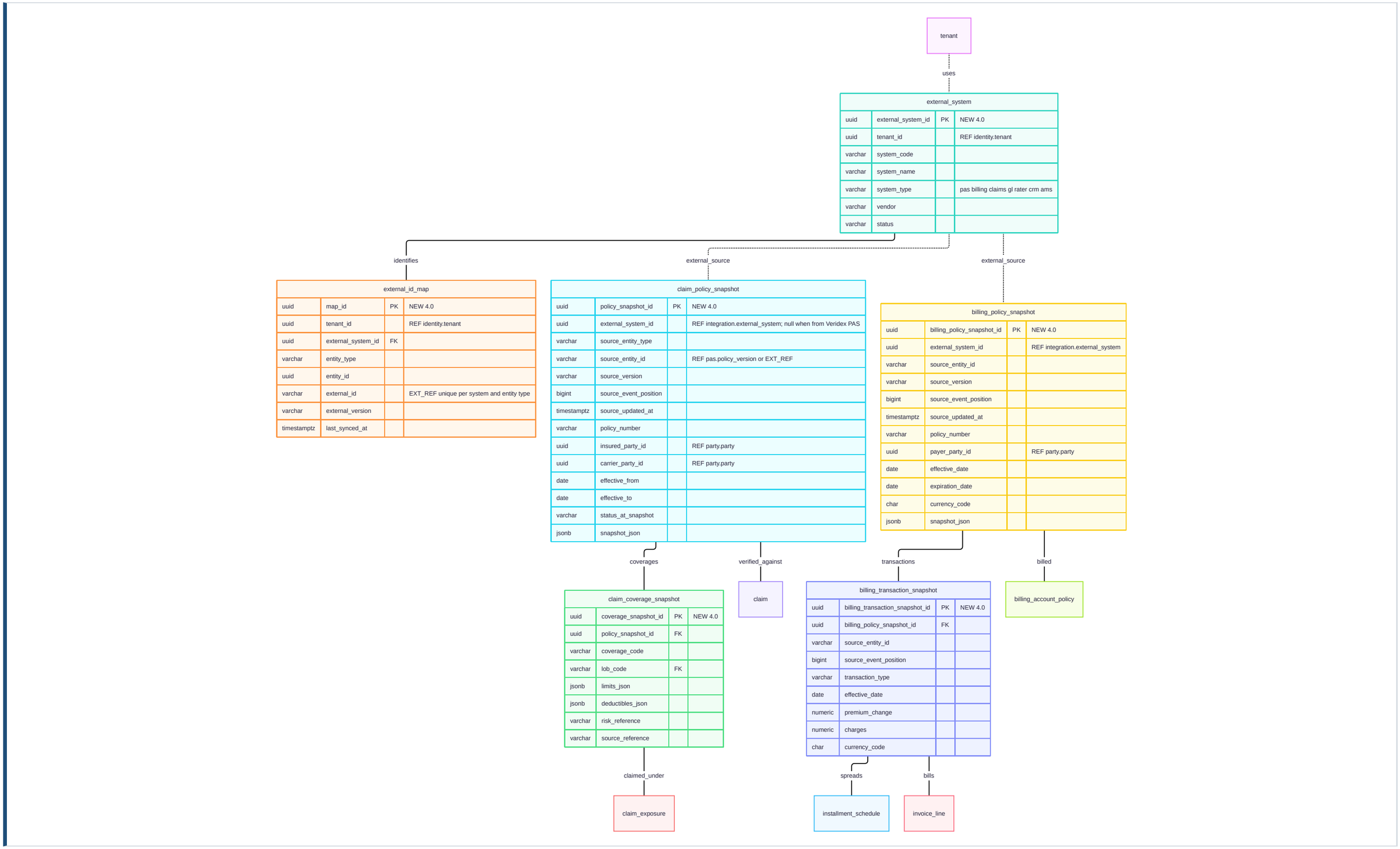
35. Standalone operation: snapshots & external references

schema: billing **schema: claims** **schema: integration** **store: Domain SQL** **store: Platform SQL** What lets Claims, Billing and other modules run without the Veridex PAS. Each consuming module keeps its own snapshot of the policy data it needs, filled from our modules by events or from the customer's systems through connectors, and every outside record is mapped to its external id.

v4.0 Entire module. `external_system` + `external_id_map`: every customer system and the mapping between their ids and ours, unique per system and entity type.

v4.0 `claim_policy_snapshot` + `claim_coverage_snapshot`: Claims verifies coverage against its own snapshot, filled from the Veridex PAS or the customer's PAS. `claim` and `claim_exposure` point at the snapshot instead of PAS tables.

v4.0 `billing_policy_snapshot` + `billing_transaction_snapshot`: Billing invoices from its own snapshot of policy transactions. Every snapshot records its source, version, event position and freshness.



36. Integration contracts, messaging & migration

schema: integration **store: Platform SQL** How modules and customer systems talk safely: versioned API and event contracts, subscriptions, per-module outbox and inbox giving at-least-once delivery with idempotent consumers, so business effects happen effectively once, webhooks, certified connectors, data migration and reconciliation.

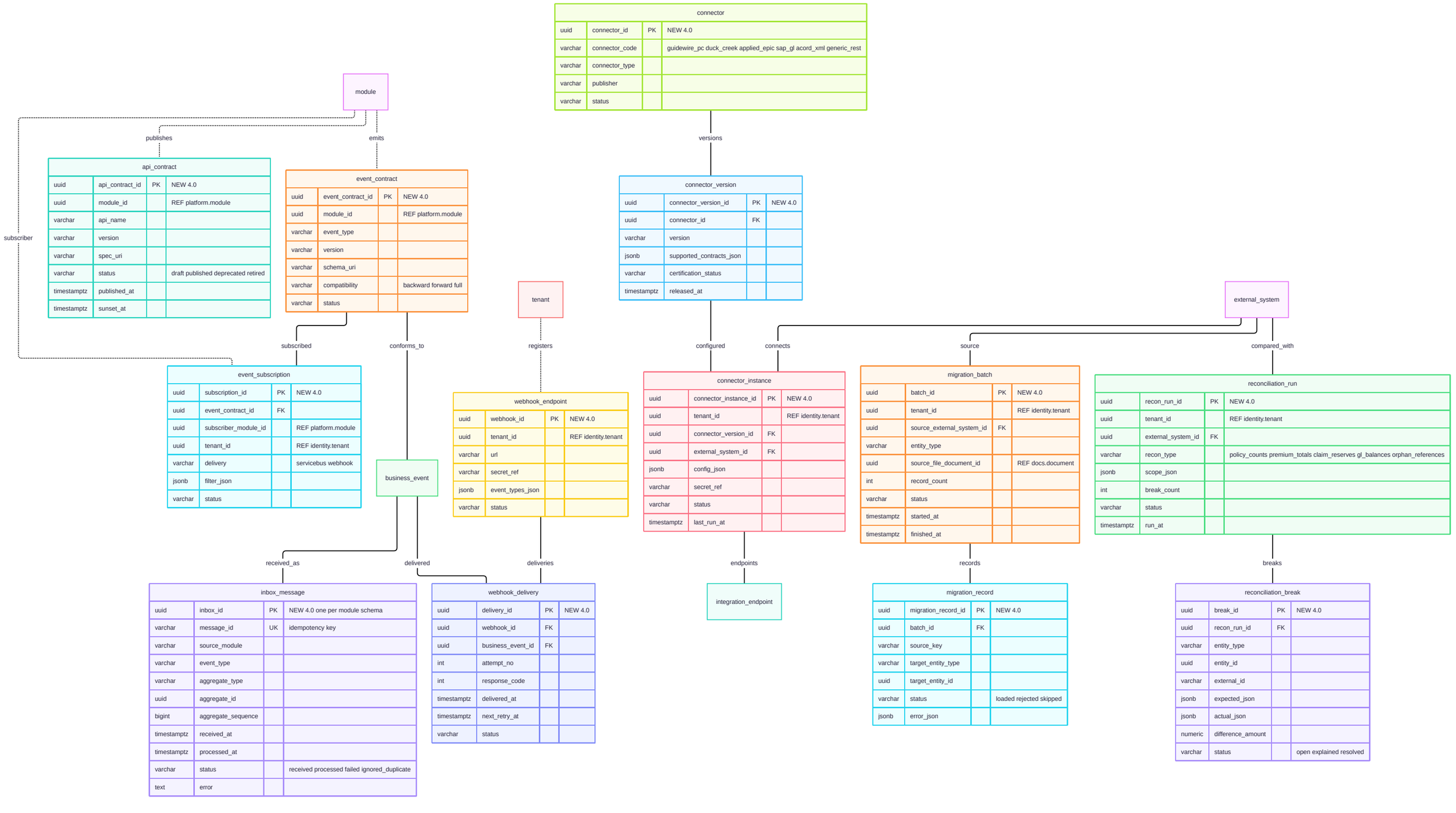
v4.1 Delivery guarantee stated precisely: at-least-once delivery with idempotent consumers and effectively-once business effects. Outbox and inbox cannot guarantee true exactly-once delivery across every broker, connector and webhook.

v4.0 `webhook_endpoint` + `webhook_delivery`, and `connector`, `connector_version`, `connector_instance`: customer webhooks and certified connectors to systems such as Guidewire, Duck Creek, Applied Epic or SAP.

v4.0 Entire module. `api_contract` and `event_contract`: versioned, published contracts with compatibility rules and sunset dates; `event_subscription` records who consumes what.

v4.0 `migration_batch`, `migration_record`, `reconciliation_run` and `reconciliation_break`: conversion with row-level results, and reconciliation against source systems, including orphaned cross-module references.

v4.0 `business_event` is each module's outbox and now carries `aggregate_sequence`, `global_position` and its contract; `inbox_message` in each module makes processing idempotent (a duplicate message id is ignored).



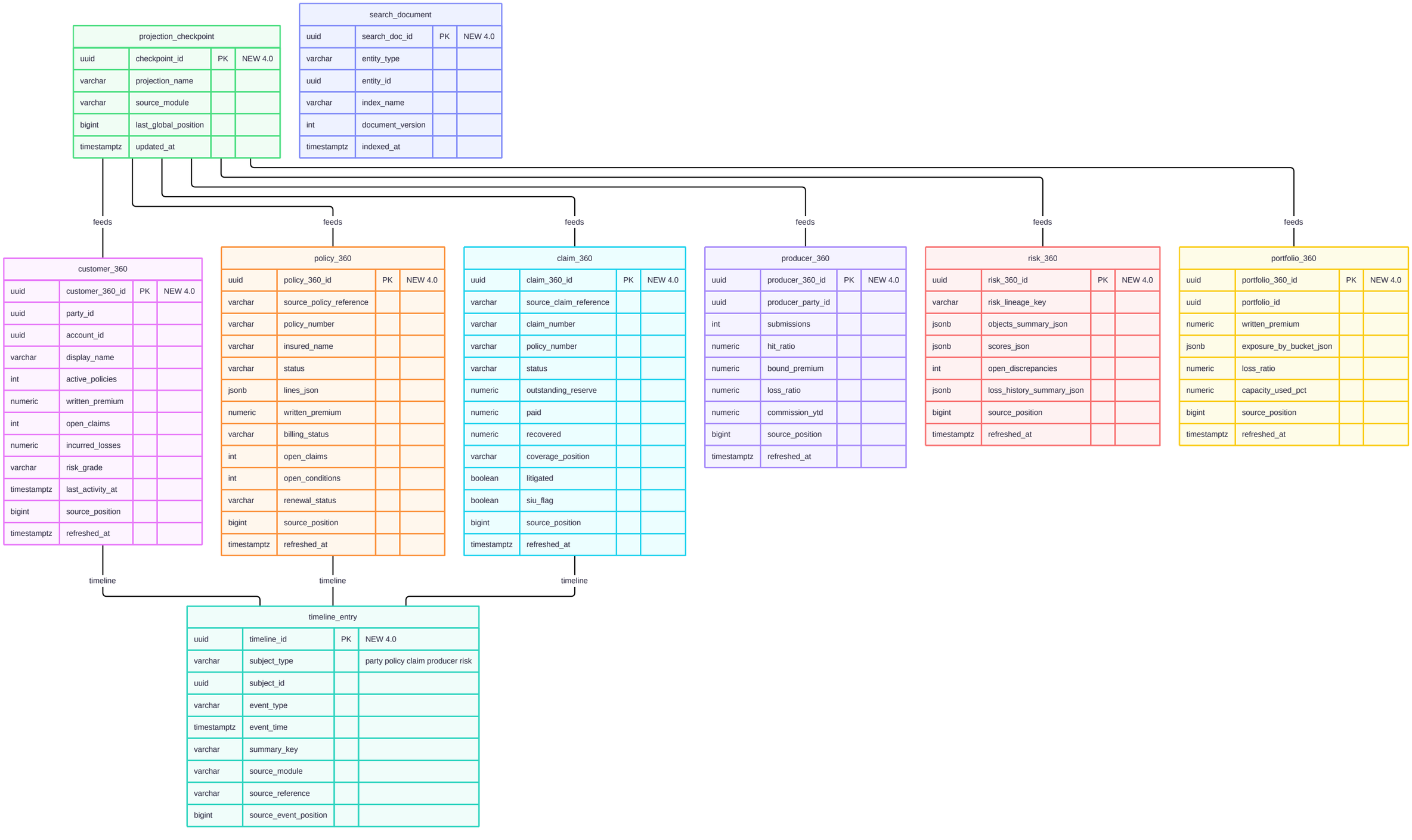
37. Insight: 360 views, timeline & search

schema: insight **store: Derived projections** Read models built from events for Customer, Policy, Claim, Producer, Risk and Portfolio 360 views, one timeline across all modules, and the registry of what is in the search index. Rebuildable at any time from events; never written by transactions directly.

v4.0 Entire module (insight schema, with portfolio from module 26). Six 360 projections, each recording the last event position it reflects; `projection_checkpoint` tracks progress per projection so any view can be rebuilt from events.

v4.0 `timeline_entry`: one chronological feed per customer, policy, claim, producer or risk across every module. `search_document` registers what is in the search index (for example Azure AI Search) for unified search.

v4.0 Projection ids are plain values, not foreign keys: projections depend on events, never on module tables.



38. Extensions & content governance

schema: platform **store: Platform SQL** Customers extend the platform through governed plugins at declared extension points, never through code forks; and the platform records whether content is a Veridex template or carrier-authored, and who attested responsibility for it.

v4.0 Entire module. `extension_point`, `plugin`, `plugin_version` and `plugin_installation`: signed, certified, sandboxed plugins installed per customer at declared points; the only code-level extension allowed.

v4.0 `content_attestation`: before a Veridex template, form, rating plan or product pack goes live for a carrier, the carrier records that it has adopted or adapted the content and is responsible for its contractual language, rates, filings and regulatory approval. `template.content_origin` and `form_definition.content_origin` mark where content came from.

party

publishes

responsible_party

tenant

attests

tenant_user

by

content_attestation

uuid	attestation_id	PK	NEW 4.0
uuid	tenant_id		REF identity.tenant
varchar	content_type		template form rating_plan rule_set product_pack coverage_wording
varchar	content_reference		
varchar	content_origin		veridex_template carrier_authored carrier_adapted
uuid	responsible_party_id		REF party.party
uuid	attested_by_user_id		REF identity.tenant_user
varchar	attested_role		
varchar	attestation_text_version		
timestamptz	attested_at		

plugin

uuid	plugin_id	PK	NEW 4.0
uuid	publisher_party_id		REF party.party
varchar	plugin_code		
varchar	plugin_name		
varchar	plugin_type		rule_function connector ui_widget document_function
varchar	status		

module

declares

versions

installs

extension_point

uuid	extension_point_id	PK	NEW 4.0
uuid	module_id	FK	
varchar	point_code		before_bind after_rate custom_validation document_merge claim_triage
varchar	contract_schema_uri		
varchar	execution_mode		sync async
int	timeout_ms		

plugin_version

uuid	plugin_version_id	PK	NEW 4.0
uuid	plugin_id	FK	
varchar	version		
varchar	package_uri		
varchar	content_hash		
varchar	signature_ref		
varchar	certification_status		submitted certified rejected revoked
timestamptz	certified_at		
varchar	sandbox_profile		

at

installed

plugin_installation

uuid	installation_id	PK	NEW 4.0
uuid	tenant_id		REF identity.tenant
uuid	plugin_version_id	FK	
uuid	extension_point_id	FK	
jsonb	config_json		
boolean	enabled		
uuid	installed_by_user_id		REF identity.tenant_user
timestamptz	installed_at		

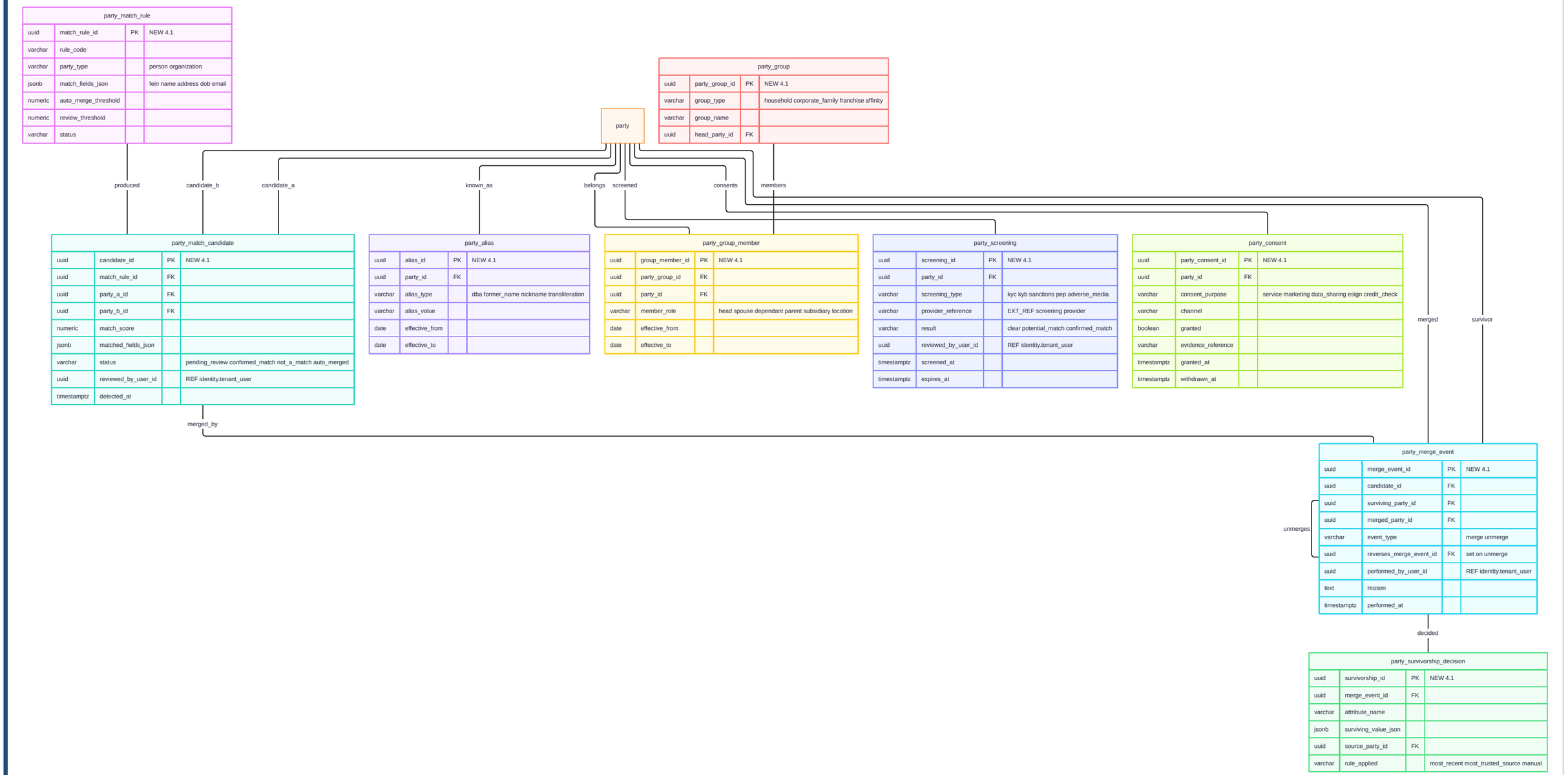
39. Party master data

schema: party **store: Platform SQL** Mastered identities for every person and organisation: duplicate matching, merge and unmerge with history, survivorship of attribute values, aliases, households and business groups, KYC/KYB and sanctions screening references, and consent held by the party rather than by a platform login.

v4.1 Entire module (party schema). `party_match_rule` + `party_match_candidate`: duplicate detection with review and auto-merge thresholds.

v4.1 `party_merge_event` + `party_survivorship_decision`: merges and unmerges kept as history, with which value survived and why. `party` gains `mdm_status` and `merged_into_party_id`; a merged party is never deleted.

v4.1 `party_alias`, `party_group`, `party_group_member`: alternate names, households and business families. `party_screening`: KYC/KYB, sanctions and PEP references. `party_consent`: consent held by the party, separate from `user_consent` on platform logins.



40. Billing operations

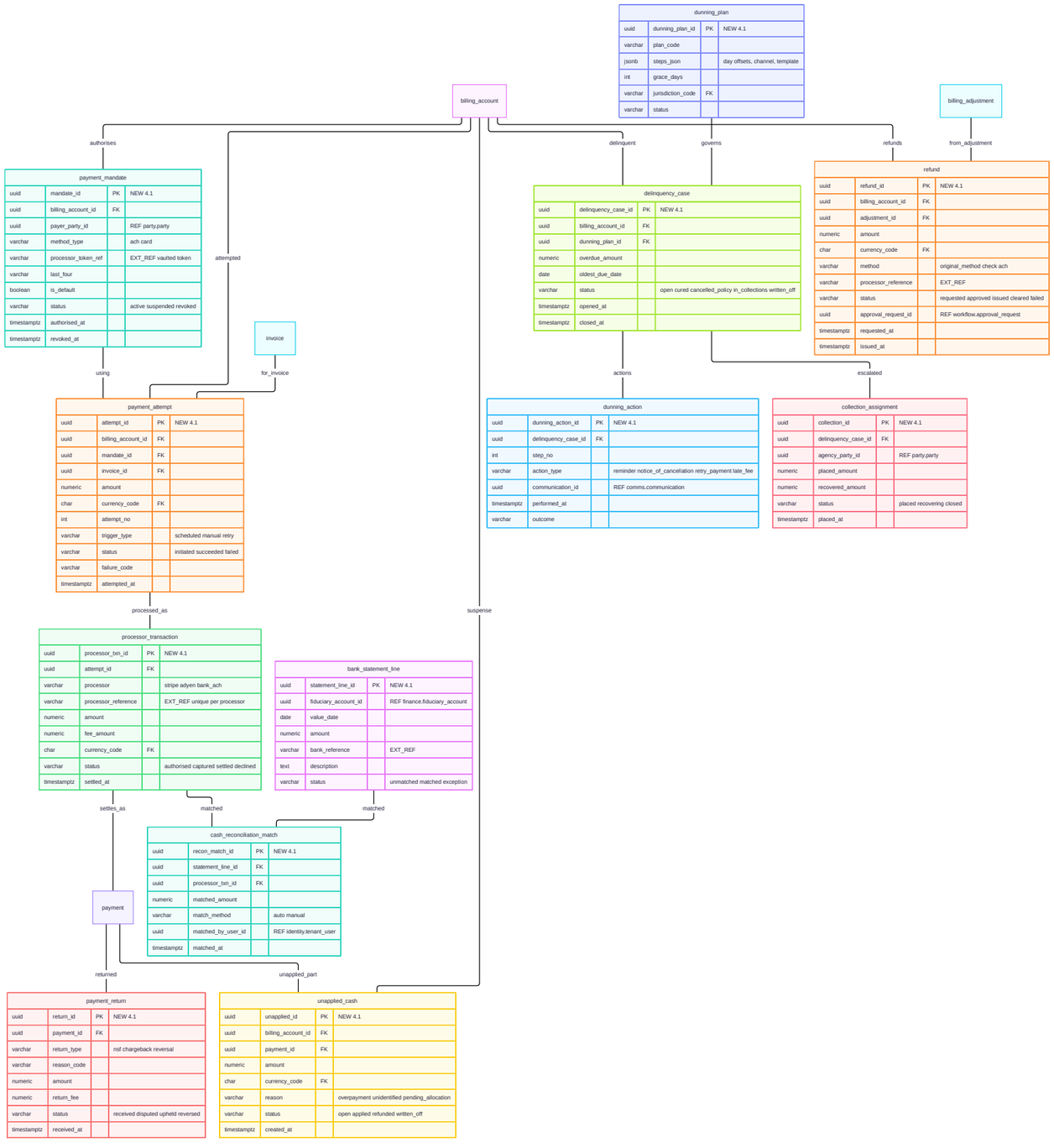
schema: billing **store: Domain SQL** How money is actually collected: stored payment mandates, every payment attempt and processor transaction, returns such as NSF and chargebacks, unapplied cash, delinquency and dunning, collections, bank reconciliation and the refund lifecycle.

v4.1 Entire module (billing schema). `payment_mandate`, `payment_attempt` and `processor_transaction`: stored authority to charge, every attempt including retries, and what the processor did.

v4.1 `bank_statement_line` + `cash_reconciliation_match`: processor and bank reconciliation. `refund`: requested, approved, issued and cleared, with approval.

v4.1 `payment_return`: NSF, chargebacks and reversals; `unapplied_cash`: overpayments and unidentified cash held in suspense until applied, refunded or written off.

v4.1 `dunning_plan`, `delinquency_case`, `dunning_action` and `collection_assignment`: delinquency from first reminder to collections, with every notice sent through Communication.



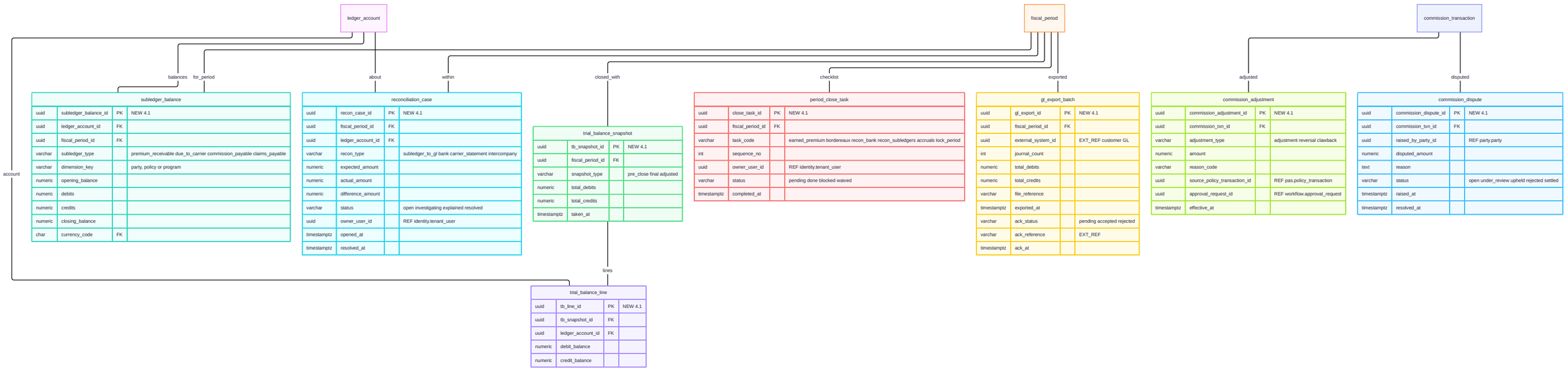
41. Finance close & commission adjustments

schema: finance **store: Domain SQL** What Finance & Accounting needs to be sold on its own: subledger balances, reconciliation cases, trial-balance snapshots, the period-close checklist, GL export with acknowledgement, and commission adjustments, reversals, clawbacks and disputes.

v4.1 Entire module (finance schema). `subledger_balance`, `reconciliation_case`, `trial_balance_snapshot` + `trial_balance_line` and `period_close_task`: a controlled month-end close.

v4.1 `gl_export_batch`: exports to the customer's GL with totals and the acknowledgement received back.

v4.1 `commission_adjustment` (adjustment, reversal, clawback on cancellation) and `commission_dispute`: commission corrections never overwrite the original transaction.

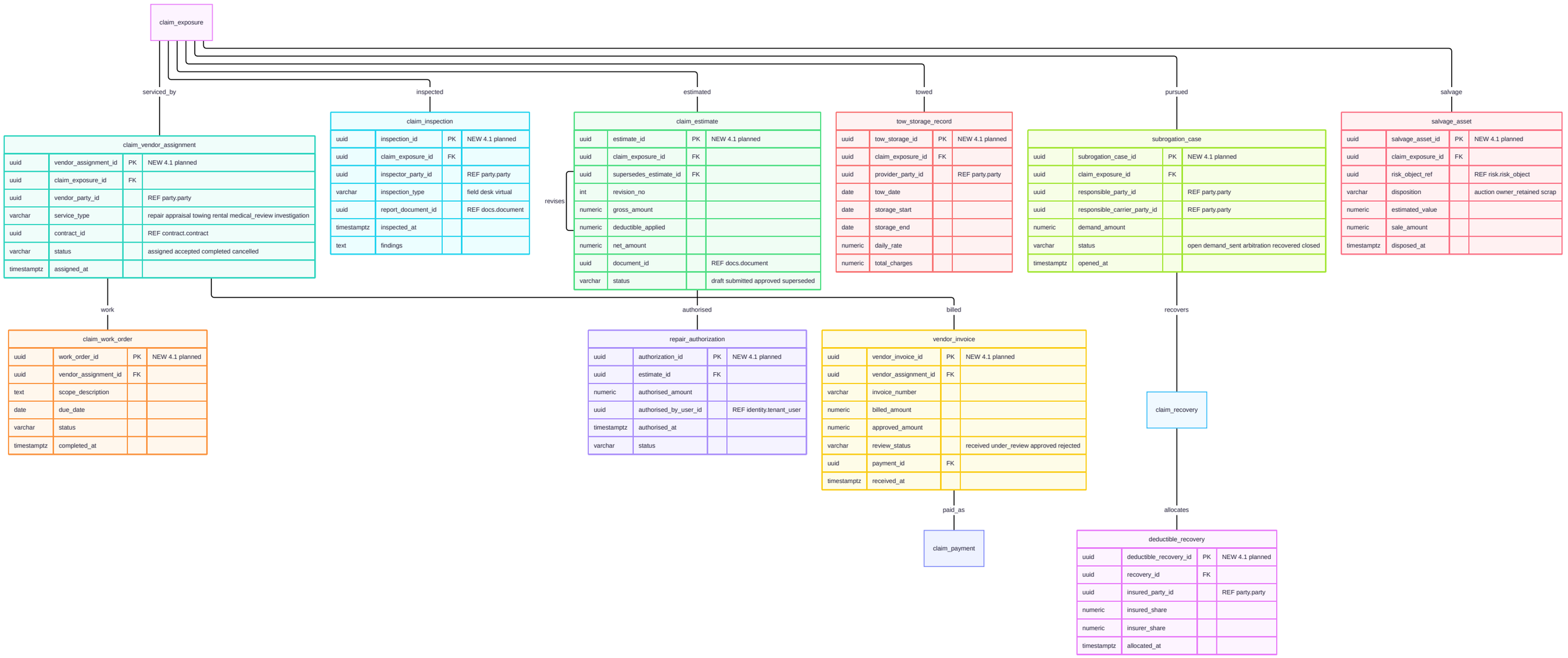


42. Claims servicing (planned)

schema: claims **store: Domain SQL** **planned, not generally available** The target model for claims servicing, planned after the first Claims release: vendor assignment, work orders, inspections, estimates and revisions, repair authorisation, towing and storage, vendor invoices, a structured subrogation case, salvage, and deductible recovery allocation.

v4.1 Planned module (claims schema), shown as the target model so the first Claims release does not block it. Vendor assignment, work orders, inspections, estimates with revisions and repair authorisation.

v4.1 Towing and storage, vendor invoice review (paid as a `claim_payment`), a structured `subrogation_case` feeding `claim_recovery`, salvage disposition, and allocation of recoveries between insured deductible and insurer.

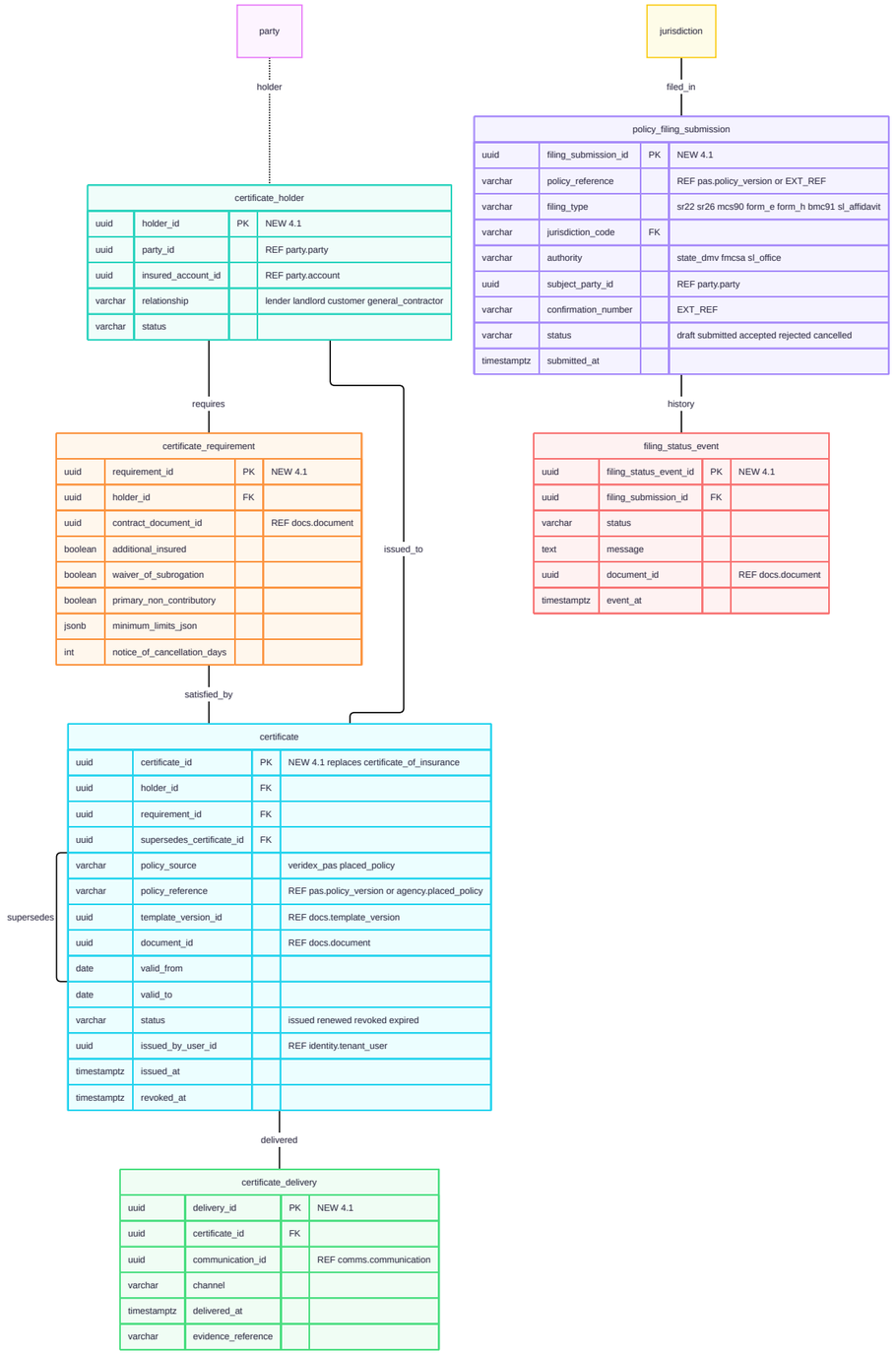


43. Certificates & proof-of-coverage filings

schema: certificate **store: Domain SQL** Certificates of insurance for any policy, Veridex-issued or placed with an outside market, with holder requirements, renewal and revocation and delivery evidence; and policy-level filings with authorities (for example SR-22, MCS-90, Form E or surplus lines affidavits), separate from product rate and form filings.

v4.1 Entire module (certificate schema), sold with Policy Administration and the Broker / Agency Portal. `certificate_holder` + `certificate_requirement`: who needs proof of cover and on what terms. `certificate` replaces the agency-only `certificate_of_insurance` and works for Veridex PAS policies and externally placed policies, with renewal through `supersedes_certificate_id`, revocation and delivery evidence.

v4.1 `policy_filing_submission` + `filing_status_event`: proof-of-coverage filings with authorities (SR-22, MCS-90, Form E, surplus lines affidavits) and their history, separate from product rate and form filings in `regulatory_filing`.



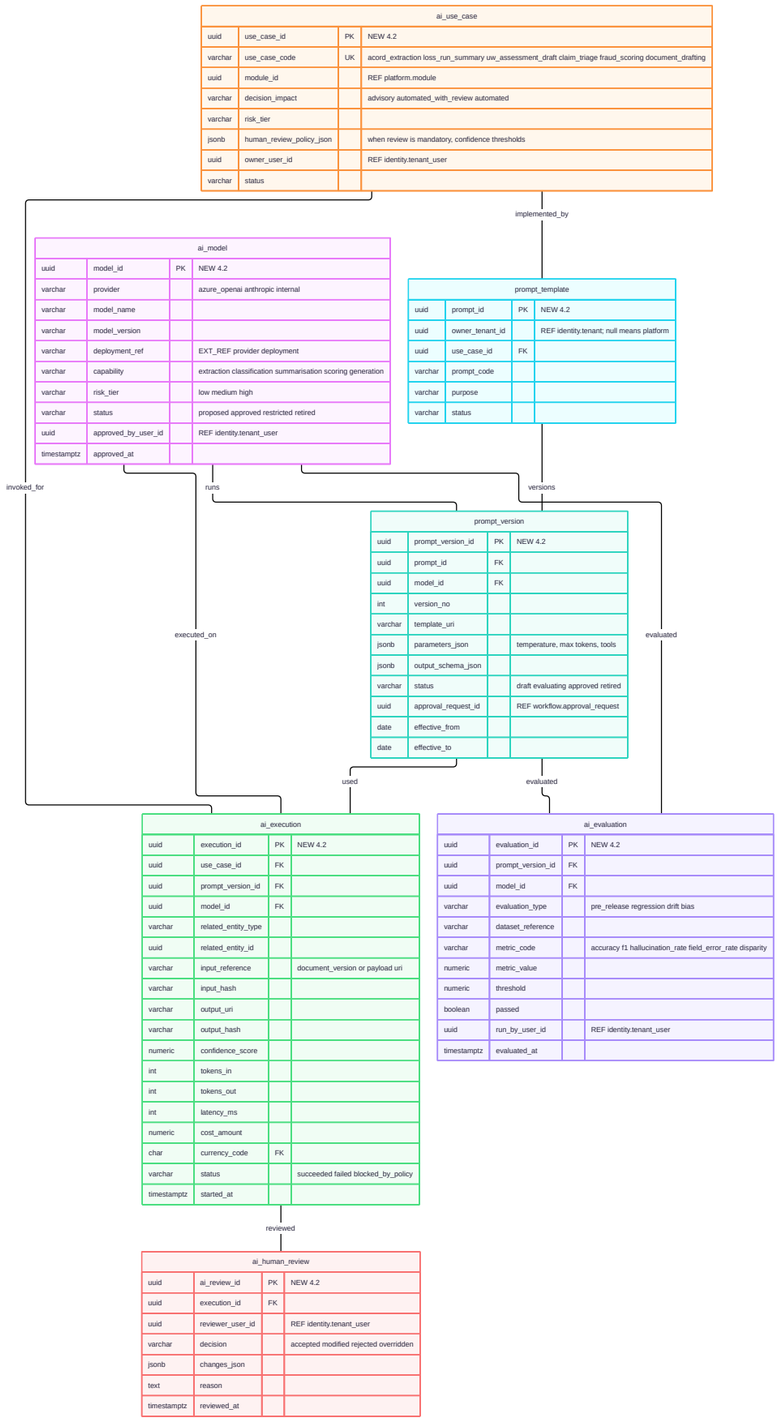
44. AI governance

schema: ai **store: Platform SQL** Every AI-assisted step on the platform is governed: approved models, registered use cases with their human-review policy, versioned prompts, every execution with its inputs, outputs, confidence, tokens and cost, evaluations before and after release, and human review or override of AI output. Together these are the audit evidence for any AI-influenced decision.

v4.2 Entire module (ai schema, part of Foundation). `ai_model` and `ai_use_case`: only approved models, and only registered use cases, each with a decision-impact level and a human-review policy.

v4.2 `prompt_template` + `prompt_version`: versioned prompts bound to a model, with parameters, output schema and approval; `ai_evaluation`: pre-release, regression, drift and bias evaluations with thresholds.

v4.2 `ai_execution`: every call with input and output hashes, confidence, tokens, latency and cost (partitioned by month); `ai_human_review`: acceptance, modification, rejection or override with reason. `data_observation.ai_execution_id` links AI-extracted values to the execution that produced them.



45. Reporting & regulatory reporting

schema: reporting **store: Platform SQL** Configurable reports with versions, schedules, recipients, runs and delivery; regulatory report submissions with their status history; and the boundary to the analytical warehouse or lakehouse, fed from module events rather than direct reads of module tables.

v4.2 Entire module (reporting schema, sold with Insight 360). `report_definition` → `report_version` → `report_schedule` + `report_recipient` → `report_run` → `report_delivery`: configurable reports without code, delivered through Communication.

v4.2 `regulatory_report_submission` + `regulatory_submission_event`: statistical, market-conduct and similar submissions to regulators, with confirmation and history; distinct from product filings and proof-of-coverage filings.

v4.2 `warehouse_feed`: the analytical boundary. Each module publishes events under a contract; feeds load them into the lakehouse with a watermark. Analytics never read module tables directly, and heavy reporting runs on the lakehouse or Insight projections, not on transactional schemas.

